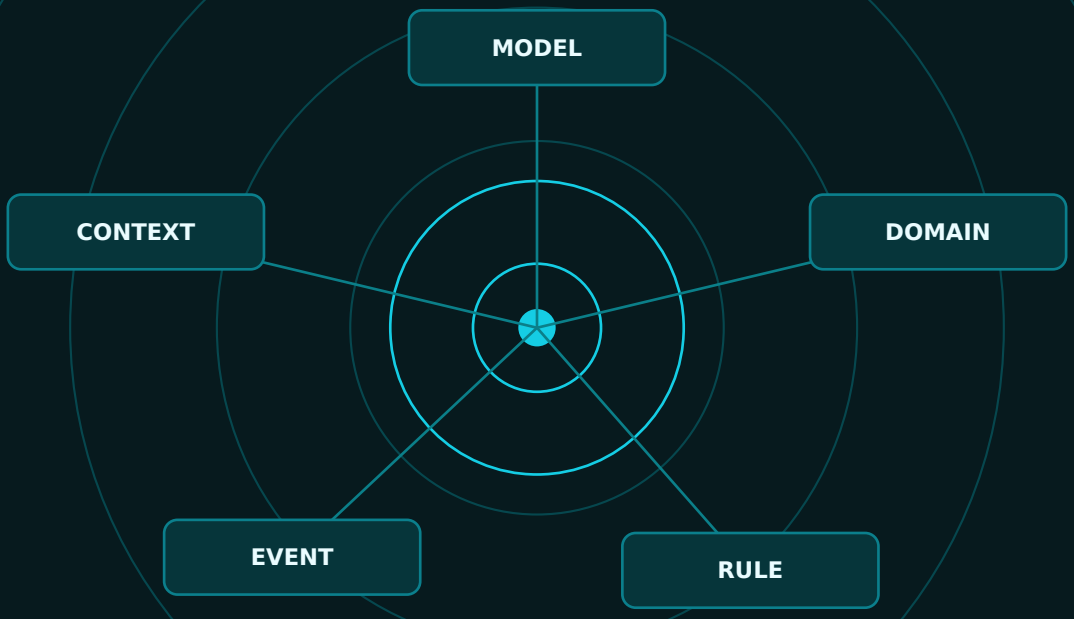
NEXUS-1 - DOMAIN ARCHITECTURE

FROM DOMAIN TO TWIN

Domain-Driven Design from Zero
Explained Through the NEXUS-1 Digital Twin



SCHOOL-STYLE DDD - NEXUS-1 DOMAIN MODEL - C# / SQL / EF CORE

GRIGORIOS AGATHANGELIDIS

A NEXUS-1 Companion - final edition

From Domain to Twin

Domain-Driven Design from Zero - Explained Through the NEXUS-1 Digital Twin

STANDING BOUNDARY

This volume is a software architecture and learning companion. It is not safety-class software, not certified operational guidance, and not connected to any real facility. NEXUS-1 remains a Phase-0 demonstrator: useful for learning architecture, language, modelling, and boundaries - never for operating a plant.

Final edition. The page-aware contents, sources and reference notes, glossary, and index have been added. The cover, body layout, diagrams, and back matter have been rendered and checked as a unified PDF.

STANDING RULE

Nothing in this book treats Domain-Driven Design as magic. DDD helps only when names are honest, boundaries are visible, and rules live in the place that owns them.

Contents

Page-aware contents - final edition

PAGE NUMBER RULE
The page numbers below refer to the visible PDF pages in this final edition. Back matter is included: sources and reference notes, glossary, index, and final checkpoint.

Preface

This book exists for a simple reason: many developers have used entities, services, controllers, DTOs, repositories, and database tables for years, but still feel that Domain-Driven Design belongs to another world. It often arrives with heavy language: aggregate, bounded context, ubiquitous language, anti-corruption layer. The terms are useful, but they can make the subject look more mysterious than it is.

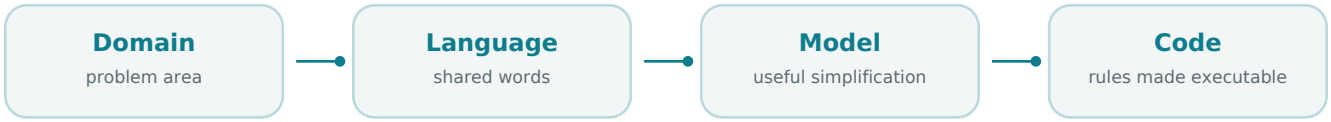
The promise of this book is different. We will start from normal software problems: names that mean different things in different screens, services that contain rules they do not own, tables that know too much, and code that lets any part of the application change any row. Only after the problem is visible will the DDD word be introduced.

NEXUS-1 is a good teaching system because it is not a shopping-cart example. It contains a plant, signals, alarms, causal reasoning, a digital twin, an advisory learning agent, audit trails, compliance records, and many screens that must still describe one coherent system. That makes DDD useful: it gives the system a language, a boundary map, and rules about ownership.

> **FOR THE READER**
> You do not need to know DDD before reading this book. You need ordinary developer experience: C#, SQL tables, EF Core, services, APIs, DTOs, and business rules. We will learn DDD slowly, in human language, with many examples, and then apply each idea to NEXUS-1.

How to Read This Book

Each chapter follows the same teaching rhythm. First, the problem is explained in plain language. Then a simple software example makes the idea familiar. Only after that do we apply the idea to NEXUS-1. When useful, we add a small C# example and a database or EF Core connection.



DDD is a feedback loop: the domain gives language; language shapes the model; the model guides code.

Figure 0.1 - The learning loop of the book. The domain gives the language; the language shapes the model; the model guides the code; the code exposes mistakes in the model.

> **WRITING RULE**
> Never introduce a DDD term before explaining the normal developer problem it solves. The word is not the lesson. The problem is the lesson; the word is the handle we attach to it.

The examples will not pretend that DDD is magic. DDD does not automatically make software better. It helps when the names are honest, the boundaries are clear, and the rules live in the place that owns them.

PART I

Domain-Driven Design from Zero

Part I is the school part of the book. It teaches DDD without assuming DDD knowledge. The language is deliberately simple. The examples begin outside NEXUS-1 and return to NEXUS-1 only after the idea is clear.

WHAT PART I GIVES YOU

By the end of Part I, a mid-level developer should understand domain, model, ubiquitous language, entity, value object, aggregate, repository, domain service, application service, domain event, bounded context, and context map well enough to recognize them in real code.

1. Why Domain-Driven Design Exists

DDD begins with a frustration most developers know. The database has tables. The API has DTOs. The UI has screens. The services have methods. But the business meaning is scattered everywhere. A rule lives in one controller, another rule in a service, another in a stored procedure, and another in a validation attribute. The software works, until it becomes hard to change safely.

Domain-Driven Design is an answer to that problem. It asks the team to model the problem area directly, give important words one controlled meaning inside clear boundaries, and place rules near the concepts that own them.

PLAIN EXPLANATION

DDD is not mainly about fancy class diagrams. It is about making the meaning of the software visible. It asks: what is this system really about, what words do we use, who owns each rule, and where should changes be allowed?

A developer-friendly way to say it is this: DDD helps you avoid building a large application where the database knows one story, the code knows another, the UI knows a third, and nobody can say which one is the truth.

Developer pain	DDD response	NEXUS-1 example
The same word means different things.	Create bounded contexts.	Unit can mean physical unit, engineering unit, or organization unit.
Rules are scattered across controllers and services.	Move rules into the domain model or domain services.	A RootCauseCase protects its evidence and verdict.
Every table is treated like every other table.	Identify aggregate roots and consistency boundaries.	Policy, TwinModel, Unit, and RootCauseCase are not just passive rows.
Outputs are misunderstood as commands.	Name commands, events, and recommendations separately.	An RL recommendation is not an actuator command.

DDD also teaches humility. The model is not the real world. It is the part of the real world the software needs in order to make decisions. NEXUS-1 does not model every bolt, cable, procedure, or human action in a plant. It models the concepts needed by the demonstrator: units, signals, alarms, causal nodes, twin snapshots, policies, recommendations, audit entries, and compliance findings.

COMMON MISTAKE

Do not start a DDD project by creating folders named Domain, Application, and Infrastructure and assuming the work is done. Folders are not the model. The model is the language, rules, boundaries, and behavior those folders contain.

HONEST BOUNDARY

DDD is not required for every application. A small CRUD tool may not need the full discipline. DDD becomes valuable when the domain is rich enough that names, rules, ownership, and consistency boundaries matter.

2. Domain, Model, and Language

The word domain means the problem area your software is about. For a banking system, the domain may involve accounts, payments, limits, fraud checks, and statements. For a hospital system, it may involve patients, appointments, diagnoses, prescriptions, and clinical records. For NEXUS-1, the domain is an industrial digital-twin demonstrator: a software view of plant structure, signals, alarms, causal reasoning, model state, advisory learning, and accountability.

A model is not a complete copy of the domain. It is a useful simplification. A good model leaves things out on purpose. It includes the concepts that help the software make correct decisions and excludes details that do not serve the current purpose.

PLAIN EXPLANATION

The domain is the world you care about. The model is the useful version of that world inside the software. The language is the set of words the team agrees to use when talking about both.

This matters because code follows language. If the team says alarm, event, symptom, cause, evidence, and verdict as if they are interchangeable, the software will eventually confuse them too. DDD asks us to separate those words before they become bugs.

```
// A model is not the whole plant. It is the part this software needs.
public sealed class AlarmEvent
{
    public int AlarmEventId { get; private set; }
    public int UnitId { get; private set; }
    public string SignalTag { get; private set; } = null!;
    public DateTime RaisedAtUtc { get; private set; }
    public AlarmSeverity Severity { get; private set; }

    // Notice: this object records an alarm. It does not decide root cause.
}
```

In this small class, the language already protects the design. An AlarmEvent records that something crossed a configured alarm condition. It does not decide why it happened. That decision belongs to the root-cause context.

NEXUS-1 SENTENCE TO REMEMBER

A Unit is not a string. A Signal is not an alarm. An AlarmEvent is not a cause. A Hypothesis is not a verdict. A TwinModel is not the plant. A Recommendation is not a command.

Term	Human meaning	NEXUS-1 example
Domain	The real problem area.	Industrial digital twin, alarms, root cause, advisory learning.
Model	A useful simplification of the domain.	Unit, Signal, AlarmEvent, RootCauseCase, TwinSnapshot, Policy.
Language	The shared words used by developers and domain thinkers.	Alarm is not cause; recommendation is not command.

HONEST BOUNDARY

The NEXUS-1 model is educational and architectural. It is not a validated nuclear engineering model and not a live operational model. The point here is software meaning: how concepts are named, separated, connected, and protected.

3. Entities: Objects with Identity

An entity is something that remains the same thing over time, even when its data changes. This is the simplest useful definition. A customer can change email. A machine can change status. A unit can change power output. The values change, but the thing is still the same thing.

Most developers already use entities because most database tables have primary keys. But DDD asks a slightly deeper question: does this object have a life of its own in the domain, or is it only a collection of values? If it has identity and a lifecycle, it is probably an entity.

PLAIN EXPLANATION

An entity is not important because it has many properties. It is important because the system must recognize it as the same thing across time. Identity is the point.

A simple business example is Customer. The customer may change address, phone, email, or status, but the same CustomerId tells the system that this is still the same customer. Equality is not based on all current values. Equality is based on identity.

```
public sealed class Customer
{
    public int CustomerId { get; private set; }
    public string Name { get; private set; } = null!;
    public string Email { get; private set; } = null!;

    public void ChangeEmail(string newEmail)
    {
        if (string.IsNullOrWhiteSpace(newEmail))
            throw new InvalidOperationException("Email is required.");

        Email = newEmail;
    }
}
```

The object is not just a bag of setters. It owns a small rule: the email cannot be empty. In richer models, entities protect more important rules, but the principle is the same. An entity should not only hold data; it should protect the meaning of its own change.

Candidate	Entity?	Why
Unit	Yes	It has stable identity and many changing states: power, status, alarms, twin state, and history.
Signal	Yes	It has a stable tag and is referenced by readings, alarms, and twin bindings.
AlarmEvent	Yes	It has lifecycle: raised, acknowledged, cleared, grouped into a flood, used as evidence.
RootCauseCase	Yes	It has an investigation lifecycle: opened, enriched, hypotheses tested, verdict recorded.
Temperature 323 C	No	This is a value. It matters by value and unit, not by identity.

NEXUS-1 EXAMPLE

The same UnitId can appear in ReactorFleet, AlarmManagement, RootCause, DigitalTwin, ReinforcementLearning, Audit, and Reporting. That does not mean all those contexts own the Unit. ReactorFleet owns the physical unit identity. Other contexts reference it.

This distinction is very important. A context may reference an entity owned by another context without becoming the owner of that entity. RootCause may investigate Unit NX1-U1, but RootCause does not own the definition of NX1-U1. It borrows the identity through a passport such as UnitId.

```
public sealed class Unit
{
    public int UnitId { get; private set; }
    public string Code { get; private set; } = null!; // NX1-U1
    public string Name { get; private set; } = null!;
    public UnitStatus Status { get; private set; }

    public void MarkOnline()
    {
        if (Status == UnitStatus.Decommissioned)
            throw new InvalidOperationException(
                "A decommissioned unit cannot be marked online.");

        Status = UnitStatus.Online;
    }
}
```

The example is intentionally small. The important point is that the entity owns a rule about its own lifecycle. The rule is not hidden in a controller. The controller may request the action, but the entity protects the meaning of the action.

COMMON MISTAKE

A common mistake is to call every EF Core class an entity in the DDD sense. EF Core uses the word entity for mapped classes. DDD uses the word entity for a domain concept with identity. Many mapped classes are only lookup rows, history rows, or value-like records. Do not confuse ORM vocabulary with domain vocabulary.

HONEST BOUNDARY

Not every NEXUS-1 table deserves rich entity behavior. Some tables are reference data, audit shadows, read models, or generated lookup tables. DDD does not require making every row into a behavioral object. It requires knowing which concepts own important rules.

4. Value Objects: Meaning Without Identity

A value object is something described completely by its values. It does not need its own identity. Two values with the same content are considered the same value. This is why value objects are often small, immutable, and easy to compare.

The easiest example is money. If two values are both 100 EUR, we usually do not care which object instance produced them. We care about the amount and the currency. The same idea appears everywhere in engineering systems: 155 bar, 323 C, 900 MWe, 0.66 confidence, a time window from 17:00 to 17:05.

PLAIN EXPLANATION

An entity answers: which thing is this? A value object answers: what value is this? Identity belongs to entities. Meaning by content belongs to value objects.

```
public readonly record struct Money(decimal Amount, string Currency);

public readonly record struct EngineeringValue(
    decimal Value,
    string UnitSymbol);

var pressure = new EngineeringValue(155m, "bar");
var hotLeg = new EngineeringValue(323m, "C");
```

The record struct example is simple, but the design idea is serious. By wrapping a decimal with a unit symbol, the code stops pretending that every number is the same kind of number. A pressure, a temperature, a power value, and a confidence score should not all be anonymous decimals floating through the system.

Value object	What it protects	NEXUS-1 use
EngineeringValue	Number plus engineering unit.	323 C, 155 bar, 900 MWe.
SignalRange	Allowed lower and upper limits.	Normal operating range for a signal.
ConfidenceScore	A probability-like value between 0 and 1.	Grounding confidence or causal confidence.
DelayWindow	A minimum and maximum propagation time.	Causal edge timing in root-cause reasoning.
TagName	A validated signal tag string.	FT-7, FV-104, NX1-U1.PWR.

NEXUS-1 EXAMPLE

A TwinDivergence can use value objects to say exactly what disagrees: the modelled value, the measured value, the unit, the difference, and the timestamp. The divergence itself may be an entity or event, but the measured quantities inside it are values.

```
public readonly record struct ConfidenceScore
{
    public decimal Value { get; }

    public ConfidenceScore(decimal value)
    {
        if (value < 0m || value > 1m)
            throw new ArgumentOutOfRangeException(nameof(value),
                "Confidence must be between 0 and 1.");

        Value = value;
    }
}

public readonly record struct DelayWindow(
    TimeSpan Minimum,
    TimeSpan Maximum)
{
    public bool Contains(TimeSpan observedDelay) =>
        observedDelay >= Minimum && observedDelay <= Maximum;
}
```

This is where value objects become more than style. They prevent invalid values from travelling through the system. A ConfidenceScore cannot be 1.7. A DelayWindow knows how to test a delay. A TagName could validate its format. The value carries meaning with it.

In EF Core, value objects are often implemented as owned types or converted columns. The implementation choice comes later in the book. For now, the domain lesson is enough: if the concept has no independent lifecycle and is meaningful by its content, it is a strong candidate for a value object.

Question	Entity answer	Value object answer
Does it need identity?	Yes.	No.
Can its properties change over time?	Often yes.	Usually replace with a new value.
How do we compare it?	By Id.	By all values.
NEXUS-1 example	Unit, Signal, RootCauseCase.	EngineeringValue, DelayWindow, ConfidenceScore.

COMMON MISTAKE

A common mistake is to create an Id for every small concept just because it will be stored somewhere. Storage is not the same as identity. If the domain does not need to track the thing as the same thing over time, an Id may be accidental complexity.

HONEST BOUNDARY

Value objects are not always convenient in every ORM mapping. Sometimes a simple column is enough, and sometimes an owned type is appropriate. The domain decision comes first: what is this concept? The persistence decision comes second: how should this concept be stored cleanly?

5. Aggregates and Aggregate Roots

Entities and value objects explain what individual concepts are. Aggregates explain how a small group of concepts stays correct together. This is one of the most useful DDD ideas for real projects, because many bugs come from changing one row while forgetting the rule that belongs to the whole group.

Think about an Order and its OrderLines. If outside code can insert lines, remove lines, change quantities, and mark the order as submitted from anywhere, the order can easily become invalid. DDD answers by putting a boundary around the small group that must be consistent. One object is the entry point. That object is the aggregate root.

PLAIN EXPLANATION

An aggregate is a small group of objects that must stay correct together. The aggregate root is the object outside code talks to when it wants to change the group. The root protects the rules.

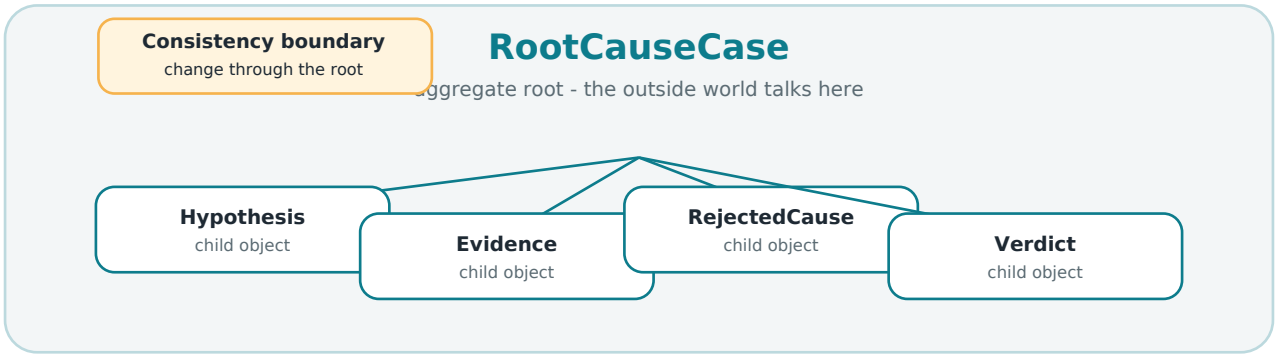


Figure 5.1 - A NEXUS-1 aggregate example. The RootCauseCase is the root. It protects hypotheses, evidence, rejected causes, and the final verdict.

The drawing is not the important part. The important part is the rule: outside code should not freely edit the children. If the case is responsible for the verdict, then the case should decide when evidence is enough, when a hypothesis can be rejected, and when the case can be closed.

```
public sealed class RootCauseCase
{
    private readonly List<Evidence> _evidence = new();
    private readonly List<Hypothesis> _hypotheses = new();

    public IReadOnlyCollection<Evidence> Evidence => _evidence;
    public IReadOnlyCollection<Hypothesis> Hypotheses => _hypotheses;
    public string? FinalVerdict { get; private set; }

    public void AddEvidence(Evidence evidence)
    {
        if (FinalVerdict is not null)
            throw new InvalidOperationException("A closed case cannot change.");

        _evidence.Add(evidence);
    }

    public void CloseWithVerdict(string verdict)
    {
        if (!_evidence.Any())
            throw new InvalidOperationException(
                "A root-cause case cannot close without evidence.");

        FinalVerdict = verdict;
    }
}
```

The example is intentionally simple, but the design direction is powerful. The aggregate is not just a table with child tables. It is a consistency boundary. It says: these things change together, and this root controls the change.

Aggregate candidate	Root	Children or protected data	Main rule
Order	Order	OrderLine, totals, status	Cannot submit an empty order.
Root-cause investigation	RootCauseCase	Hypotheses, evidence, rejected candidates, verdict	Cannot close without grounded evidence.
Digital twin model	TwinModel	Signal bindings, snapshots, divergences	A twin must declare what it mirrors.
Learned policy	Policy	Q-table entries, training metadata, allowed actions	A recommendation must stay inside validated bounds.

A common mid-level developer mistake is to think that every parent-child table relationship is an aggregate. It is not. A database relationship says rows are connected. An aggregate says consistency must be protected through one root. Those are related ideas, but not the same idea.

NEXUS-1 EXAMPLE

AlarmEvent and RootCauseCase should not be the same aggregate. An alarm belongs to the alarm-management lifecycle. A root-cause case uses alarms as evidence or symptoms, but the case owns the diagnostic conclusion. This separation protects the idea that an alarm is not automatically a cause.

Aggregate size matters. A huge aggregate is painful because every change wants to load and lock too much data. A tiny aggregate may fail to protect important rules. The practical goal is not to make the model large. The goal is to make the consistency boundary honest.

COMMON MISTAKE

Do not make Unit the aggregate root of the whole plant and put every signal, alarm, event, twin snapshot, root-cause case, and policy inside it. That would be a monster aggregate. Unit is central identity, but many contexts should reference it rather than be owned inside it.

HONEST BOUNDARY

Aggregate design is judgment. There is rarely one perfect answer. In this book we use aggregates as practical consistency boundaries, not as decoration. If a group of objects does not need to be changed together to stay correct, it may not need to be one aggregate.

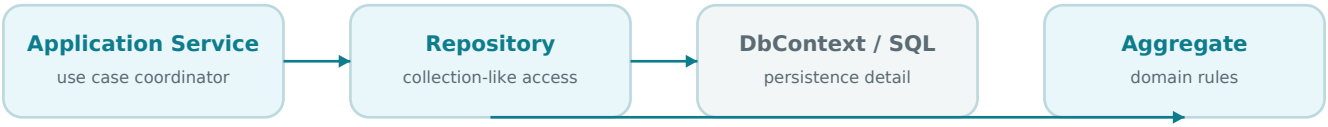
6. Repositories and Persistence Ignorance

Once you have aggregates, you need a way to load and save them. Many developers reach immediately for DbContext everywhere. That can work in small applications, but in a richer domain it often leaks persistence details into places where the business meaning should be clearer.

A repository is a collection-like object for aggregates. It hides the mechanics of persistence and gives the application a language that matches the domain: get a case, save a case, find an open policy, load a unit by code. The repository is not the business rule owner. The aggregate owns rules. The repository helps retrieve and persist aggregates.

PLAIN EXPLANATION

A repository is like a domain-focused collection. It lets the application load and save aggregates without making every use case think in SQL joins, tracking states, Include chains, or table names.



The repository hides persistence mechanics, but the aggregate still owns the business rules.

Figure 6.1 - Repository position in a DDD-friendly .NET application. The application service coordinates the use case; the repository handles persistence; the aggregate protects rules.

```
public interface IRootCauseCaseRepository
{
    Task<RootCauseCase?> GetByIdAsync(
        RootCauseCaseId id,
        CancellationToken ct);

    Task AddAsync(
        RootCauseCase rootCauseCase,
        CancellationToken ct);

    Task SaveChangesAsync(CancellationToken ct);
}
```

This interface is deliberately small. It speaks in domain language: RootCauseCase. It does not expose table names, EF Core entity states, connection strings, or SQL commands. The infrastructure layer can implement it with EF Core, but the application use case does not need to know the details.

Persistence ignorance does not mean pretending that the database does not exist. It means the domain model should not be shaped only by storage mechanics. The model should express the business meaning first. The mapping can then translate that model into SQL and EF Core configuration.

Here is a small application service using a repository. Notice the separation of responsibilities. The service coordinates the use case. The repository loads and saves. The aggregate enforces the rule that a case cannot close without evidence.

```
public sealed class CloseRootCauseCaseService
{
    private readonly IRootCauseCaseRepository _cases;

    public CloseRootCauseCaseService(IRootCauseCaseRepository cases)
    {
        _cases = cases;
    }

    public async Task HandleAsync(
        CloseRootCauseCaseCommand command,
        CancellationToken ct)
    {
        var rootCase = await _cases.GetByIdAsync(command.CaseId, ct)
            ?? throw new InvalidOperationException("Root-cause case not found.");

        rootCase.CloseWithVerdict(command.Verdict);

        await _cases.SaveChangesAsync(ct);
    }
}
```

The service does not inspect child evidence rows directly to decide if closure is allowed. It asks the aggregate to close. If the aggregate refuses, the use case fails safely. This is the difference between orchestration and rule ownership.

Concern	Good home	Why
Business rule	Aggregate or domain service	The concept that owns the rule should protect it.
Use-case flow	Application service	Coordinates command, repository, transaction, and integration.
SQL mapping	Infrastructure / EF Core configuration	Persistence detail should be explicit but not dominate the domain model.
Dashboard query	Read model or query service	A read screen does not always need full aggregate behavior.

NEXUS-1 EXAMPLE

The RootCause repository should not expose a generic UpdateEvidenceRow method to every caller. Evidence changes should go through the RootCauseCase when they affect the diagnostic story. By contrast, a reporting screen may use a read model query because it is not changing the case.

Repositories are also a place where teams can overdo DDD. A repository for every lookup table is usually not useful. Lookup tables, read models, audit histories, telemetry streams, and reporting views may be better served by direct query services or EF Core queries in the infrastructure/application boundary.

COMMON MISTAKE

Do not create repositories as thin wrappers around every DbSet just to follow a pattern. A repository is useful when it protects aggregate loading and saving. If it only repeats DbSet methods with different names, it may add noise instead of clarity.

HONEST BOUNDARY

EF Core already implements Unit of Work and Repository-like behavior through DbContext and DbSet. This book does not deny that. The DDD question is whether exposing DbContext directly in a use case helps or hurts the clarity of the domain. Use repositories where they make the model clearer, not because a diagram says so.

Part I checkpoint

By this point, the first six ideas have a practical shape. We have not finished Part I, but we already have enough vocabulary to describe a real model without hiding behind database tables.

Idea	Plain meaning	NEXUS-1 anchor
Entity	Something with identity over time.	Unit, Signal, RootCauseCase.
Value object	Meaning by value, not identity.	EngineeringValue, DelayWindow.
Aggregate	A consistency boundary with one root.	RootCauseCase protects evidence and verdict.
Repository	A collection-like way to load and save aggregates.	RootCauseCaseRepository hides persistence details.

7. Domain Services and Application Services

Many developers use the word service for almost everything: user service, email service, alarm service, report service, root-cause service. DDD does not forbid that, but it asks a useful question: what kind of service is this, and what responsibility does it really have?

The most important separation is between an application service and a domain service. An application service coordinates a use case. It receives a command, loads aggregates, calls domain behavior, saves changes, and may trigger integration. A domain service contains a domain rule that does not naturally belong inside one entity or aggregate.

PLAIN EXPLANATION

An application service is the conductor. It does not play every instrument; it coordinates the use case. A domain service is a business decision helper when the rule is real domain logic but does not fit naturally inside one object.

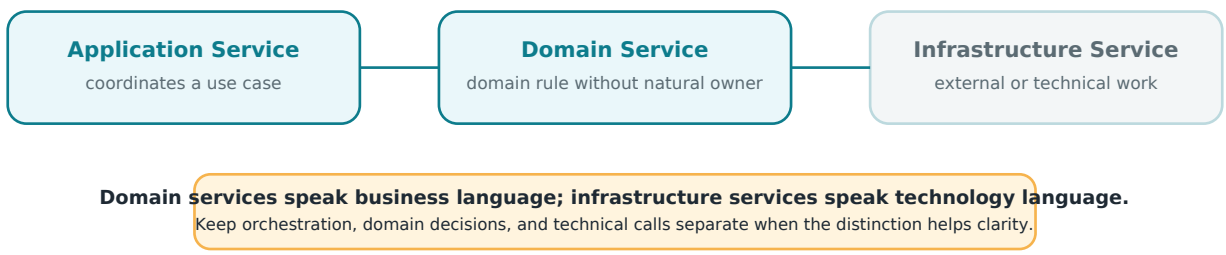


Figure 7.1 - Three service types. The same word service can hide very different responsibilities. DDD becomes useful when those responsibilities are named clearly.

A simple software example is discount calculation. Placing a basic rule inside Order may be fine. But a rule that compares customer history, product category, campaign period, and risk status may not belong naturally inside only Order or only Customer. A domain service can express that decision in business language.

```
public interface IDiscountPolicy
{
    Money CalculateDiscount(
        Customer customer,
        Order order,
        Campaign campaign);
}
```

The interface is not technical. It does not say SQL, HTTP, cache, or EF Core. It says discount policy. That is a domain phrase.

In NEXUS-1, a similar case appears in diagnostics. A RootCauseCase owns its hypotheses and verdict, but scoring a candidate cause may need information from the causal graph, alarm timing, telemetry witness, and evidence rules. That decision may be represented by a domain service such as CausalCandidateScorer.

```
public interface ICausalCandidateScorer
{
    CandidateScore Score(
        RootCauseCase rootCauseCase,
        CausalCandidate candidate,
        AlarmFlood alarmFlood,
        TelemetryWindow telemetryWindow);
}
```

This does not mean the domain service controls persistence or talks directly to the browser. It expresses a domain calculation: how strongly this candidate explains the incident. The application service can coordinate loading the data and saving the result.

Service type	Owns	NEXUS-1 example
Application service	Use-case flow, transaction boundary, calling repositories.	OpenRootCauseCaseService, AcknowledgeAlarmService.
Domain service	Business/domain decision that does not fit one entity.	CausalCandidateScorer, TwinDivergenceClassifier.
Infrastructure service	Technical work and external systems.	EmailSender, FileStorage, TelemetryClient.

COMMON MISTAKE

Do not put all business logic into application services just because they are easy to inject. If the rule is part of the domain, give it a domain home: an aggregate method or a domain service. Otherwise the model becomes anemic and the real system lives in procedural scripts.

HONEST BOUNDARY

Not every method needs a special DDD name. Small CRUD-like features can remain simple. DDD is most useful where the language, rules, and consistency matter. In NEXUS-1, diagnostics, digital-twin divergence, recommendations, and audit-worthy decisions deserve more care than a simple lookup edit.

8. Domain Events

A domain event is a fact that happened in the domain. It is not a request. It is not a command. It does not say please do this. It says this happened, and the rest of the system may care.

PLAIN EXPLANATION

A command asks for a change. A domain event records that a change has already happened. The difference is small in wording and huge in architecture.

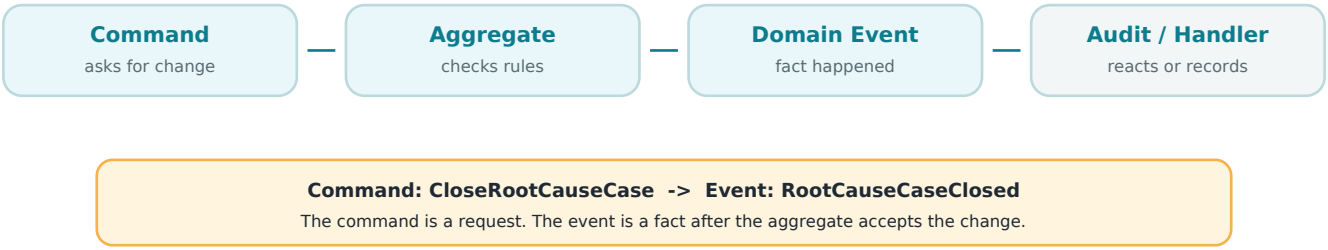


Figure 8.1 - Command versus event. The command may be rejected. The event exists only after the aggregate accepts the change.

In a normal business system, examples are OrderPlaced, PaymentReceived, ShipmentDispatched, PasswordChanged. The names are past tense because they are facts. In NEXUS-1, the same idea becomes very useful because many facts must be traceable.

Command	Domain event	Meaning
AcknowledgeAlarm	AlarmAcknowledged	A user accepted responsibility for seeing the alarm.
OpenRootCauseCase	RootCauseCaseOpened	A diagnostic case now exists.
RejectHypothesis	HypothesisRejected	A candidate explanation was tested and rejected.
RecordTwinDivergence	TwinDivergenceRecorded	The model and measurement disagreed at a point.
GenerateRecommendation	PolicyRecommendationGenerated	An advisory recommendation was produced.

The event should contain enough information to be useful, but it should not become a giant copy of the whole database. A good event says what happened, when, to which identity, and sometimes why. The exact amount depends on whether the event is used only inside the application, written to audit, or sent to another bounded context.

```
public sealed record HypothesisRejected(
    RootCauseCaseId CaseId,
    HypothesisId HypothesisId,
    string Reason,
    DateTime OccurredAtUtc);
```

Domain events also help separate the core action from secondary reactions. When a hypothesis is rejected, the RootCauseCase should not know every possible consequence. It should record the fact. A handler can write an audit entry, update a read model, or notify another part of the system.

```
public sealed class HypothesisRejectedAuditHandler
{
    private readonly IAuditWriter _audit;

    public Task HandleAsync(HypothesisRejected e, CancellationToken ct)
    {
        return _audit.WriteAsync(
            subject: $"RootCauseCase:{e.CaseId}",
            action: "HypothesisRejected",
            detail: e.Reason,
            occurredAtUtc: e.OccurredAtUtc,
            ct: ct);
    }
}
```

This style fits NEXUS-1 because auditability is not an afterthought. A rejected candidate is part of the diagnostic story. A policy recommendation is not a command. A twin divergence is not automatically a failure, but it is a fact worth recording.

NEXUS-1 EXAMPLE

PolicyRecommendationGenerated should be a domain event in the advisory context, not a plant-control command. It means the learning policy produced advice. It does not mean the rod moved, and it does not grant control authority.

COMMON MISTAKE

Do not name events in the present tense, such as RejectHypothesisEvent. That sounds like a command. Prefer past-tense facts: HypothesisRejected, AlarmAcknowledged, TwinDivergenceRecorded.

HONEST BOUNDARY

Domain events are not automatically distributed messages. Inside one application they may be simple in-memory records handled after SaveChanges. If they cross process or context boundaries, they need stronger rules: durability, ordering, retries, idempotency, and versioning. This book introduces the idea first, then applies it carefully to NEXUS-1.

Part I checkpoint after Chapter 8

The school-style foundation now has eight core ideas. The reader can distinguish identity from value, aggregate roots from child objects, repositories from DbContext, application services from domain services, and commands from domain events.

Idea	Plain meaning	NEXUS-1 anchor
Entity	Something with identity over time.	Unit, Signal, RootCauseCase.
Value object	Meaning by value, not identity.	EngineeringValue, DelayWindow.
Aggregate	A consistency boundary with one root.	RootCauseCase protects evidence and verdict.
Repository	A collection-like way to load and save aggregates.	RootCauseCaseRepository hides persistence details.
Application service	Coordinates a use case.	CloseRootCauseCaseService.
Domain service	Domain decision without one natural entity owner.	CausalCandidateScorer.
Domain event	A fact that happened.	HypothesisRejected, TwinDivergenceRecorded.

9. Bounded Contexts

By now the reader has seen entities, value objects, aggregates, repositories, services, and events. These are tactical ideas: they help us design the inside of a model. Bounded contexts are strategic. They help us decide where one model stops and another model begins.

The easiest way to understand a bounded context is not to start with a diagram. Start with a word that means too many things. In ordinary software, the same word often appears in different parts of the system with different meanings. When we ignore that, the code becomes confusing. DDD gives us a boundary around each meaning.

PLAIN EXPLANATION

A bounded context is a part of the system where words have one clear meaning and rules belong to one owner. Inside the boundary, the language is controlled. Outside the boundary, the same word may mean something else.

Word	Meaning in one context	Meaning in another context
Unit	A physical generating unit in ReactorFleet.	A measurement unit such as MW, bar, or C in CorePlatform.
Event	An alarm event in AlarmManagement.	A domain fact in RootCause or Audit.
Policy	A learned advisory policy in ReinforcementLearning.	A security or compliance policy in Security/Compliance.
Case	A diagnostic root-cause case.	A legal, compliance, or workflow case in another system.

This is why bounded contexts matter. They protect meaning. Without the boundary, developers start asking unclear questions: Which Unit do you mean? Which Policy? Which Event? The answer should be visible from the context.

In a C# solution, a bounded context may appear as a namespace, a folder, a project, or a module. In SQL Server, it may appear as a schema. In NEXUS-1, the schema approach is especially useful because each sector can have its own tables and vocabulary while still connecting to the others through explicit passports such as UnitId or SignalId.

NEXUS-1 EXAMPLE

ReactorFleet.Unit and CorePlatform.EngineeringUnit should not be merged just because they both contain the word unit. One is a physical machine. The other is a measurement concept. The boundary prevents a naming collision from becoming a design error.

A bounded context does not mean isolation. Contexts must talk to each other. The point is that they talk through explicit relationships, contracts, events, or identifiers. They do not silently share every table or every meaning.

```
namespace Nexus.Domain.ReactorFleet;

public sealed class Unit
{
    public UnitId UnitId { get; private set; }
    public string Code { get; private set; } = null!;
    public string Name { get; private set; } = null!;
}

namespace Nexus.Domain.CorePlatform;

public sealed class EngineeringUnit
{
    public EngineeringUnitId EngineeringUnitId { get; private set; }
    public string Symbol { get; private set; } = null!; // MW, bar, C
    public string QuantityKind { get; private set; } = null!;
}
```

The namespaces already teach the lesson. A Unit in ReactorFleet is not the same thing as an EngineeringUnit in CorePlatform. The code makes the language explicit before the database even appears.

Now connect this idea to the database. If ReactorFleet owns the physical Unit, other contexts should reference it rather than redefining it. AlarmManagement can record UnitId on an alarm. RootCause can record UnitId on a case. ReinforcementLearning can store a policy for a Unit. But the physical identity of the unit still belongs to ReactorFleet.

Context	Owns	May reference
ReactorFleet	Plant, Unit, physical components.	Engineering units, audit metadata.
Instrumentation	Signal, tag, reading definitions.	Unit, component, engineering unit.
AlarmManagement	Alarm definitions, alarm events, alarm lifecycle.	Signal, Unit.
RootCause	RootCauseCase, hypothesis, evidence, verdict.	AlarmEvent, Signal, Unit.
DigitalTwin	TwinModel, binding, snapshot, divergence.	Unit, Signal.
ReinforcementLearning	TrainingRun, Policy, QTableEntry, Recommendation.	Unit, twin state, audit entries.

COMMON MISTAKE

A common mistake is to treat a bounded context as just a folder. A folder can help, but the real boundary is semantic: who owns the language and rules? If the code is in separate folders but everyone still changes everyone else concepts freely, the boundary is only cosmetic.

HONEST BOUNDARY

A bounded context is not automatically a microservice. Sometimes one bounded context becomes one service. Sometimes several contexts live in one application. Sometimes one context spans more than one technical component. The DDD boundary is about meaning first. Deployment is a later decision.

10. Context Maps and Integration Boundaries

A bounded context tells us where one model stops. A context map tells us how the models relate. This is where strategic DDD becomes very practical. It prevents a large system from becoming a fog of accidental dependencies.

Most real systems are not one clean model. They are a set of models that must cooperate. One context may provide the identity of a Unit. Another may record alarms for that Unit. Another may use those alarms as evidence. Another may show a dashboard. The context map says who depends on whom and how meaning crosses the boundary.

PLAIN EXPLANATION

A context map is a map of relationships between bounded contexts. It shows which context owns a concept, which context consumes it, and what kind of translation or contract is needed at the boundary.

ReactorFleet owns Unit identity	Instrumentation owns Signal and Tag	AlarmManagement owns AlarmEvent lifecycle
DigitalTwin binds model to signals	RootCause tests hypotheses with evidence	ReinforcementLearning owns advisory policy
Audit records facts and changes	Compliance interprets evidence against rules	Reporting read models and exports

Figure 10.1 - A simplified NEXUS-1 context map. Each box owns its own words. The arrows and passports are added as the chapters refine the model.

One important phrase in the NEXUS-1 series is passport. A passport is a controlled way for one context to point at something owned by another context. In the database this often appears as a foreign key. In code it may appear as a strongly typed Id. In an API it may appear as a stable resource identifier.

Passport	Owned by	Used by	Meaning
UnitId	ReactorFleet	AlarmManagement, DigitalTwin, RootCause, ReinforcementLearning	This fact belongs to this physical unit.
SignalId / Tag	Instrumentation	AlarmManagement, DigitalTwin, RootCause	This reading or alarm refers to this measured signal.
AlarmEventId	AlarmManagement	RootCause, Audit, Reporting	This diagnostic evidence came from this alarm event.
RootCauseCaseId	RootCause	Audit, Compliance, Reporting	This audit or report item belongs to this diagnostic case.

Integration boundaries matter because each context has its own truth. AlarmManagement can say that an alarm happened. It cannot say that the alarm was the root cause. RootCause can use the alarm as evidence, but it owns the diagnostic conclusion. ReinforcementLearning can generate advice, but it does not own plant control authority.

NEXUS-1 PRINCIPLE

An AlarmEvent is not a RootCause. A Recommendation is not a Command. A TwinDivergence is not automatically a Failure. A context map helps protect these distinctions because it makes ownership visible.

```
public readonly record struct UnitId(int Value);
public readonly record struct SignalId(int Value);
public readonly record struct AlarmEventId(int Value);

public sealed record AlarmRaised(
    AlarmEventId AlarmEventId,
    UnitId UnitId,
    SignalId SignalId,
    DateTime RaisedAtUtc);
```

Strongly typed identifiers are a small coding technique, but they support a large architectural idea. They make it harder to pass a SignalId where a UnitId is expected. They also make the passport visible in code review.

Context maps also help decide where translation is needed. Sometimes one context can use the identifier of another context directly. Sometimes it needs an anti-corruption layer - a translation layer that prevents a foreign model from leaking into the local model. For this introductory book, the important idea is simple: do not let one context silently redefine another contexts language.

Relationship type	Human explanation	NEXUS-1 example
Direct passport	One context references another by a stable Id.	AlarmEvent stores UnitId.
Published language	One context publishes terms others are allowed to use.	Instrumentation publishes Signal and Tag definitions.
Customer/supplier	One context depends on another context output.	DigitalTwin depends on Instrumentation signals.
Anti-corruption layer	Translation protects local meaning.	External historian data translated into NEXUS-1 SignalReading.
Shared kernel	A very small shared model used by multiple contexts.	Strongly typed Ids or common audit metadata.

COMMON MISTAKE

Do not create a context map only after the system is already tangled. The earlier you draw ownership and relationships, the easier it is to avoid hidden coupling. Even a simple map is better than no map.

HONEST BOUNDARY

Context maps are not final diagrams carved in stone. They change as the model becomes clearer. The value is not artistic perfection; the value is making dependencies and ownership discussable before they become production accidents.

Part I checkpoint after Chapter 10

Part I is now complete. The reader has a school-style foundation in the main DDD ideas and can move into NEXUS-1 with enough vocabulary to understand the domain model rather than only the screens or tables.

DDD idea	Human meaning	NEXUS-1 anchor
Domain	The problem area the software is about.	Industrial digital-twin demonstrator.
Model	A useful simplification of that problem.	Units, signals, alarms, cases, twins, policies.
Entity	Something with identity over time.	Unit, Signal, RootCauseCase.
Value object	Meaning by value, not identity.	EngineeringValue, DelayWindow.
Aggregate	A consistency boundary with one root.	RootCauseCase protects evidence and verdict.
Repository	A collection-like way to load and save aggregates.	RootCauseCaseRepository.
Service	A place for coordination or domain logic that does not fit an entity.	CloseRootCauseCaseService, CausalCandidateScorer.
Domain event	A fact that happened.	HypothesisRejected, TwinDivergenceRecorded.
Bounded context	A boundary where words have one meaning.	ReactorFleet, Instrumentation, RootCause.
Context map	A map of context relationships.	Passports between Unit, Signal, Alarm, Case, Policy.

TRANSITION TO PART II

Part II now changes the question. We stop asking, What is DDD? and start asking, What is NEXUS-1 as a domain model? The language, subdomains, contexts, aggregates, events, recommendations, and passports will be defined with the DDD foundation already in place.

PART II

Reading NEXUS-1 as a Domain Model

Part II applies the DDD ideas to NEXUS-1. It does not reduce the part to ubiquitous language alone. Language matters, but Part II also covers subdomains, core versus supporting domains, bounded contexts, aggregates, domain events, commands, recommendations, passports, and the final context map.

Core Platform units, refs, settings	Reactor Fleet physical plant identity	Instrumentation signals and readings	Digital Twin model/reality binding
Alarm Mgmt alarm lifecycle	Root Cause hypotheses and evidence	RL Advisory policy and advice	Audit / Compliance what changed and proof

Figure II.1 - Preview context map. The exact design will be refined chapter by chapter. The key idea is ownership: each context owns its own words and rules, and crossings are explicit.

PART II FOCUS

Part II answers: What is the NEXUS-1 domain? Which parts are core? Which parts support the core? What are the contexts? What concepts does each context own? Which events matter? Which outputs are only recommendations? How do contexts cross boundaries without smuggling meaning?

11. NEXUS-1 as a Domain, Not Just a Dashboard

A dashboard shows information. A domain model explains what the information means. This difference is the starting point of Part II. NEXUS-1 has many screens - plant fleet, alarms, root cause, digital twin, optimization, audit, compliance - but the domain is not the screens. The domain is the problem area those screens are trying to make understandable.

PLAIN EXPLANATION

A screen is how a user looks at the system. A domain is what the system is about. DDD asks us to look behind the screens and name the real concepts: units, signals, alarms, cases, evidence, models, policies, recommendations, and audit records.

For a mid-level developer, this is a familiar shift. A controller action is not the domain. A DTO is not the domain. A database table is not automatically the domain. They are ways to expose, move, or store the domain. The domain is the set of meanings and rules that survive when the UI changes.

Surface view	Domain view	NEXUS-1 example
Screen	Concepts and rules behind the screen.	The Alarm screen shows AlarmEvents, but the domain distinguishes alarm, flood, evidence, and cause.
DTO	Message crossing a boundary.	A RootCauseCaseDto may display a verdict, but the RootCauseCase owns the rule for closing the case.
Table	Persistence shape.	RootCause.CausalNode stores graph nodes, but the domain asks what a causal node means.
Button	Command or query.	Acknowledge is a command; opening a chart is a query.

NEXUS-1 becomes more interesting when read this way. The Root Cause screen is not only a graph visualization. It represents a diagnostic language: candidate causes, evidence, rejected hypotheses, propagation, and verdict. The Optimization screen is not only a grid. It represents an advisory learning context: state, action, reward, policy, and recommendation. The Digital Twin screen is not only telemetry. It represents the relationship between a model and the measured world.

NEXUS-1 EXAMPLE

When the console shows a Unit selector, a DDD reading asks: Which context owns Unit? Which other contexts may reference it? Is the selected unit a physical identity, a measurement unit, or an organization unit? The screen shows a choice; the domain model protects the meaning of that choice.

The first DDD move is therefore not to draw classes. It is to separate views from meanings. A single concept may appear on many screens, and one screen may combine concepts owned by several contexts.

NEXUS-1 screen or area	Concepts visible on screen	Likely owning contexts
Plant Fleet	Plant, Unit, reactor status, power, availability.	ReactorFleet, CorePlatform.
Instrumentation / Trends	Signal, tag, measurement, engineering value.	Instrumentation, CorePlatform.
Alarm Flood	AlarmEvent, severity, acknowledgement, flood window.	AlarmManagement.
Root Cause Graph	CausalNode, CausalEdge, evidence, hypothesis, verdict.	RootCause, AlarmManagement, Instrumentation.
Digital Twin	TwinModel, SignalBinding, TwinSnapshot, TwinDivergence.	DigitalTwin, ReactorFleet, Instrumentation.
Optimization (RL)	State, action, reward, Q-table entry, policy, recommendation.	ReinforcementLearning, DigitalTwin.
Audit / Compliance	AuditEntry, finding, evidence trail, regulation reference.	Audit, Compliance.

This table shows why NEXUS-1 is a good DDD teaching system. It is not a single CRUD module. It is a connected domain where the same plant identity is reused by alarms, signals, twin models, root-cause cases, and recommendations. Without language and boundaries, this kind of system becomes a large set of tables and screens that accidentally agree. With DDD, the agreement becomes explicit.

COMMON MISTAKE

Do not start DDD by asking, How many tables do I need? Start by asking, What concepts exist, who owns them, and what rules protect them? Tables matter, but if they come first, the database can accidentally become the only model.

HONEST BOUNDARY

NEXUS-1 is still a demonstration system. This chapter reads it as a domain model for learning architecture, not as a certified operational model of a real facility. The DDD value here is clarity of meaning, not operational authority.

12. The NEXUS-1 Ubiquitous Language

Ubiquitous language is the shared language used by developers, domain experts, documentation, tests, code, tables, and screens. The phrase sounds formal, but the idea is simple: when the system says Signal, AlarmEvent, RootCauseCase, Policy, or Recommendation, everyone should know what that word means and what it does not mean.

PLAIN EXPLANATION

A ubiquitous language is a controlled vocabulary. It prevents the team from using the same word for three different things or three different words for the same thing. In code, this becomes class names, method names, table names, event names, and test names.

NEXUS-1 already has a strong vocabulary. The DDD task is to make that vocabulary explicit and to protect the dangerous distinctions. The most important distinctions are often negative: a signal is not an alarm; an alarm is not a cause; evidence is not a verdict; a recommendation is not a command; the twin is not the plant.

Term	Plain meaning in NEXUS-1	Important distinction
Unit	A physical generating unit represented by the plant model.	Not an engineering unit of measure.
Signal	A named measurable value from instrumentation.	Not automatically an alarm.
Tag	A stable identifier used to refer to a signal.	Not merely a display label.
Measurement	A value observed for a signal at a time.	Not a diagnosis.
AlarmEvent	A configured alarm condition that occurred.	Not a root cause.
AlarmFlood	A cluster of related alarm events close in time.	Not proof of multiple independent failures.
Evidence	A grounded fact used in reasoning.	Not opinion and not final verdict.
Hypothesis	A candidate explanation under test.	Not accepted truth.
Verdict	The accepted conclusion of a case.	Not just a text note.
TwinModel	A software model of a physical unit or subsystem.	Not the real plant.
TwinDivergence	A recorded disagreement between model and measurement.	Not automatically a failure.
Policy	A learned mapping from state to action.	Not a safety procedure.
Recommendation	Advisory output proposed by a model or policy.	Not a command.

This table should become one of the reference pages of the book. It gives the reader safe language before the book starts drawing context boundaries and aggregates. It also helps a developer choose names in C#, SQL, and tests.

A useful test for a ubiquitous language is whether the words can appear directly in code without translation. If the domain says RootCauseCase, the code should not call the same thing DiagnosticSession in one layer, IncidentAnalysis in another, and CaseRecord in the database unless there is a very good reason. Every unnecessary synonym weakens the model.

```
// Good: the code speaks the same language as the chapter.
public sealed class RootCauseCase
{
    public RootCauseCaseId Id { get; private set; }
    public IReadOnlyCollection<Evidence> Evidence => _evidence;
    public Verdict? FinalVerdict { get; private set; }

    public void AddEvidence(Evidence evidence) { ... }
    public void CloseWithVerdict(Verdict verdict) { ... }
}

// Less clear: names drift away from the language.
public sealed class DiagnosticSessionRecord
{
    public string ResultText { get; set; } = "";
}
```

The first version is easier to discuss. A developer, tester, or reader can say: the RootCauseCase closes with a Verdict after Evidence exists. The second version forces translation: is ResultText a verdict, a note, a recommendation, or a UI label?

NEXUS-1 SENTENCE

A Unit produces Signals. Signals produce Measurements. Some Measurements satisfy AlarmDefinitions and become AlarmEvents. An AlarmFlood may open a RootCauseCase. The case tests Hypotheses with Evidence and closes with a Verdict. A TwinModel may record TwinDivergence. An RL Policy may produce a Recommendation, but not a Command.

That paragraph is not just prose. It is a compressed domain model. Almost every noun can become a class, value object, event, table, or test fixture. Almost every verb suggests behavior or use-case flow.

COMMON MISTAKE

Do not let technical names replace domain names too early. Names like Record, Data, Info, Manager, Processor, and Handler may be useful in infrastructure, but they should not be the main vocabulary of the domain model.

HONEST BOUNDARY

The language will evolve. Early names are not sacred. DDD does not ask us to guess perfect names on day one. It asks us to notice when a word becomes important and then use it consistently. In NEXUS-1, the language should remain honest about the demonstrator boundary: model, recommendation, and simulation must never be named as if they were real plant authority.

Part II Chapter Plan

Chapter	Question answered	NEXUS-1 emphasis
11. NEXUS-1 as a Domain, Not Just a Dashboard	What problem area does the system model?	From screens to domain meaning.
12. The NEXUS-1 Ubiquitous Language	Which words must be controlled?	Unit, Signal, AlarmEvent, Evidence, Policy, Recommendation.
13. The Main Subdomains of NEXUS-1	What large problem areas exist?	Plant model, diagnostics, twin, learning, accountability.
14. Core, Supporting, and Generic Domains	What is strategically central?	Root cause, digital twin, plant model, advisory learning.
15. Bounded Contexts in the NEXUS-1 Platform	Where does each language stop?	The seventeen schema sectors as context candidates.
16. Aggregates in NEXUS-1	Which objects protect consistency?	Unit, RootCauseCase, TwinModel, Policy.
17. Domain Events in NEXUS-1	Which facts happened?	AlarmRaised, HypothesisRejected, DivergenceRecorded.
18. Commands, Decisions, and Recommendations	What asks, what decides, what only advises?	Recommendation is not command.
19. Context Relationships and Passports	How do contexts cross safely?	UnitId, SignalId, Tag, AlarmEventId as passports.
20. The NEXUS-1 Context Map	How does the full model fit together?	A readable map of ownership and integration.

PART III

Implementing DDD in NEXUS-1

Part III turns the domain model into practical .NET architecture. It connects DDD to project structure, SQL schemas, EF Core mapping, aggregate roots, value objects, application services, repositories, queries, read models, audit entries, and reporting screens.

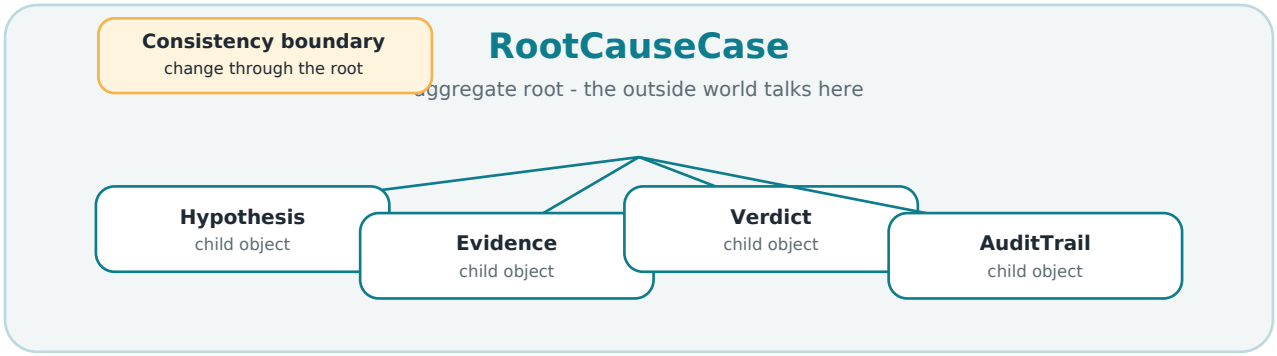


Figure III.1 - Example aggregate. The RootCauseCase owns hypotheses, evidence, and verdict. The important rule is not the drawing; the important rule is controlled change through the root.

PART III FOCUS

Part III answers: How do we implement the model without losing the meaning? Where do rules live? What becomes an entity? What becomes an owned value object? What belongs in EF Core configuration? When do we use read models instead of aggregates?

Part III Chapter Plan

Chapter	Practical result	Developer connection
21. From Bounded Context to Project Structure	Folders and projects mirror boundaries.	Domain, Application, Infrastructure, Web/API.
22. From Bounded Context to SQL Schema	A context can map to a SQL schema.	ReactorFleet.Unit, RootCause.CausalNode.
23. From Aggregate Root to C# Entity	Entities protect behavior and identity.	RootCauseCase, TwinModel, Policy.
24. From Value Object to Owned Type	Small values become safe types.	EngineeringValue, SignalRange, ConfidenceScore.
25. From Domain Rule to Method	Rules move near the concept that owns them.	Close case only with evidence.
26. From Command to Application Service	Use cases coordinate work.	AcknowledgeAlarm, OpenRootCauseCase.
27. From Domain Event to Audit Entry	Facts become traceable records.	HypothesisRejected -> AuditEntry.
28. Repositories, DbContext, and Unit of Work	Persistence supports the model, not the reverse.	EF Core behind repositories where useful.
29. Queries, Read Models, and Reporting Screens	Reads do not need to load full aggregates.	Dashboards, exports, operator views.
30. Final Architecture: Domain, Data, Twin, and Decision	Everything is tied together.	The semantic architecture of NEXUS-1.

NEXT BUILD

The next step should be to expand Chapter 13 and Chapter 14: the main subdomains of NEXUS-1, then core, supporting, and generic domains. The same PDF quality and approved cover should be preserved.

CHAPTER 13

The Main Subdomains of NEXUS-1

A large system is easier to understand when we stop seeing it as one giant object. DDD asks us to split the problem area into subdomains: smaller regions of meaning, each with its own reason to exist.

In ordinary developer language, a subdomain is a part of the business or engineering problem that deserves its own vocabulary. It is not yet a project, a database schema, or a microservice. It is first a region of the problem.

WHAT YOU ALREADY KNOW

You already do this informally. In a web application, login, orders, payments, reporting, and notifications are different problem areas. You may put them in folders, modules, services, or tables. DDD simply asks you to make those boundaries explicit and to ask who owns the rules inside each one.

NEXUS-1 has many screens, but screens are not the real boundary. A screen is a view. A subdomain is a meaning area behind the view. The Plant Fleet screen belongs mostly to the physical-plant subdomain. The Root Cause screen belongs to diagnostic reasoning. The Optimization screen belongs to advisory learning.

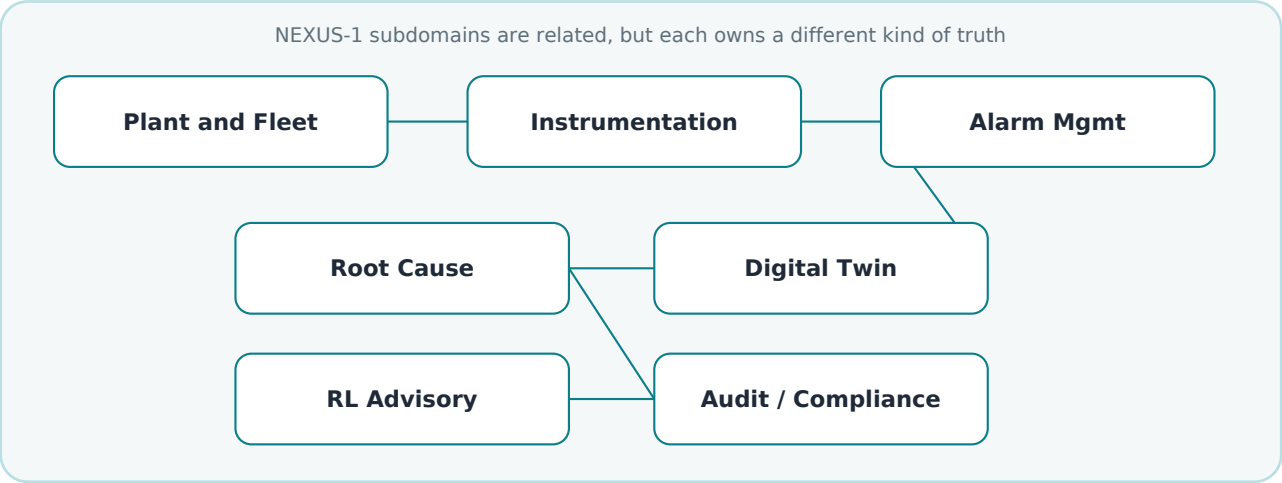


Figure 13.1 - A developer-friendly view of the main NEXUS-1 subdomains. The point is not to freeze the diagram forever, but to show that the platform is not a flat dashboard.

Subdomain	Plain meaning	Typical NEXUS-1 words
Plant and Fleet	What physical things exist and how they are arranged.	Plant, Unit, Reactor, Pump, Valve, Component
Instrumentation	What can be measured and how readings are named.	Signal, Tag, Reading, EngineeringValue

Alarm Management	What abnormal condition was detected and how its lifecycle is handled.	AlarmEvent, AlarmState, Severity, Acknowledgement
Root Cause	What explanation is being tested and what evidence supports it.	RootCauseCase, Hypothesis, Evidence, Verdict
Digital Twin	How the model is bound to measured reality.	TwinModel, SignalBinding, Snapshot, Divergence
RL Advisory	How simulated experience becomes readable advice.	Agent, State, Action, Reward, Policy, Recommendation
Audit / Compliance	What must be remembered, proved, and reviewed.	AuditEntry, Finding, EvidenceTrace, Regulation

A helpful rule is this: if two parts of the system use different nouns, different rules, and different reasons for change, they are probably different subdomains. Instrumentation changes when new tags and readings are needed. Root Cause changes when reasoning and evidence rules change. RL Advisory changes when training, state design, and policy review change.

The subdomain view protects the reader from a common mistake: calling the whole system simply "the digital twin". NEXUS-1 contains a digital twin, but it also contains alarm lifecycle, diagnostic reasoning, advisory learning, audit, compliance, security, and reporting. The digital twin is one important subdomain, not the entire domain.

COMMON MISTAKE

Do not use the menu structure as the domain model. A screen may combine data from many subdomains. A domain model asks who owns the meaning and rules, not where the user clicks.

A small software example

Imagine a maintenance ticket system. One screen may show the machine, the ticket, the technician, the spare parts, and the invoice. That does not mean all those ideas belong to one model. The machine belongs to asset management. The ticket belongs to maintenance. The invoice belongs to accounting. The screen joins them for human convenience.

NEXUS-1 works the same way. A single operator view may show unit power, active alarms, a suspected root cause, and a recommendation. But DDD asks us to keep the meanings separate: power is a measurement, an alarm is a lifecycle event, a hypothesis is a diagnostic candidate, and a recommendation is advisory output.

HONEST BOUNDARY

At this stage, a subdomain map is not an implementation promise. It does not say that every subdomain must become a separate service, database, or deployment. It says only that the concepts deserve separate ownership and clear language.

CHAPTER 14

Core, Supporting, and Generic Domains

DDD does not treat every part of a system as equally important. Some parts are the reason the system exists. Some parts help that reason work. Some parts are necessary but ordinary. This distinction is called strategic design.

For a mid-level developer, the idea is simple: do not spend the same design energy everywhere. Spend the most careful modelling effort where the system is special.

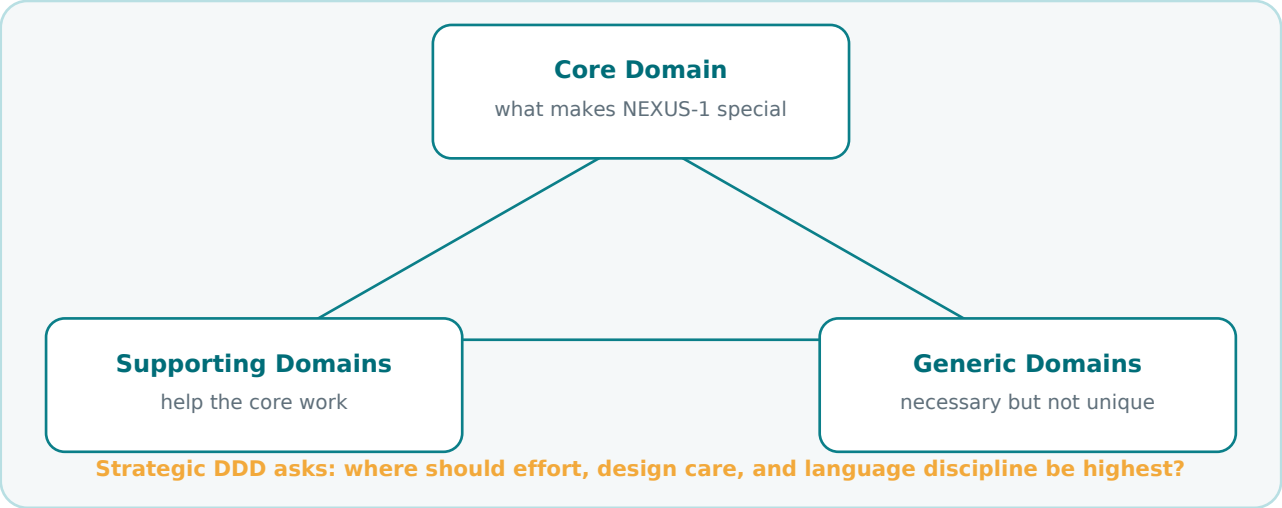


Figure 14.1 - Strategic DDD separates the special heart of the product from supporting and generic capabilities.

In a normal business system, payments or pricing may be core. User login may be generic. Reporting may be supporting. In NEXUS-1, the answer is different because the purpose is different. The special value is the way the demonstrator connects plant structure, live-like signals, causal explanation, twin divergence, and advisory learning into one understandable architecture.

The NEXUS-1 core domain

The core domain is the part that makes NEXUS-1 intellectually distinctive. It is where the most careful language and modelling discipline should go. For this book, the core is not one table and not one screen. It is a cluster of closely related capabilities: plant-aware digital twin modelling, deterministic root-cause reasoning, and interpretable advisory learning.

Core area	Why it is core	Example model concepts
Digital Twin binding	It connects the physical unit to the model and records where they agree or diverge.	TwinModel, SignalBinding, TwinSnapshot, TwinDivergence
Root Cause reasoning	It turns alarm floods into tested explanations instead of loud lists.	RootCauseCase, CausalNode, Hypothesis, Evidence, Verdict

Plant operational model	It gives the rest of the system a shared physical identity.	Unit, Reactor, Component, SystemNode
Advisory learning	It turns simulated experience into readable recommendations, not hidden control.	Policy, QTableEntry, TrainingRun, Recommendation

Calling these areas core does not mean they are certified, complete, or operational. It means they are where the educational and architectural value of NEXUS-1 is concentrated.

```
public sealed class TwinDivergence
{
    public int TwinDivergenceId { get; private set; }
    public int TwinModelId { get; private set; }
    public string SignalTag { get; private set; } = null!;
    public decimal ModelValue { get; private set; }
    public decimal MeasuredValue { get; private set; }
    public DateTime ObservedAtUtc { get; private set; }

    public decimal Difference => MeasuredValue - ModelValue;
}
```

This small class is core-domain language. It does not say "error" too early. It says divergence: the model and the measured world disagree by a certain amount at a certain moment. That word is safer and more honest.

Supporting domains

Supporting domains are important, but they support the core rather than define the special value of the system. In NEXUS-1, instrumentation, alarm management, maintenance, robotics, radiation monitoring, and emergency preparedness can be read this way. They are not unimportant. They are the infrastructure of meaning that lets the core work.

Supporting domain	How it supports the core	Good design question
Instrumentation	Feeds signals and readings to alarms, root cause, and the twin.	Is the tag stable and unambiguous?
Alarm Management	Provides lifecycle events that root cause can reason from.	Is this alarm a symptom, not a conclusion?
Maintenance	Connects components to work orders and asset history.	Does history explain this behaviour?
Robotics	Represents missions and inspections that may produce evidence.	Is the robot output evidence or action?
Emergency Preparedness	Connects scenarios, plans, and exercises to operational readiness.	Which procedure or exercise is being tested?

Generic domains

Generic domains are capabilities many systems need: identity, authorization, audit storage, localization, feature flags, reporting exports, and general settings. They still need quality. They still need tests. But they are not where NEXUS-1 should invent unusual domain language unless there is a clear reason.

Generic domain	Why it is generic	Practical implementation tendency
Security	Most systems need users, roles, permissions, and policies.	Use proven patterns and framework support.
Audit	Most serious systems need change history and traceability.	Standardize it and make it hard to bypass.
Reporting	Dashboards and exports are common across systems.	Prefer read models and projections.
Core Platform	Settings, languages, feature flags, units, and references are common platform needs.	Keep it boring, stable, and explicit.

COMMON MISTAKE

Generic does not mean low quality. It means do not turn ordinary platform plumbing into fake domain complexity. A login system does not become more valuable because every permission is wrapped in dramatic language.

Why this matters for architecture

Strategic design changes where we put effort. The core domain deserves deeper examples, sharper invariants, better diagrams, and more domain tests. Supporting domains deserve clean boundaries and reliable integration. Generic domains deserve proven implementation, stable conventions, and minimal surprise.

HONEST BOUNDARY

This classification is a design judgment, not a law. A future NEXUS-1 version could make maintenance or robotics more central. When the purpose changes, the strategic map can change. DDD does not freeze the system; it gives us a language for discussing the change.

Mini cases for the reader

The following cases are deliberately simple. Their job is to train the reader to classify concepts before jumping into code.

Case	Question	Better DDD answer
A pump has high vibration.	Is this a root cause?	Not by itself. It is a measurement or alarm candidate until the diagnostic context tests it.
The model predicts 323 C but the signal reads 330 C.	Is this a failure?	Not automatically. It is a TwinDivergence first. Other contexts may interpret it later.
The policy suggests +2 pcm.	Is this a command?	No. It is a Recommendation. Command authority is separate and stricter.
A Unit appears in alarms, root cause, and RL tables.	Is Unit owned by all of them?	No. ReactorFleet owns physical Unit identity. Others reference it by passport.
A report combines alarms, recommendations, and audit entries.	Is reporting the owner of those facts?	No. Reporting reads and projects facts owned elsewhere.

PART II PROGRESS

We have now moved from a flat view of NEXUS-1 to a strategic domain view. Chapter 13 separated the main subdomains. Chapter 14 ranked them as core, supporting, or generic. The next chapters can now discuss bounded contexts, aggregates, events, commands, recommendations, and passports with much clearer purpose.

NEXT BUILD

The next continuation should expand Chapter 15 and Chapter 16: bounded contexts in the NEXUS-1 platform, then aggregates in NEXUS-1. The approved cover page and the same PDF style should be preserved.

CHAPTER 15

Bounded Contexts in the NEXUS-1 Platform

A bounded context is the DDD answer to a very ordinary developer problem: the same word does not always mean the same thing everywhere. If we do not draw a boundary, the code slowly becomes a dictionary of accidents.

In NEXUS-1, a bounded context is not only a theory word. It is visible in the names of the system: ReactorFleet, Instrumentation, AlarmManagement, RootCause, DigitalTwin, ReinforcementLearning, Audit, Compliance, and the other sectors. Each sector owns a part of the language and a part of the rules.

WHAT YOU ALREADY KNOW

You already use boundaries when you create namespaces, folders, projects, modules, controllers, services, or database schemas. DDD asks you to make the boundary mean something: inside it, words and rules are controlled. Outside it, other contexts may use different words or different meanings.

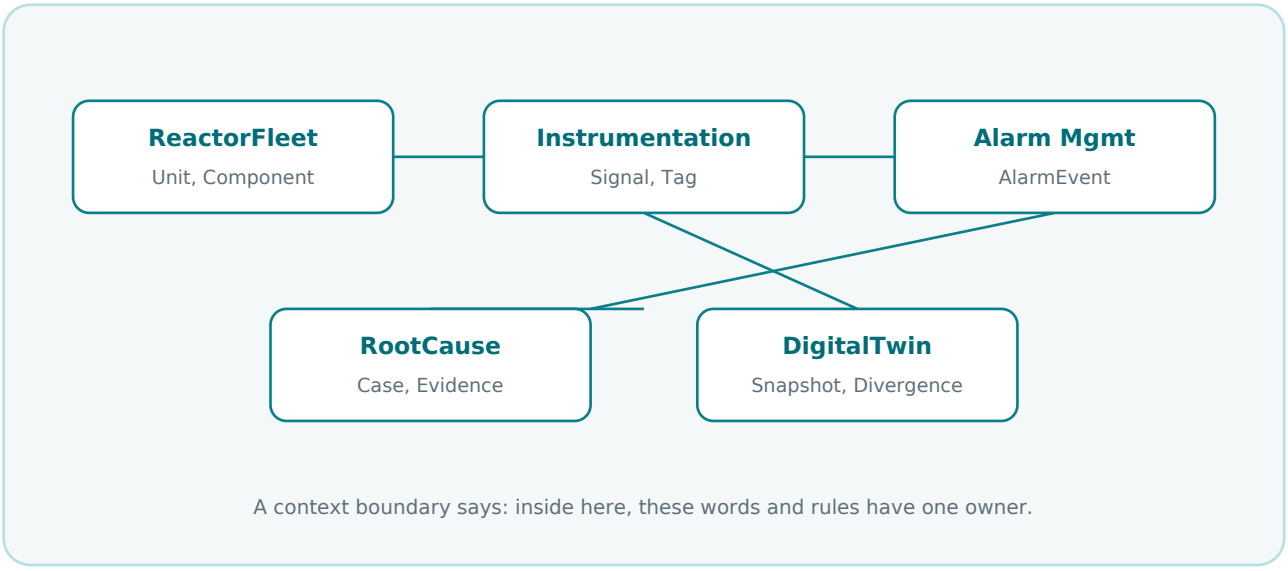


Figure 15.1 - Bounded contexts in NEXUS-1 are not walls. They are meaning boundaries with explicit passports between them.

Context	Owns this language	Must not decide
ReactorFleet	Plant, Unit, Reactor, Component, SystemNode	Whether an alarm is the root cause.
Instrumentation	Signal, Tag, Reading, EngineeringValue	Whether a reading is operationally important.
AlarmManagement	AlarmEvent, Severity, AlarmState, Acknowledgement	The causal origin of the event.

RootCause	Case, Hypothesis, Evidence, RejectedCause, Verdict	The physical identity of a Unit or Signal.
DigitalTwin	TwinModel, Binding, Snapshot, Divergence	Whether an advisory policy may command equipment.
ReinforcementLearning	State, Action, Reward, Policy, Recommendation	Safety authority or protection logic.
Audit	AuditEntry, ChangeRecord, EvidenceTrace	The domain meaning of the thing being audited.

This table is one of the practical benefits of DDD. It helps a developer resist the temptation to put every rule in the nearest service. If AlarmManagement owns AlarmEvent, it can record that something happened. But it should not become the place where root cause is decided. That decision belongs to the diagnostic context.

The Unit example: one word, several meanings

The word Unit is a perfect DDD teaching example. Without a boundary, it is dangerous. With a boundary, it is precise.

Name	Meaning	Example
ReactorFleet.Unit	A physical generating unit in the plant model.	NX1-U1, Unit 1, Unit 2
CorePlatform.EngineeringUnit	A unit of measure used for values.	MW, bar, C, pcm
Organization.Unit	A team or department in the organisation model.	Operations, Maintenance, Security

A mid-level developer may ask: why not just call them different names? Sometimes we do. But real domains often reuse ordinary words. DDD does not panic when this happens. It says: keep each meaning inside the context where it is true.

```
namespace Nexus.Domain.ReactorFleet;
public sealed class Unit
{
    public int UnitId { get; private set; }
    public string Code { get; private set; } = null!;
}

namespace Nexus.Domain.CorePlatform;
public sealed class EngineeringUnit
{
    public int EngineeringUnitId { get; private set; }
    public string Symbol { get; private set; } = null!;
}
```

COMMON MISTAKE

Do not solve every naming conflict by inventing long artificial class names. Sometimes a short domain word is correct inside its own context. The fully qualified name tells the truth: ReactorFleet.Unit is not CorePlatform.EngineeringUnit.

Context passports

A context passport is the explicit reference that lets one context point to another without pretending it owns the other context. In the SQL books, that passport often appears as a foreign key. In code, it may appear as an identifier value, an integration event, or a read model reference.

```
public sealed record UnitId(int Value);

public sealed class AlarmEvent
{
    public int AlarmEventId { get; private set; }
    public UnitId UnitId { get; private set; }
    public string SignalTag { get; private set; } = null!;
}
```

How contexts should talk

A bounded context is not useful if everything reaches through it casually. Contexts should talk through explicit forms: identifiers, events, application services, read models, or carefully designed database relationships. The form depends on the architecture, but the intent is the same: crossing the boundary should be visible.

Communication form	Plain meaning	NEXUS-1 example
Identifier passport	One context stores the identity of something owned elsewhere.	AlarmEvent stores UnitId.
Domain event	One context says that a fact happened.	TwinDivergenceRecorded.
Read model	A query-shaped projection combines facts for display.	Fleet dashboard combines alarms, power, status.
Application service	A use case coordinates several contexts.	OpenRootCauseCaseFromAlarmFlood.
Foreign key	The database enforces a passport relationship.	RootCauseCase references Unit.

HONEST BOUNDARY

A bounded context is not automatically a microservice. In this book, many contexts may live in one database and one application. DDD is first about meaning and ownership. Deployment is a later architectural choice.

Mini case

Suppose the RootCause screen shows Unit 1, fourteen alarms, one rejected sensor candidate, and an advisory recommendation. Which context owns the screen? The answer is: no single context owns the whole screen. The screen is a composition. ReactorFleet owns the Unit identity. AlarmManagement owns the alarm events. RootCause owns the case and verdict. ReinforcementLearning owns the recommendation. The UI assembles them; it should not steal their rules.

CHAPTER 16

Aggregates in NEXUS-1

Aggregates are where DDD becomes practical code. A bounded context tells us where language belongs. An aggregate tells us which small group of objects must stay correct together when the system changes.

The human explanation is simple: an aggregate is a small consistency boundary. One object is the root. Outside code talks to the root. The root protects the children from being changed in ways that break the rules.

WHAT YOU ALREADY KNOW

You have already seen this pattern in ordinary code. An Order protects its OrderLines. A BlogPost protects its Comments. A ShoppingCart protects its Items. You do not usually want random code to insert a child row without respecting the rules of the whole object.

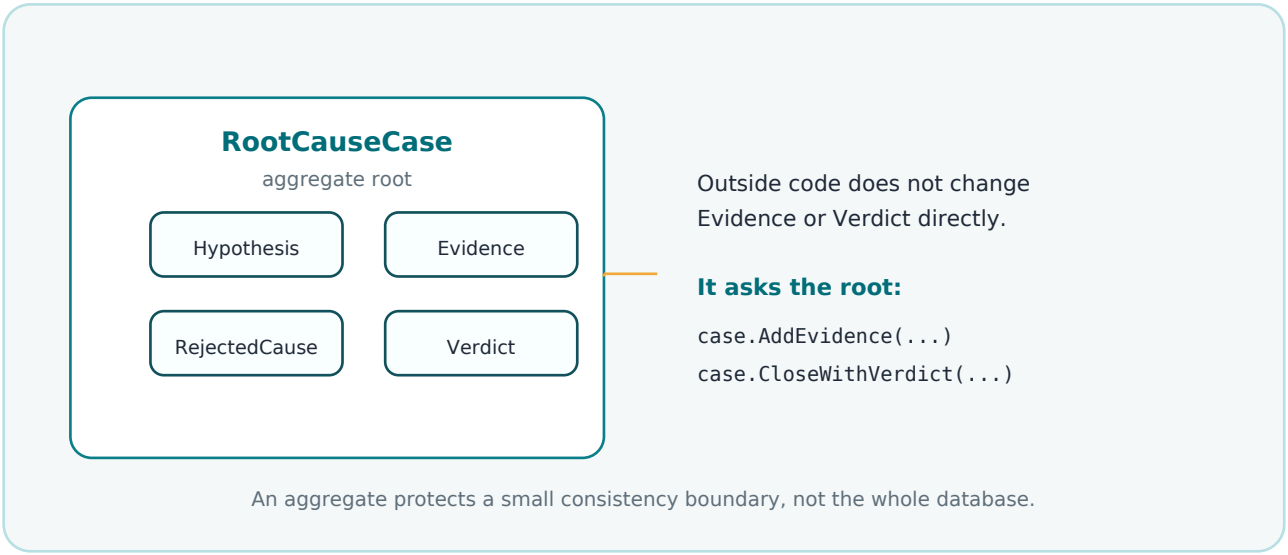


Figure 16.1 - RootCauseCase is a natural aggregate example because evidence, hypotheses, rejected candidates, and verdict must remain consistent.

A good aggregate is not a giant object graph. It is not a whole module. It is not every table with a foreign key. It is the small cluster of objects that must be changed together to protect a rule.

Candidate aggregate root	Owns / protects	Example rule
RootCauseCase	Hypotheses, evidence, rejected candidates, verdict	A case cannot close without evidence.
TwinModel	Signal bindings, snapshots, divergence records	A binding must point to a known signal tag.
TrainingRun	Episodes, reward history, produced policy	A policy cannot be marked ready before training completes.

AlarmFlood	Grouped alarm events and flood metadata	A flood cannot contain alarms from unrelated time windows.
Unit	Physical identity and directly owned plant structure	A reactor belongs to one physical unit.

RootCauseCase as an aggregate

RootCauseCase is the clearest NEXUS-1 aggregate because its children are not independent facts. Evidence, hypotheses, rejected candidates, and the final verdict must tell one consistent story. If they can be changed separately from anywhere, the case becomes untrustworthy.

```
public sealed class RootCauseCase
{
    private readonly List<Evidence> _evidence = new();
    private readonly List<Hypothesis> _hypotheses = new();

    public IReadOnlyCollection<Evidence> Evidence => _evidence;
    public IReadOnlyCollection<Hypothesis> Hypotheses => _hypotheses;
    public string? Verdict { get; private set; }

    public void AddEvidence(Evidence evidence)
    {
        if (Verdict is not null)
            throw new InvalidOperationException("Closed cases cannot be changed.");

        _evidence.Add(evidence);
    }

    public void CloseWithVerdict(string verdict)
    {
        if (!_evidence.Any())
            throw new InvalidOperationException("A verdict requires evidence.");

        Verdict = verdict;
    }
}
```

This example is not trying to be complete production code. Its job is to show where the rule lives. The rule does not live in the controller. It does not live only in SQL. It does not live in a random helper. The rule lives in the aggregate because the aggregate is responsible for the consistency of the case.

> **COMMON MISTAKE**
>
> Do not make every EF entity an aggregate root. If every table has its own repository and every child can be changed directly, you have entities but not aggregates. DDD is asking which object protects the rule.

Aggregate roots and repositories

Usually, repositories load and save aggregate roots, not every child object separately. That does not mean every system must be pure. It means the write path should respect the aggregate boundary. Queries and reports may read projections differently.

```
public interface IRootCauseCaseRepository
{
    Task<RootCauseCase?> GetAsync(int caseId, CancellationToken ct);
    Task AddAsync(RootCauseCase rootCauseCase, CancellationToken ct);
    Task SaveChangesAsync(CancellationToken ct);
}
```

Choosing aggregate boundaries in NEXUS-1

Aggregate design is a judgment. The goal is not to create the largest object. The goal is to protect the rules with the smallest useful boundary.

Question	If yes	If no
Must these objects change together to remain valid?	They may belong in one aggregate.	They may be separate aggregates.
Can a child make sense without the parent?	Maybe separate it, or reference by id.	Keep it inside the parent boundary.
Would loading this aggregate become huge?	Consider smaller aggregate plus domain events.	The boundary may be acceptable.
Is this mostly for reporting?	Use a read model, not a larger aggregate.	Keep rules in the write model.
Does another context own this concept?	Reference it by passport.	It may be owned locally.

For example, a Unit may reference many components, signals, alarms, cases, snapshots, and policies across the wider platform. That does not mean Unit should aggregate all of them. Unit owns physical identity. Other contexts reference it. This is where bounded contexts and aggregates must work together.

HONEST BOUNDARY

This book uses aggregate examples to teach design thinking. It does not claim there is only one correct aggregate model for NEXUS-1. A production system would refine aggregate boundaries through use cases, transaction needs, performance, testing, and team ownership.

NEXT BUILD

The next continuation should expand Chapter 17 and Chapter 18: Domain Events in NEXUS-1, then Commands, Decisions, and Recommendations. The corrected full-size cover page and the same PDF style should be preserved.

CHAPTER 17

Domain Events in NEXUS-1

A domain event is a fact that happened inside the domain. It is not a request. It is not a plan. It is not a maybe. It is the system saying: this thing has already occurred.

For a mid-level developer, the easiest way to remember it is this: commands are written in the imperative mood, events are written in the past tense. AcknowledgeAlarm is a command. AlarmAcknowledged is an event.

WHAT YOU ALREADY KNOW

You already see this difference in normal applications. PlaceOrder asks the system to do something. OrderPlaced records that the order was accepted. SendEmail asks for an action. EmailSent records that the action happened. DDD gives this pattern a formal place in the model.

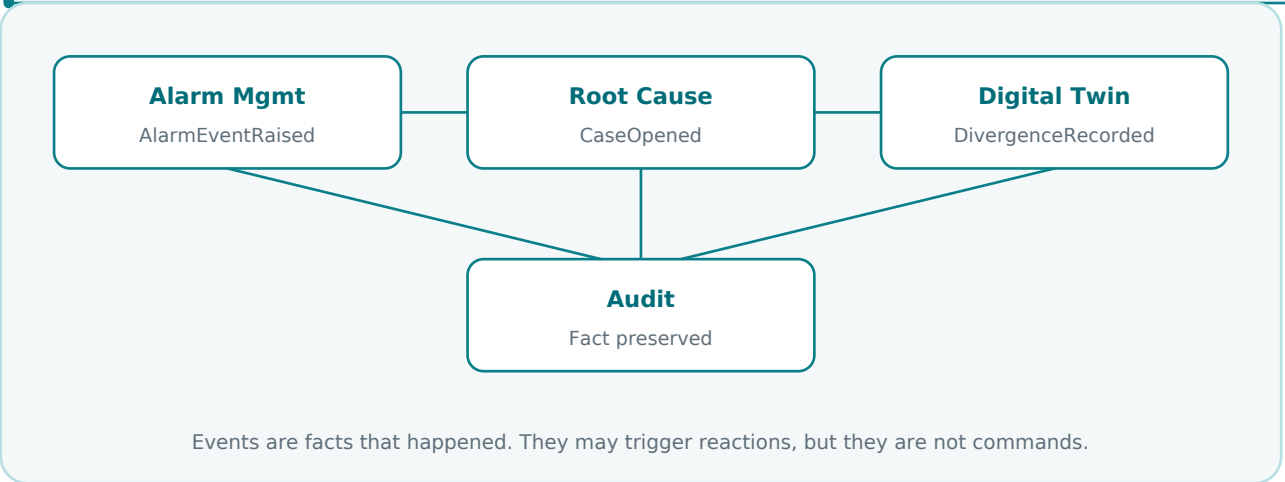


Figure 17.1 - A simple event flow across NEXUS-1 contexts. The event is a fact; other contexts may react to it, but they do not own the original fact.

Why events matter in NEXUS-1

NEXUS-1 is full of important facts: an alarm was raised, a flood was detected, a hypothesis was rejected, a divergence was recorded, a recommendation was generated, an audit entry was appended. These facts are not just logs. They are domain history.

Event	Plain meaning	Owning context
AlarmRaised	A monitored condition crossed an alarm rule.	Alarm Management
AlarmAcknowledged	A user accepted visibility of an alarm.	Alarm Management
AlarmFloodDetected	A group of alarms was recognized as a flood.	Alarm Management
RootCauseCaseOpened	A diagnostic investigation was started.	Root Cause
HypothesisRejected	A candidate explanation was tested and ruled out.	Root Cause
TwinDivergenceRecorded	The model and measurement disagreed at a recorded point.	Digital Twin

COMMON MISTAKE

Do not use events as a polite way to hide commands. RecommendationGenerated does not mean MoveRod. DivergenceRecorded does not mean TripPlant. AlarmRaised does not mean RootCauseFound. The event records a fact; interpretation belongs to the context that owns that rule.

Events should say exactly what happened

A good event name is boring and precise. It should not exaggerate. In NEXUS-1 this matters because several concepts are close but not identical. An alarm is not a cause. A divergence is not necessarily a failure. A recommendation is not a command.

```
public sealed record AlarmRaised(
    int UnitId,
    string SignalTag,
    string AlarmCode,
    string Severity,
    DateTime RaisedAtUtc);

public sealed record HypothesisRejected(
    int RootCauseCaseId,
    int HypothesisId,
    string Reason,
    DateTime RejectedAtUtc);

public sealed record TwinDivergenceRecorded(
    int TwinModelId,
    string SignalTag,
    decimal ModelValue,
    decimal MeasuredValue,
    DateTime ObservedAtUtc);
```

These examples are deliberately small. A production implementation might add correlation identifiers, causation identifiers, user identity, version, schema name, and audit metadata. The important thing is the language: each event says one domain fact and does not smuggle in a conclusion.

Event vs audit entry

A domain event and an audit entry are related, but they are not the same thing. The event belongs to the domain language. The audit entry belongs to traceability. One says what happened in business terms; the other preserves who, when, where, and under which technical conditions it was recorded.

Concept	Purpose	Example
Domain event	Expresses a meaningful fact in domain language.	HypothesisRejected
Audit entry	Preserves traceability and accountability.	User X rejected hypothesis Y at time T
Integration message	Carries information to another process or context.	RootCauseCaseClosed message on a bus

HONEST BOUNDARY

This book uses domain events to teach meaning and architecture. It does not claim that the current browser-based Phase-0 console runs a production event bus. In this demonstrator, the event language is a modelling discipline first; later implementation can choose in-memory dispatch, database outbox, message broker, or simple audit persistence.

CHAPTER 18

Commands, Decisions, and Recommendations

Many architecture mistakes begin when three different things are treated as one: a command, a decision, and a recommendation. NEXUS-1 is a good place to separate them because the difference is not academic. It is a safety boundary.

A command asks the system to do something. A decision is the domain rule deciding whether that thing is allowed. A recommendation is advice produced for a human or another layer to consider. A recommendation should never pretend to be authority.

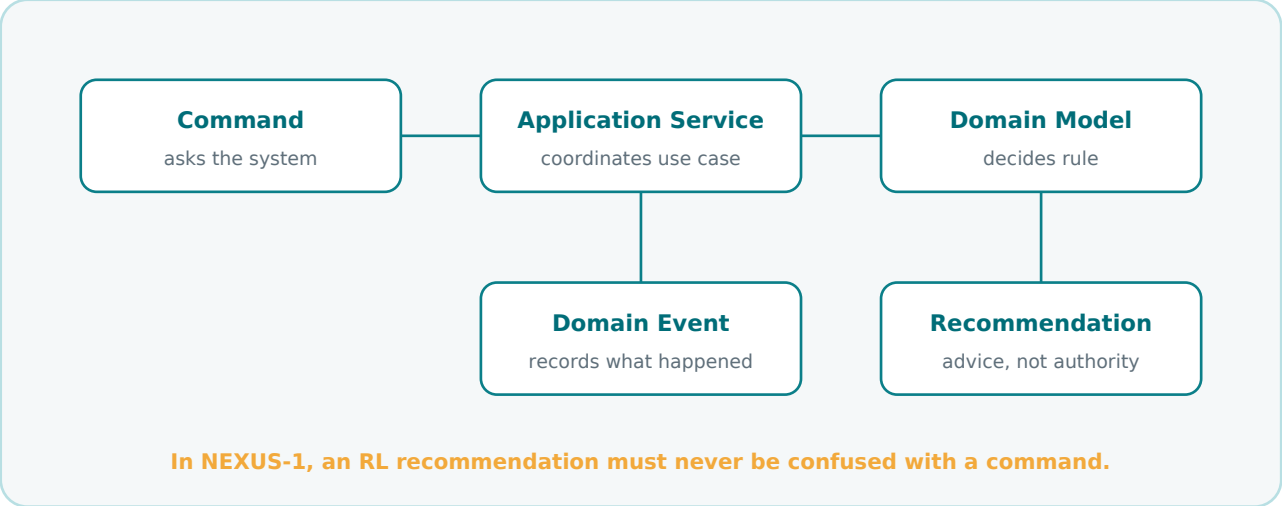


Figure 18.1 - Commands, decisions, events, and recommendations must stay separate. This is one of the most important DDD lessons for NEXUS-1.

The simple developer distinction

Thing	Question it answers	NEXUS-1 example
Command	What is being requested?	OpenRootCauseCase, AcknowledgeAlarm
Decision	Is the requested change valid according to domain rules?	Can this case be closed with no evidence?
Event	What fact happened after success?	RootCauseCaseClosed, AlarmAcknowledged

Recommendation	What does the advisory model suggest?	Suggest +2 pcm, hold, or insert gently

Notice the shape. Commands and recommendations can both point toward action, but they do not have the same authority. A command enters a use case. A recommendation enters a review or advisory workflow. The code should make that difference visible.

```
public sealed record AcknowledgeAlarmCommand(
    int AlarmEventId,
    string UserId,
    DateTime RequestedAtUtc);

public sealed record PolicyRecommendation(
    int UnitId,
    string StateCode,
    string SuggestedAction,
    decimal Confidence,
    DateTime GeneratedAtUtc);
```

COMMON MISTAKE

Do not name an advisory output as if it were an actuator command. MoveRodCommand and RodNudgeRecommendation are not the same thing. In NEXUS-1, the learning policy advises; it does not command.

Application services coordinate commands

An application service is not the place to hide domain decisions. It receives a request, loads the right aggregate, calls domain methods, saves changes, and records events. It is a coordinator. The rule belongs as close as possible to the object that owns the rule.

```
public sealed class CloseRootCauseCaseHandler
{
    private readonly IRootCauseCaseRepository _cases;
    private readonly IAuditSink _audit;

    public async Task Handle(CloseRootCauseCaseCommand command)
    {
        var rcaCase = await _cases.GetAsync(command.RootCauseCaseId);

        rcaCase.CloseWithVerdict(command.Verdict, command.UserId);

        await _cases.SaveAsync(rcaCase);
        await _audit.AppendAsync(rcaCase.DomainEvents);
    }
}
```

The handler does not decide whether the case has enough evidence. The aggregate should protect that invariant. The handler coordinates the use case and makes the result durable.

NEXUS-1 decisions that deserve explicit rules

Decision	Should live near	Reason
Can a root-cause case close?	RootCauseCase aggregate	It knows hypotheses, evidence, and verdict state.

Can an alarm be acknowledged?	AlarmEvent aggregate or domain service	It knows alarm lifecycle state.
Can a twin divergence be classified?	Digital Twin / diagnostic policy	Classification depends on context-specific meaning.
Can a recommendation become action?	Separate supervisory/safety context	Advisory learning must not own command authority.

HONEST BOUNDARY

This chapter deliberately keeps the command side conservative. NEXUS-1 is a demonstrator. The RL policy can generate recommendations, and the book can model how they would be reviewed, but it must not blur that into automatic control authority. In this architecture, advice remains advice until a separate, validated authority accepts it.

Mini cases for the reader

Use these cases to test the separation between command, decision, event, and recommendation.

Situation	Tempting name	Better name
A user clicks a button to acknowledge an alarm.	AlarmAcknowledgedCommand	AcknowledgeAlarmCommand
The system records that the alarm was acknowledged.	AcknowledgeAlarmEvent	AlarmAcknowledged
The RL policy suggests holding rods steady.	HoldRodsCommand	RodHoldRecommendation
The root-cause engine rules out FT-7.	DeleteHypothesis	HypothesisRejected
The twin detects model-measurement disagreement.	ModelFailed	TwinDivergenceRecorded

PART II PROGRESS

Part II has now defined NEXUS-1 as a domain model, introduced its language, classified its subdomains, identified bounded contexts and aggregates, and separated events, commands, decisions, and recommendations. The next continuation can complete the context relationships and passports, then close Part II with the NEXUS-1 context map.

NEXT BUILD

Continue with Chapter 19: Context Relationships and Passports, and Chapter 20: The NEXUS-1 Context Map. Preserve this cover page with minimal margins and the same PDF quality.

CHAPTER 19

Context Relationships and Passports

After a system is split into bounded contexts, the next question is obvious: how do the contexts talk without becoming one tangled model again? DDD does not say that contexts must never connect. It says the connection must be explicit, named, and honest.

In NEXUS-1, that explicit connection is often called a passport. A passport is the small piece of identity that allows one context to refer to something owned by another context without pretending to own it.

PLAIN MEANING

A passport is not a copy of the other context. It is a controlled reference. ReactorFleet owns the physical Unit. AlarmManagement can store UnitId on an alarm event, but that does not make AlarmManagement the owner of the Unit.

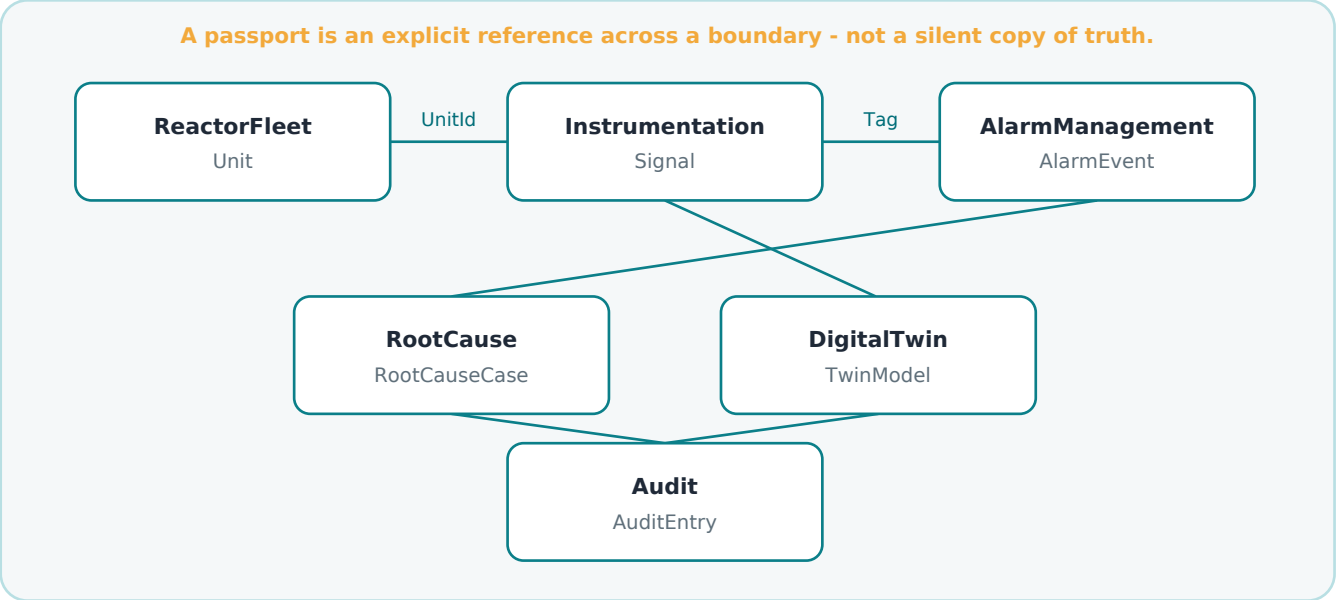


Figure 19.1 - Contexts can be connected without being merged. The passport carries identity across the boundary.

Why this matters

Most enterprise systems become messy not because they have relationships, but because they have unclear relationships. A copied name, a duplicated status, or an informal string key can look harmless at first. Later it becomes the place where two parts of the system disagree and nobody knows which one is true.

Relationship style	Developer meaning	NEXUS-1 example	Risk if unclear
Reference by passport	One context points to another context owner by a stable key.	AlarmEvent.UnitId points to ReactorFleet.Unit.	Low, if ownership is respected.
Published fact	One context emits an event that another may consume.	TwinDivergenceRecorded can be read by Audit or Reporting.	Medium, if event meaning is vague.
Read projection	A context builds a read-only view from facts owned elsewhere.	Reporting joins alarms, cases, and recommendations.	Low, if it does not become an owner.
Duplicated truth	Two contexts store the same business fact as if each owns it.	Unit status copied into many tables and edited in many places.	High. This creates disagreement.

COMMON MISTAKE

A foreign key is not automatically a good boundary. A good passport must still respect ownership. If another context starts changing the meaning of the referenced object, the passport has become a smuggling route.

Ownership is the key rule

The easiest way to keep the model clean is to ask one question again and again: who owns the truth?

Fact	Owner context	Other contexts may do
Physical identity of NX1-U1	ReactorFleet	Reference it by UnitId.
Meaning of a signal tag	Instrumentation	Bind it, alarm on it, display it, or cite it.
Lifecycle of an alarm event	AlarmManagement	Use it as input or evidence.
Final diagnostic verdict	RootCause	Read it, audit it, report it.
Model-to-measurement divergence	DigitalTwin	Use it as evidence or display it.
Learned advisory policy	ReinforcementLearning	Read recommendations, never treat them as commands.
Immutable change trace	Audit	Link to it, but not rewrite it.

A mid-level developer can use this table as a practical checklist during implementation. Before adding a column, DTO property, endpoint, or EF navigation, ask whether the current context owns that fact or only needs a passport to it.

Small C# example

```
public sealed class AlarmEvent
{
    public int AlarmEventId { get; private set; }

    // Passport to the physical unit.
    // AlarmManagement references the Unit; it does not own the Unit.
    public int UnitId { get; private set; }

    // Passport to the signal identity.
    public string SignalTag { get; private set; } = null!;

    public AlarmSeverity Severity { get; private set; }
    public AlarmState State { get; private set; }

    public void Acknowledge(UserId userId, DateTime atUtc)
    {
        if (State != AlarmState.Raised)
            throw new InvalidOperationException("Only a raised alarm can be acknowledged.");

        State = AlarmState.Acknowledged;
        AcknowledgedBy = userId;
        AcknowledgedAtUtc = atUtc;
    }
}
```

The class contains UnitId and SignalTag, but it does not contain the whole Unit object and it does not define the Signal. That keeps AlarmManagement focused on its own job: alarm lifecycle.

HONEST BOUNDARY

The word passport is a teaching term used by this book. DDD literature uses terms such as context relationship, reference, published language, shared kernel, or anti-corruption layer. The NEXUS-1 word is intentionally practical: it helps a developer remember that crossing a boundary requires explicit identity.

Passports and databases

In the SQL Server version of NEXUS-1, many passports become foreign keys. This is useful because the database can protect the reference. But DDD still asks a deeper question than the database does: what does the reference mean?

```
CREATE TABLE AlarmManagement.AlarmEvent (
    AlarmEventId INT IDENTITY PRIMARY KEY,
    UnitId       INT NOT NULL,
    SignalTag    NVARCHAR(80) NOT NULL,
    RaisedAtUtc  DATETIME2 NOT NULL,
    SeverityId   INT NOT NULL,

    CONSTRAINT FK_AlarmEvent_Unit
        FOREIGN KEY (UnitId)
        REFERENCES ReactorFleet.Unit(UnitId)
);
```

This foreign key says the alarm points to a real unit in the fleet model. It does not say AlarmManagement may redefine what that unit is. The database protects existence; the domain model protects meaning.

Database mechanism	DDD meaning	Developer warning
Foreign key	A passport to an owner in another context.	Do not treat it as shared ownership.
Unique index	A rule that a name or code has one meaning in a context.	Scope the uniqueness to the right boundary.
View / read model	A projection for reading across contexts.	Do not update domain truth through it.
Audit table	Evidence that something changed or happened.	Do not confuse log storage with domain behaviour.

SEE IT IN NEXUS-1

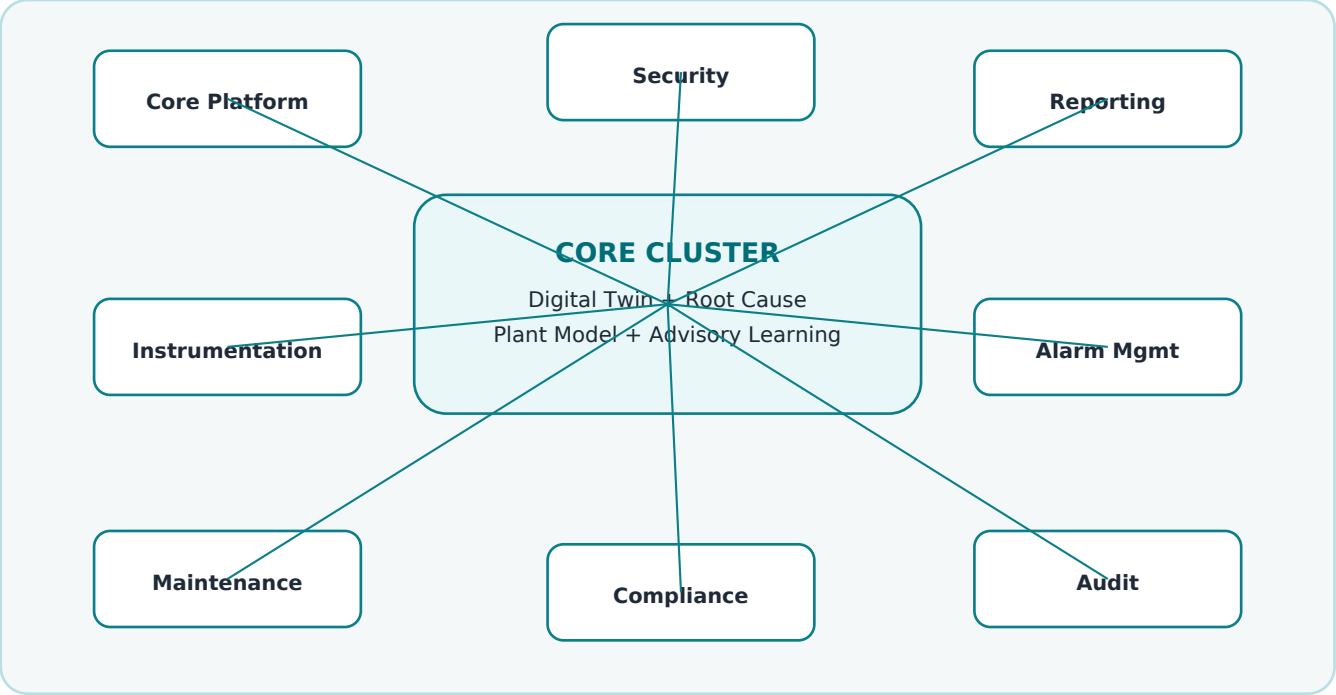
Open Plant Fleet, Root Cause Graph, and Optimization (RL). They are separate screens, but they follow the same selected Unit. That is the visible version of the passport idea: many views, one physical unit identity.

CHAPTER 20

The NEXUS-1 Context Map

A context map is the drawing that keeps the bounded contexts from becoming a list of names. It shows how the parts of the system relate, which contexts are central, and where the important boundaries are.

For NEXUS-1, the context map is not a deployment diagram. It does not say which process runs where. It is a map of meaning and ownership. It helps a developer understand why the same Unit appears in many places, why an alarm is not a cause, and why a recommendation is not a command.



The core cluster is not one table or one controller. It is the place where NEXUS-1 becomes more than a dashboard: the plant model, digital twin, root-cause reasoning, and advisory policy meet while still keeping their boundaries.

How to read the map

Map element	Plain interpretation	Design consequence
Core cluster	The special domain value of NEXUS-1.	Use the most careful language and tests here.
Instrumentation -> Core	Signals feed the twin, alarms, and reasoning.	Signal identity must be stable.
Alarm Management -> Root Cause	Alarm events become symptoms or evidence.	Alarm severity must not become causality.
Audit around everything	Important changes and conclusions must be traceable.	Domain events should be auditable.
Reporting outside the core	Reports combine facts owned elsewhere.	Reporting reads; it should not own domain truth.

A map for developers, not only architects

A useful context map should answer practical questions. Where should this class live? Which context owns this rule? Should this service call another service directly? Is this table authoritative or only a projection?

DEVELOPER CHECKLIST

When adding a new NEXUS-1 feature, ask: 1) Which context owns the vocabulary? 2) Which aggregate protects consistency? 3) Is this a command, event, query, or recommendation? 4) Does the feature need a passport to another context? 5) Is the output authoritative truth or a read model?

Example: adding a new diagnostic note

Suppose we add a feature that lets an engineer attach a note to a root-cause investigation. A flat architecture might simply add DiagnosticNote to any convenient table. A DDD approach asks where the note belongs.

Question	Answer for this feature
Which context owns it?	RootCause, because the note is part of an investigation.
Which aggregate protects it?	RootCauseCase. Notes should be added through the case.
Is it a command or event?	AddDiagnosticNote is a command; DiagnosticNoteAdded is an event.
Does it need passports?	It may reference UserId and perhaps AlarmEventId, but it does not own either.
Should Reporting edit it?	No. Reporting may display it through a read model.

```
public sealed record AddDiagnosticNoteCommand(
    int RootCauseCaseId,
    UserId AuthorId,
    string Text);

public sealed record DiagnosticNoteAdded(
    int RootCauseCaseId,
    UserId AuthorId,
    DateTime AddedAtUtc);
```

The command asks for a change. The event records that the change happened. The context map tells us the command belongs to the RootCause application layer, not to Reporting, not to Audit, and not to AlarmManagement.

Part II checkpoint

Part II has now moved NEXUS-1 from screens to domain architecture. The platform can be read as a domain model, not merely as a collection of UI panels.

Chapter	What it added
11 - NEXUS-1 as a Domain	The system is a model of concepts, rules, and decisions, not just a dashboard.
12 - Ubiquitous Language	Words such as Unit, Signal, AlarmEvent, Hypothesis, Divergence, and Recommendation were controlled.
13 - Main Subdomains	The platform was divided into meaningful regions.
14 - Core / Supporting / Generic	Strategic importance was separated from ordinary plumbing.
15 - Bounded Contexts	Each region received a boundary of vocabulary and ownership.
16 - Aggregates	Consistency boundaries were identified.
17 - Domain Events	Important facts were named as things that happened.
18 - Commands / Decisions / Recommendations	Requests, conclusions, and advisory outputs were separated.
19 - Passports	Cross-context references were made explicit.
20 - Context Map	The complete meaning map was drawn.

HONEST BOUNDARY

The context map is not the final truth of every future NEXUS-1 version. It is the best map for this demonstrator and this book. If the system evolves, the map should evolve with it. The value is not that the drawing never changes; the value is that changes become discussable.

NEXT BUILD

The next continuation should begin Part III: Implementing DDD in NEXUS-1. Start with Chapter 21, From Bounded Context to Project Structure, then Chapter 22, From Bounded Context to SQL Schema.

PART III

IMPLEMENTING DDD IN NEXUS-1

From domain meaning to C# projects, SQL schemas, EF Core mappings, application services, read models, and reviewable architecture.

BOUNDARIES - AGGREGATES - PROJECTS - SCHEMAS - SERVICES

CHAPTER 21

From Bounded Context to Project Structure

Part II described the meaning of NEXUS-1. Part III turns that meaning into implementation. The first implementation decision is not the database and not the controller. It is the shape of the codebase.

A mid-level .NET developer already knows folders, projects, namespaces, services, entities, DTOs, and DbContext. DDD does not throw those away. It gives them a reason to exist and a rule for where each thing belongs.

PLAIN MEANING

Project structure is the physical memory of the domain model. If the domain says ReactorFleet, RootCause, DigitalTwin, and ReinforcementLearning are different contexts, the code should not hide them inside one anonymous Models folder.

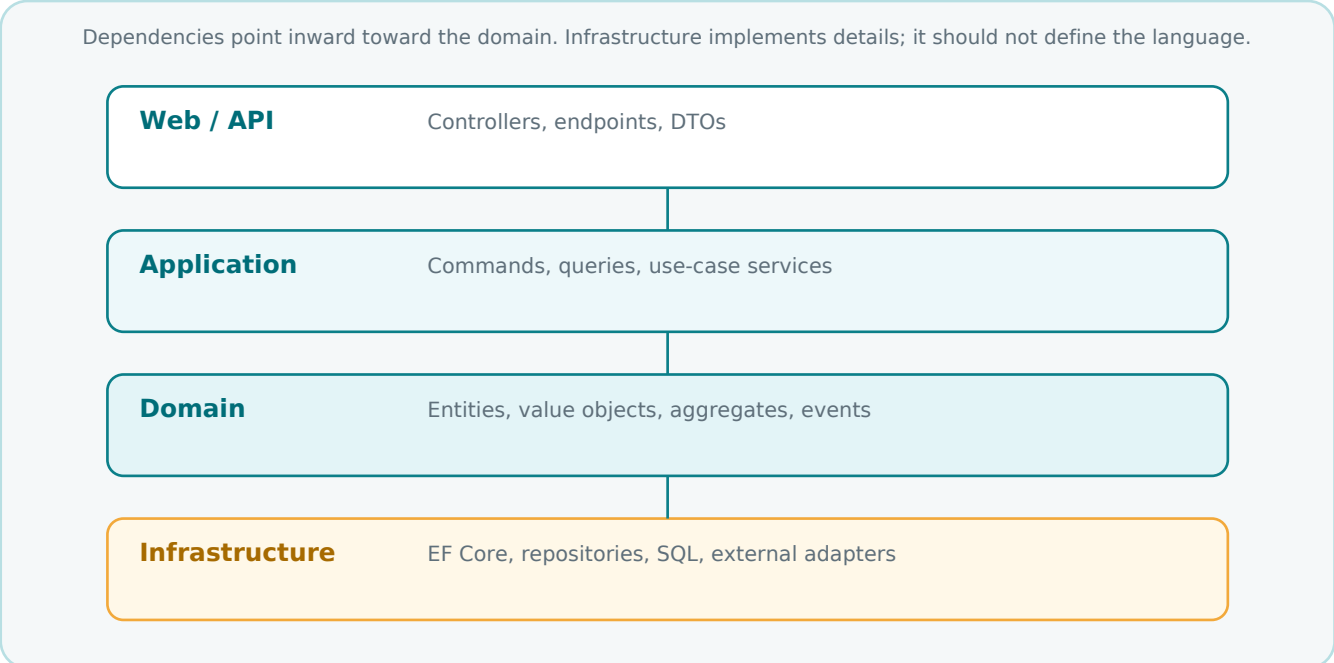


Figure 21.1 - A practical layered structure. The domain language belongs in the Domain layer; infrastructure implements persistence and external details.

The simple rule

A bounded context should be visible in the code. The reader should be able to open the solution and see the same boundaries that appear in the context map. This does not require a microservice per context. It requires clear ownership in the code.

Layer / project	What belongs there	NEXUS-1 example
Nexus.Domain	Pure domain concepts: entities, value objects, aggregates, domain events, domain services.	Unit, SignalTag, RootCauseCase, TwinDivergence, PolicyRecommendation.
Nexus.Application	Use cases: commands, queries, application services, interfaces for repositories and gateways.	OpenRootCauseCaseCommand, RecordTwinDivergenceHandler.
Nexus.Infrastructure	EF Core DbContext, repository implementations, SQL mappings, external adapters.	NexusContext, RootCauseCaseRepository, UnitConfiguration.
Nexus.Web / API	Controllers, endpoints, request/response DTOs, authentication boundary.	RootCauseController, AlarmEventsController.
Nexus.Reporting	Read models and projections for screens and reports.	FleetOverviewReadModel, AlarmFloodSummaryView.

COMMON MISTAKE

Do not create folders named Entities, Services, Repositories, DTOs and then throw every context into them. That groups code by technical type, not by meaning. In a domain-heavy system, meaning must be visible first.

Better namespace shape

```
Nexus.Domain/
  ReactorFleet/
    Unit.cs
    UnitCode.cs
    UnitStatusChanged.cs
  Instrumentation/
    Signal.cs
    SignalTag.cs
    EngineeringValue.cs
  RootCause/
    RootCauseCase.cs
    Hypothesis.cs
    Evidence.cs
    RootCauseCaseClosed.cs
  DigitalTwin/
    TwinModel.cs
    SignalBinding.cs
    TwinDivergence.cs
  ReinforcementLearning/
    Policy.cs
    Recommendation.cs

Nexus.Application/
  RootCause/
    OpenRootCauseCaseCommand.cs
    OpenRootCauseCaseHandler.cs
  DigitalTwin/
    RecordTwinDivergenceCommand.cs

Nexus.Infrastructure/
  Persistence/
    NexusContext.cs
    Configurations/
      ReactorFleet/
      RootCause/
      DigitalTwin/
```

This structure is boring in the best way. A developer who works on RootCause knows where to look. A developer who works on DigitalTwin does not scroll through alarm rules. A reviewer can see whether a change crosses a boundary.

Application services coordinate; domain objects decide

A common confusion is to put all rules into services. That creates a system where entities are only data containers. DDD prefers the opposite: application services coordinate the use case, while aggregates protect their own rules.

```
public sealed class CloseRootCauseCaseHandler
{
    private readonly IRootCauseCaseRepository _cases;
    private readonly IUnitOfWork _unitOfWork;

    public async Task Handle(CloseRootCauseCaseCommand command)
    {
        var rootCauseCase = await _cases.Get(command.RootCauseCaseId);

        rootCauseCase.CloseWithVerdict(
            verdict: command.Verdict,
            closedBy: command.UserId,
            closedAtUtc: DateTime.UtcNow);

        await _unitOfWork.SaveChangesAsync();
    }
}

// The handler coordinates loading and saving.
// RootCauseCase owns the rule: a case cannot close without evidence.
```

Question	Application service	Domain aggregate
Who loads data?	Yes. It asks repositories for aggregates.	No. It should not know SQL or EF Core.
Who checks domain rules?	Only coordinates.	Yes. It protects consistency.
Who sends email or writes files?	May call infrastructure interfaces.	No.
Who records a domain event?	May dispatch after save.	Usually raises the event when state changes.

SEE IT IN NEXUS-1

Think of the Root Cause Graph screen. The screen asks for a case to be opened, evidence to be attached, or a verdict to be shown. The screen should not own the diagnostic rules. The RootCause context should own them.

Choosing modular monolith before microservices

DDD is often confused with microservices. They are related only sometimes. A bounded context is first a boundary of meaning. It may later become a service boundary, but it does not have to start that way.

Option	What it means	Good for NEXUS-1 when
Single project	Everything lives in one deployable and weak internal boundaries.	Only for a prototype or small teaching sample.
Modular monolith	One deployable, but contexts are separated by namespaces, folders, assemblies, and rules.	Best starting point for this book. It keeps DDD visible without distributed complexity.
Microservices	Each context or group of contexts may deploy independently.	Only when team, scaling, data ownership, and operations justify it.
Distributed monolith	Many services but unclear ownership and shared database confusion.	Avoid. It creates the cost of microservices without the clarity.

HONEST BOUNDARY

This book does not claim that NEXUS-1 must be implemented as microservices. For a demonstrator and teaching architecture, a modular monolith with clear bounded-context structure is often stronger. The first goal is clean meaning, not distributed deployment.

CHAPTER 22

From Bounded Context to SQL Schema

Once the code boundaries are visible, the database should not flatten them again. If the domain has bounded contexts but the database has one giant dbo namespace, the model loses a lot of its clarity at the persistence boundary.

In NEXUS-1, the practical rule is simple: one bounded context becomes one SQL Server schema. ReactorFleet owns ReactorFleet tables. RootCause owns RootCause tables. DigitalTwin owns DigitalTwin tables. Cross-context references are passports, not ownership transfers.

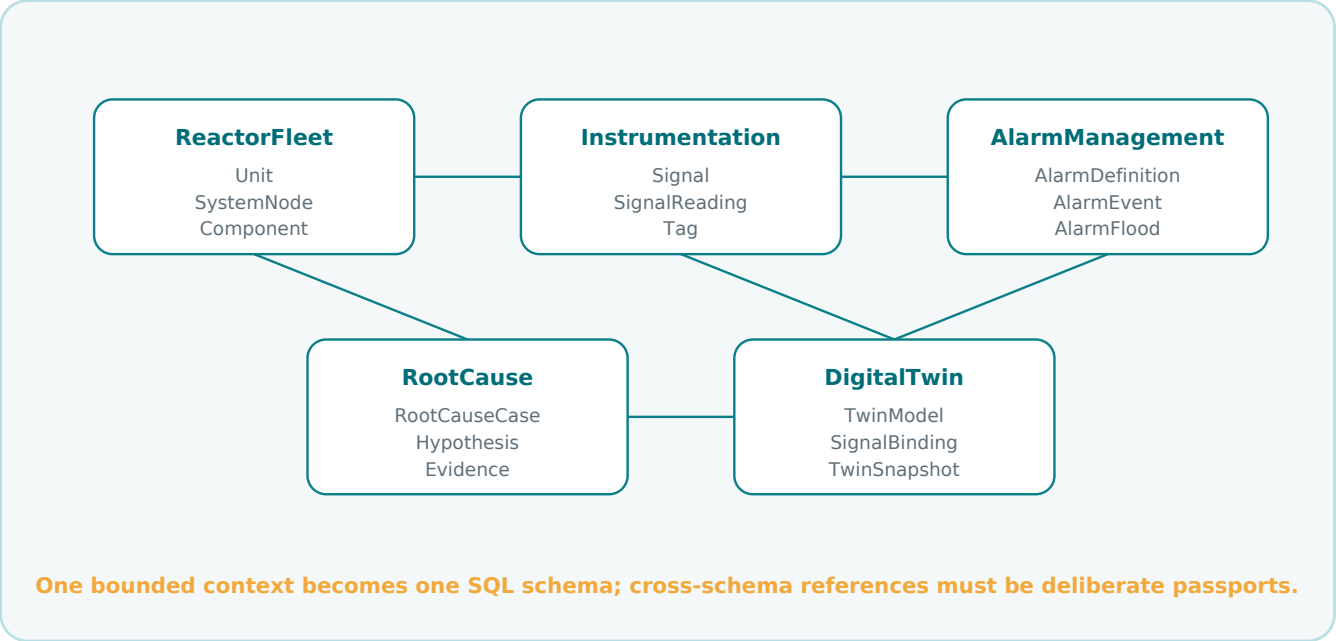


Figure 22.1 - SQL schemas mirror the bounded contexts. The database becomes readable as a domain map instead of a flat table warehouse.

Why SQL schemas fit this book

SQL Server schemas are not magic, and they do not enforce all DDD rules. But they are extremely useful because they make ownership visible in the name itself. ReactorFleet.Unit and CorePlatform.EngineeringUnit can both exist without confusing the reader. The context is part of the table name.

Bounded context	SQL schema	Example tables	Main responsibility
Reactor Fleet	ReactorFleet	Unit, Reactor, SystemNode, Component	Physical plant identity and structure.
Instrumentation	Instrumentation	Signal, SignalReading, EngineeringUnitRef	Signal identity and measurements.
Alarm Management	AlarmManagement	AlarmDefinition, AlarmEvent, AlarmFlood	Alarm lifecycle and grouping.
Root Cause	RootCause	RootCauseCase, Hypothesis, Evidence	Diagnostic reasoning and verdicts.
Digital Twin	DigitalTwin	TwinModel, SignalBinding, TwinSnapshot	Binding model state to measured reality.
Reinforcement Learning	ReinforcementLearning	Policy, QTableEntry, Episode	Advisory learning and readable policy.
Audit	Audit	AuditEntry, EntityChange	Traceability and change history.

```
CREATE SCHEMA ReactorFleet;
CREATE SCHEMA Instrumentation;
CREATE SCHEMA AlarmManagement;
CREATE SCHEMA RootCause;
CREATE SCHEMA DigitalTwin;
CREATE SCHEMA ReinforcementLearning;
CREATE SCHEMA Audit;
```

The first DDL statements are not tables. They are boundaries. The table comes after the language has a home.

Passports as foreign keys

A foreign key can represent a passport, but only when its meaning is clear. The database can enforce that a referenced Unit exists. It cannot, by itself, teach developers that ReactorFleet owns the Unit and AlarmManagement only references it.

```
CREATE TABLE AlarmManagement.AlarmEvent (
    AlarmEventId INT IDENTITY PRIMARY KEY,
    UnitId       INT NOT NULL,
    SignalTag    NVARCHAR(80) NOT NULL,
    RaisedAtUtc  DATETIME2 NOT NULL,
    SeverityId   INT NOT NULL,
    StateId      INT NOT NULL,

    CONSTRAINT FK_AlarmManagement_AlarmEvent_UnitId
        FOREIGN KEY (UnitId)
        REFERENCES ReactorFleet.Unit(UnitId)
);
```

Column	DDD meaning	Owner of truth
AlarmEventId	Identity of the alarm event inside AlarmManagement.	AlarmManagement
UnitId	Passport to the physical unit.	ReactorFleet
SignalTag	Passport or published identity of the signal.	Instrumentation
SeverityId	Alarm-management classification.	AlarmManagement / lookup
StateId	Alarm lifecycle state.	AlarmManagement

COMMON MISTAKE

Do not copy UnitName, UnitStatus, SignalDescription, and other foreign-context facts into AlarmEvent as editable truth. If a read model needs those values for display, project them deliberately. Do not let duplicated columns become a second source of truth.

Schema does not equal aggregate

One more distinction matters. A SQL schema maps well to a bounded context. It does not automatically map to an aggregate. A context may contain many aggregates, and an aggregate may use more than one table inside the same schema.

DDD idea	Database expression	NEXUS-1 example
Bounded context	Usually one SQL schema.	RootCause schema.
Aggregate root	Main table plus child tables protected by the root.	RootCause.RootCauseCase with hypotheses and evidence.
Value object	Owned columns or owned table, depending on size and reuse.	EngineeringValue as value + unit symbol, or owned type.
Domain event	Event table, outbox, or audit entry after state change.	RootCauseCaseClosed recorded for audit and integration.
Read model	View, materialized projection, or query table.	Reporting.FleetOverview.

This prevents a common mistake: assuming every table is an aggregate, or every aggregate is one table. DDD is about consistency and meaning. SQL is about storage and constraints. They should cooperate, not pretend to be the same thing.

EF Core connection

```
public sealed class RootCauseCaseConfiguration
    : IEntityTypeConfiguration<RootCauseCase>
{
    public void Configure(EntityTypeBuilder<RootCauseCase> b)
    {
        b.ToTable("RootCauseCase", "RootCause");

        b.HasKey(x => x.RootCauseCaseId)
            .HasName("PK_RootCause_RootCauseCase");

        b.HasMany(x => x.Evidence)
            .WithOne()
            .HasForeignKey(x => x.RootCauseCaseId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_Evidence_RootCauseCaseId");
    }
}
```

Migrations as reviewable domain changes

In a Code First project, a migration is not only a technical artifact. It is a reviewable record of how the model changed. A good migration should read like a deliberate change to the domain structure, not like a surprise generated by tooling.

Migration symptom	What it may reveal	How to respond
Unexpected cascade delete	EF inferred a dangerous relationship.	Configure delete behavior explicitly.
Shadow foreign key column	Navigation and FK property do not match.	Fix entity or configuration.
Table created in dbo	Context boundary was not mapped.	Add ToTable with schema.
Random constraint names	Database contract is hard to review.	Name keys, indexes, and FKs explicitly.
Duplicate lookup table	Language or ownership is unclear.	Move reference data to its owning context.

HONEST BOUNDARY

A SQL schema does not automatically make a good bounded context. It only makes the boundary visible. The real design work is still in the vocabulary, ownership, rules, aggregates, and events. The database can support DDD; it cannot replace it.

NEXT BUILD

The next continuation should proceed with Chapter 23, From Aggregate Root to C# Entity, and Chapter 24, From Value Object to Owned Type. These chapters should connect the domain model directly to C# and EF Core implementation.

CHAPTER 23

From Aggregate Root to C# Entity

Chapter 16 identified aggregates in NEXUS-1. This chapter turns that design into C# code. The goal is not to make entities complicated. The goal is to stop important rules from leaking into controllers, screens, and random services.

In DDD, an aggregate root is the object outside code is allowed to talk to when it wants to change a small consistency boundary. In C#, that usually means a class with identity, private state, controlled methods, and child collections that cannot be modified from the outside.

PLAIN MEANING

An aggregate root is the door into a small room of related objects. The outside world does not climb through the window and move the furniture. It knocks on the door. The root decides what is allowed.

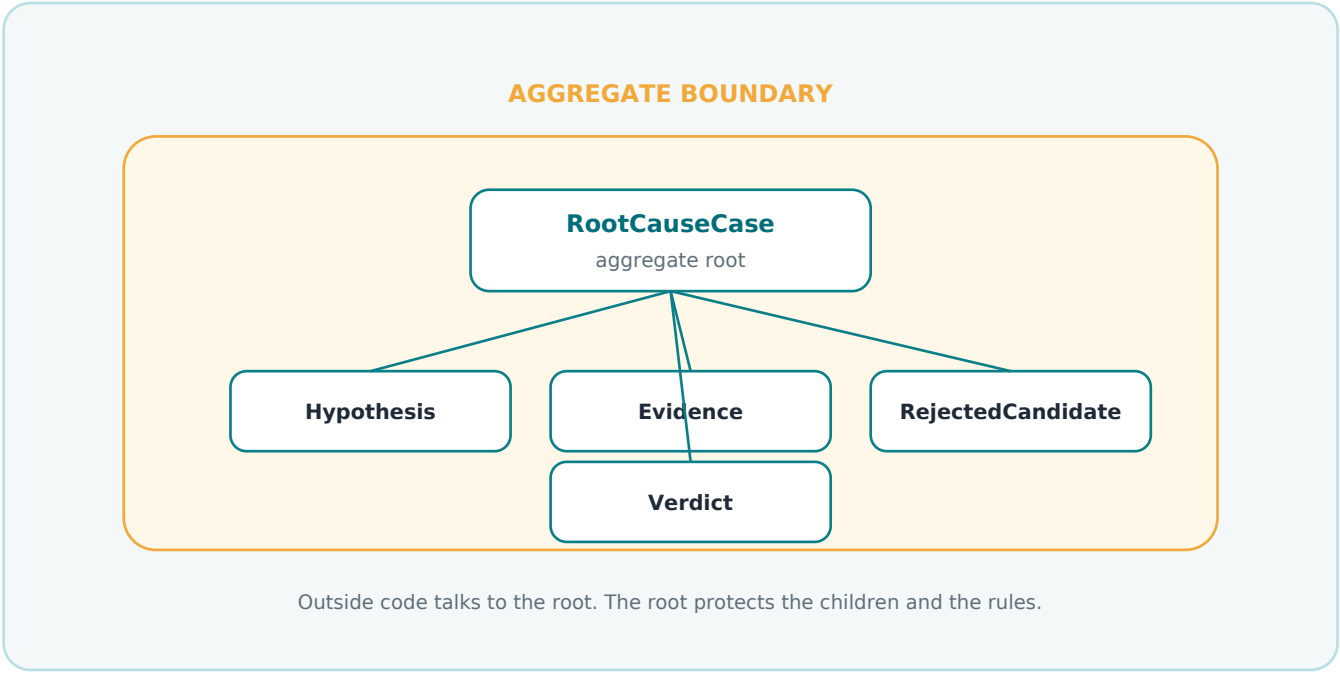


Figure 23.1 - RootCauseCase as an aggregate root. Evidence, hypotheses, rejected candidates, and verdict belong inside one consistency boundary.

Entity is not automatically aggregate root

A common mistake is to assume every EF Core entity is an aggregate root. That is not true. Many entities are children, history rows, lookups, or read-model records. An aggregate root is special because it protects rules and consistency.

C# class kind	What it means	NEXUS-1 example
Entity	Has identity and can change over time.	Unit, Signal, AlarmEvent.
Aggregate root	Entity that controls a consistency boundary.	RootCauseCase, TwinModel, Policy.
Child entity	Has identity but should be changed through its root.	Hypothesis inside RootCauseCase.
Lookup entity	Reference data with controlled lifecycle.	AlarmSeverity, UnitStatus.
Read model	Query shape for screens and reports, not a rule owner.	FleetOverviewReadModel.

COMMON MISTAKE

Do not expose public List collections from an aggregate and let any service add or remove children. That bypasses the root and moves rules into accidental places.

A better C# shape

```
public sealed class RootCauseCase
{
    private readonly List<Evidence> _evidence = new();
    private readonly List<Hypothesis> _hypotheses = new();

    private RootCauseCase() { } // EF Core

    public RootCauseCase(int unitId, string openedBy, DateTime openedAtUtc)
    {
        UnitId = unitId;
        OpenedBy = openedBy;
        OpenedAtUtc = openedAtUtc;
        Status = RootCauseCaseStatus.Open;
    }

    public int RootCauseCaseId { get; private set; }
    public int UnitId { get; private set; }
    public RootCauseCaseStatus Status { get; private set; }
    public string OpenedBy { get; private set; } = null!;
    public DateTime OpenedAtUtc { get; private set; }
    public string? FinalVerdict { get; private set; }

    public IReadOnlyCollection<Evidence> Evidence => _evidence;
    public IReadOnlyCollection<Hypothesis> Hypotheses => _hypotheses;
}
```

Methods express domain rules

The class above is still only a shape. The real value appears when domain rules become methods. A method name should read like the language of the domain, not like database manipulation.

```
public void AddEvidence(Evidence evidence)
{
    if (Status != RootCauseCaseStatus.Open)
        throw new InvalidOperationException(
            "Evidence can be added only while the case is open.");

    _evidence.Add(evidence);
}

public void RejectHypothesis(HypothesisId hypothesisId, string reason)
{
    var hypothesis = _hypotheses.Single(x => x.Id == hypothesisId);
    hypothesis.Reject(reason);
}

public void CloseWithVerdict(string verdict, string closedBy, DateTime closedAtUtc)
{
    if (!_evidence.Any())
        throw new InvalidOperationException(
            "A root-cause case cannot close without evidence.");

    if (_hypotheses.All(x => x.Status == HypothesisStatus.Rejected))
        throw new InvalidOperationException(
            "At least one hypothesis must remain supported or accepted.");

    FinalVerdict = verdict;
    ClosedBy = closedBy;
    ClosedAtUtc = closedAtUtc;
    Status = RootCauseCaseStatus.Closed;
}
```

Method	Domain sentence it protects
AddEvidence	Only an open case may receive new evidence.
RejectHypothesis	A rejected hypothesis remains part of the case history.
CloseWithVerdict	A case cannot close without evidence and a defensible conclusion.
Reopen	Only a closed case may be reopened, and the reason must be recorded.

SEE IT IN NEXUS-1

In the Root Cause screen, a rejected candidate is not garbage. It is part of the reasoning record. The aggregate root should therefore preserve rejected hypotheses instead of allowing them to be deleted casually.

EF Core mapping without losing the domain

EF Core can persist this aggregate, but EF Core should not define the language. The entity class speaks domain language. The configuration class speaks persistence language: table, key, relationship, delete behavior, and constraint names.

```
public sealed class RootCauseCaseConfiguration
    : IEntityTypeConfiguration<RootCauseCase>
{
    public void Configure(EntityTypeBuilder<RootCauseCase> b)
    {
        b.ToTable("RootCauseCase", "RootCause");

        b.HasKey(x => x.RootCauseCaseId)
            .HasName("PK_RootCause_RootCauseCase");

        b.Property(x => x.OpenedBy)
            .HasMaxLength(100)
            .IsRequired();

        b.HasMany(x => x.Evidence)
            .WithOne()
            .HasForeignKey(x => x.RootCauseCaseId)
            .OnDelete(DeleteBehavior.Restrict)
            .HasConstraintName("FK_RootCause_Evidence_RootCauseCaseId");

        b.Navigation(x => x.Evidence)
            .UsePropertyAccessMode(PropertyAccessMode.Field);
    }
}
```

Design choice	Why it matters
Private constructor	Allows EF Core materialization without making construction meaningless.
Private setters	Prevents random code from bypassing domain methods.
Read-only collections	Outside code can read children, not mutate them directly.
Field access for collections	EF Core can persist the collection without exposing a public setter.
Restrict delete	A case should not silently delete evidence because of an accidental cascade.

Aggregate design checklist

Question	Good sign	Warning sign
Who owns the rule?	The aggregate method enforces it.	A controller or UI decides it.
Can children be changed directly?	No, they are protected behind methods.	Public lists and public setters are everywhere.
Is the boundary too large?	It protects one focused consistency problem.	It tries to contain half the system.
Is the boundary too small?	It includes objects that must be correct together.	Important rules require many independent saves.
Can the aggregate be loaded reasonably?	It is not huge for normal use cases.	Loading it requires thousands of rows every time.

HONEST BOUNDARY

DDD does not require every rule to live inside an entity. Some rules belong in domain services, some in application services, and some in policies or specifications. The point is not to worship entities. The point is to stop important domain rules from becoming scattered accidents.

CHAPTER 24

From Value Object to Owned Type

A value object is a concept described by its values, not by a separate identity. This is one of the easiest DDD ideas to understand and one of the easiest to ignore in normal CRUD code.

Mid-level developers often model every concept as an entity because every concept eventually appears in code. DDD asks a simpler question: does this thing need its own identity and lifecycle, or is it just a meaningful value?

PLAIN MEANING

If two objects with the same values are interchangeable, you are probably looking at a value object. If identity matters even when values change, you are probably looking at an entity.

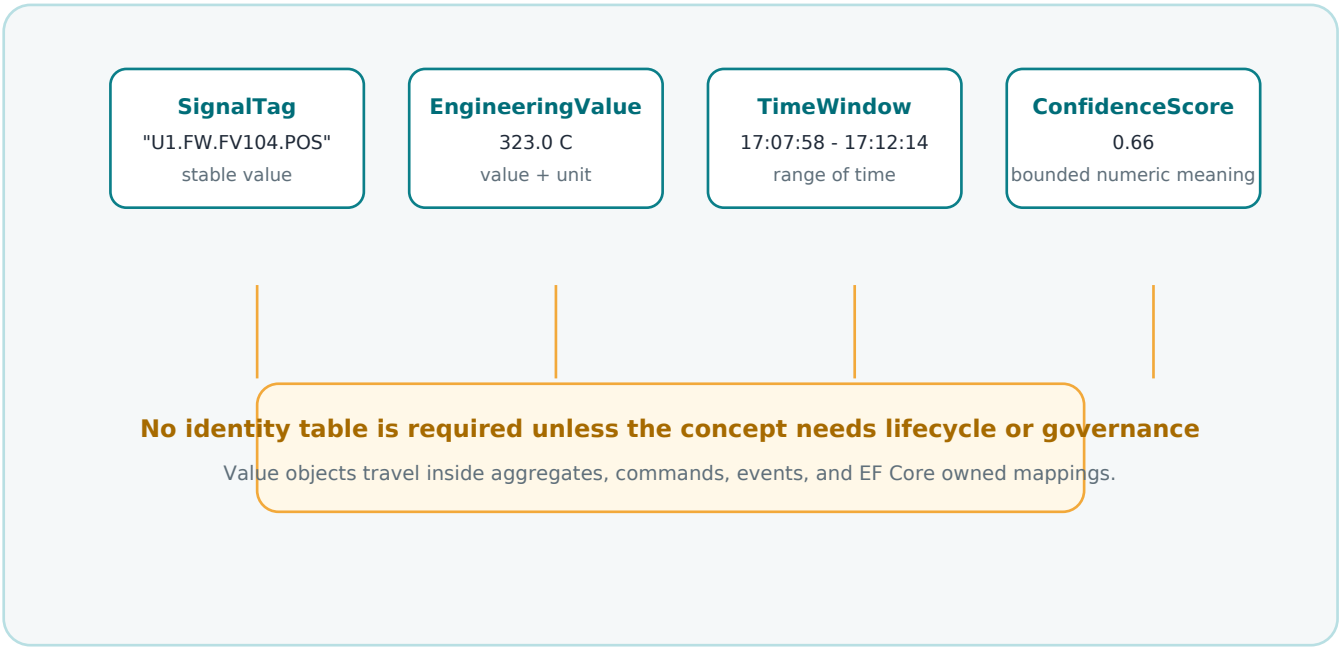


Figure 24.1 - Value objects give names to meaningful values without inventing unnecessary identity.

NEXUS-1 value objects

Value object	Values inside it	Why it is useful
SignalTag	Tag text such as U1.FW.FV104.POS.	Prevents tag strings from being treated as anonymous text.
EngineeringValue	Numeric value plus unit symbol.	Keeps 323 C different from 323 K or 323 bar.
TimeWindow	Start and end time.	Gives delay windows and alarm windows explicit meaning.
ConfidenceScore	Decimal between 0 and 1.	Prevents invalid confidence values such as -4 or 2.7.
ReactivityNudge	pcm amount within allowed advisory band.	Keeps recommendation values inside a named safety boundary.

```
public readonly record struct SignalTag(string Value)
{
    public SignalTag
    {
        if (string.IsNullOrWhiteSpace(Value))
            throw new ArgumentException("Signal tag cannot be empty.");

        if (Value.Length > 80)
            throw new ArgumentException("Signal tag is too long.");
    }

    public override string ToString() => Value;
}

public readonly record struct ConfidenceScore(decimal Value)
{
    public ConfidenceScore
    {
        if (Value < 0m || Value > 1m)
            throw new ArgumentOutOfRangeException(nameof(Value));
    }
}
```

The point is small but powerful. Instead of passing anonymous strings and decimals everywhere, the code carries meaning. A SignalTag is not just any string. A ConfidenceScore is not just any decimal.

EngineeringValue as an owned type

Some value objects are small enough to live as columns inside the owning table. EF Core calls this an owned type. It is useful when the value object does not need its own table or lifecycle.

```
public sealed class TwinDivergence
{
    public int TwinDivergenceId { get; private set; }
    public int TwinModelId { get; private set; }
    public SignalTag SignalTag { get; private set; }
    public EngineeringValue Modelled { get; private set; }
    public EngineeringValue Measured { get; private set; }
    public DateTime ObservedAtUtc { get; private set; }
}

public readonly record struct EngineeringValue(
    decimal Value,
    string UnitSymbol);
```

```
public sealed class TwinDivergenceConfiguration
    : IEntityTypeConfiguration<TwinDivergence>
{
    public void Configure(EntityTypeBuilder<TwinDivergence> b)
    {
        b.ToTable("TwinDivergence", "DigitalTwin");

        b.OwnsOne(x => x.Modelled, owned =>
        {
            owned.Property(x => x.Value)
                .HasColumnName("ModelledValue")
                .HasPrecision(18, 6);

            owned.Property(x => x.UnitSymbol)
                .HasColumnName("ModelledUnit")
                .HasMaxLength(20);
        });

        b.OwnsOne(x => x.Measured, owned =>
        {
            owned.Property(x => x.Value)
                .HasColumnName("MeasuredValue")
                .HasPrecision(18, 6);

            owned.Property(x => x.UnitSymbol)
                .HasColumnName("MeasuredUnit")
                .HasMaxLength(20);
        });
    }
}
```

When value object should not become owned type

Owned types are useful, but they are not always the answer. Some concepts look like value objects at first but require governance, lookup tables, translations, permissions, or history. Those may belong as reference entities in a platform context.

Concept	Value object or entity?	Reason
SignalTag used inside a command	Value object	It wraps and validates a tag string.
Signal registered in instrumentation catalogue	Entity	It has identity, description, lifecycle, and relationships.
EngineeringValue 323 C	Value object	It is value plus unit symbol.
EngineeringUnit catalogue row	Entity / reference data	It has code, name, conversion rules, display order, and governance.
TimeWindow for causal delay	Value object	It describes a range and can validate start before end.
AlarmSeverity lookup	Entity / reference data	The system governs allowed severities and labels.

COMMON MISTAKE

Do not make every repeated value into a lookup table. Also do not hide governed reference data inside repeated strings. The question is not "is it repeated?" The question is "does it need identity, lifecycle, governance, or relationships?"

A small but useful TimeWindow

```
public readonly record struct TimeWindow(DateTime StartUtc, DateTime EndUtc)
{
    public TimeWindow
    {
        if (EndUtc <= StartUtc)
            throw new ArgumentException(
                "A time window must end after it starts.");
    }

    public bool Contains(DateTime instantUtc) =>
        instantUtc >= StartUtc && instantUtc <= EndUtc;
}
```

Value objects improve the language

The biggest benefit is not technical. The biggest benefit is language. A method that accepts SignalTag, TimeWindow, and ConfidenceScore says more than a method that accepts string, DateTime, DateTime, and decimal. The compiler now helps protect the vocabulary.

```
// Weak language: everything is anonymous.
void AddEvidence(string tag, DateTime from, DateTime to, decimal confidence)

// Better language: the signature carries the domain.
void AddEvidence(SignalTag tag, TimeWindow window, ConfidenceScore confidence)
```

Weak primitive	Better value object	Meaning gained
string	SignalTag	This string must identify a signal.
decimal	ConfidenceScore	This number must be between 0 and 1.
int	ReactivityNudge	This value is a rod-move suggestion in pcm.
DateTime + DateTime	TimeWindow	These two values belong together and must be ordered.
decimal + string	EngineeringValue	The number and its unit cannot drift apart.

HONEST BOUNDARY

Value objects are not a cure for bad modelling. If the team does not agree what a SignalTag, ConfidenceScore, or EngineeringValue means, wrapping primitives only hides confusion inside nicer types. Define the language first; encode it second.

NEXT BUILD

The next continuation should proceed with Chapter 25, From Domain Rule to Method, and Chapter 26, From Command to Application Service. Those chapters will connect domain language to behaviour and use-case orchestration.

CHAPTER 25

From Domain Rule to Method

A domain rule is a sentence the system must respect because the domain says so. It is not a validation decoration, not only an if statement, and not something that belongs wherever a developer first notices the problem.

The practical question is simple: when the rule matters, where should the code live? In a DDD design, the answer should follow ownership. If RootCause owns the meaning of a root-cause case, then RootCause should protect the rule for closing that case. If DigitalTwin owns divergence, then DigitalTwin should protect the rule for recording one.

PLAIN MEANING

A domain rule should live near the thing whose meaning it protects. A controller receives a request. A service coordinates a use case. The domain object or domain service protects the rule.

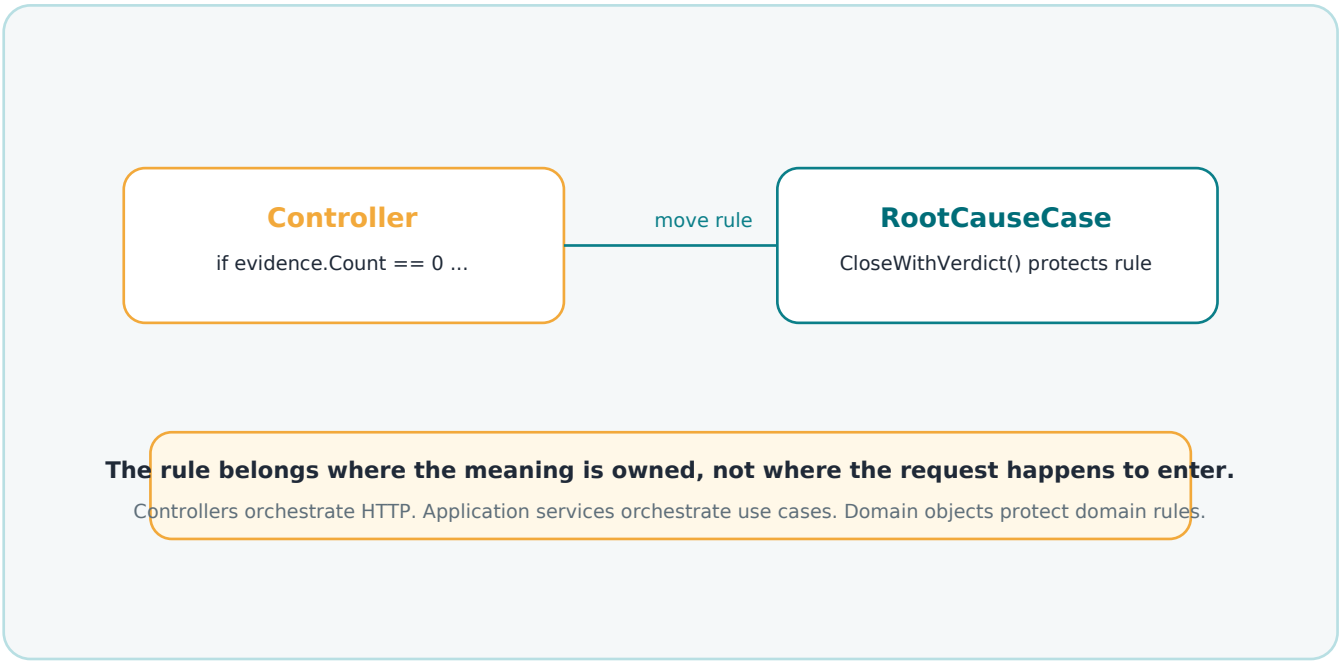


Figure 25.1 - Moving a rule from accidental orchestration into the domain model.

Rules are domain sentences

Before writing code, write the rule in human language. This keeps the model understandable for a mid-level developer and prevents the code from becoming a pile of technical checks with no meaning.

NEXUS-1 rule	Owned by	Good method name
A root-cause case cannot close without evidence.	RootCauseCase	CloseWithVerdict
A rejected hypothesis must remain visible in the case history.	RootCauseCase / Hypothesis	RejectHypothesis
A twin divergence must compare modelled and measured values using compatible units.	TwinDivergence / DigitalTwin service	RecordDivergence
An advisory recommendation must stay inside the validated band.	Policy / Recommendation	GenerateRecommendation
An alarm acknowledgement must record who acknowledged it and when.	AlarmEvent	Acknowledge

COMMON MISTAKE

Do not start by asking "which table is updated?" Start by asking "which concept is responsible for this rule?" Tables store facts. Domain methods protect meaning.

Example: closing a root-cause case

```
public void CloseWithVerdict(string verdict, string closedBy, DateTime closedAtUtc)
{
    if (Status != RootCauseCaseStatus.Open)
        throw new InvalidOperationException("Only an open case can be closed.");

    if (!_evidence.Any())
        throw new InvalidOperationException(
            "A root-cause case cannot be closed without evidence.");

    if (string.IsNullOrWhiteSpace(verdict))
        throw new ArgumentException("Verdict is required.");

    FinalVerdict = verdict;
    ClosedBy = closedBy;
    ClosedAtUtc = closedAtUtc;
    Status = RootCauseCaseStatus.Closed;
}
```

Not every rule belongs inside an entity

Sometimes a rule uses one aggregate and fits naturally in a method. Sometimes it needs several aggregates, a policy table, a calculation, or an external reading. In those cases, forcing the rule into one entity makes the model worse. DDD gives you domain services for rules that are domain rules but do not belong naturally to a single entity.

Rule shape	Good location	NEXUS-1 example
Rule uses one aggregate and its children.	Aggregate method.	RootCauseCase.CloseWithVerdict.
Rule compares multiple domain concepts.	Domain service.	CausalRankingService compares alarms, graph, and telemetry witnesses.
Rule coordinates persistence and security.	Application service.	OpenRootCauseCaseCommandHandler loads user, unit, and saves the new case.
Rule formats a screen or report.	Query/read model.	Fleet overview status summary.
Rule belongs to infrastructure.	Infrastructure service.	Sending email, writing files, calling external APIs.

Example: a small domain service

```
public sealed class CausalRankingService
{
    public RankedCause Rank(
        AlarmFlood flood,
        CausalGraph graph,
        TelemetryWindow telemetry)
    {
        var candidates = graph.FindCandidates(flood);
        var scored = candidates.Select(c => Score(c, flood, telemetry));

        return scored
            .OrderByDescending(x => x.CausalWeight)
            .First();
    }
}
```

SEE IT IN NEXUS-1

The Root Cause screen is a good example of a rule that is larger than one entity. The ranking is not a property setter. It is a domain calculation over alarm events, causal graph structure, timing windows, and telemetry witnesses.

Keep validation and domain rules separate

A DTO can check whether a field is present. A domain method checks whether an action makes sense. Both are useful, but they are not the same layer.

Question	Validation	Domain rule
What does it protect?	Input shape.	Meaning and consistency.
Example	Verdict text is required.	A case cannot close without evidence.
Where often located?	DTO validator, controller filter, request model.	Aggregate method or domain service.
Failure meaning	The request is malformed.	The requested action violates the domain.
Can it depend on current state?	Usually no, or lightly.	Often yes.

```
// Request validation: shape of the input.
public sealed class CloseRootCauseCaseRequest
{
    public string Verdict { get; init; } = string.Empty;
}

// Domain rule: meaning of the action.
case.CloseWithVerdict(request.Verdict, user.Name, clock.UtcNow);
```

HONEST BOUNDARY

DDD does not remove validation libraries such as FluentValidation. They still help at the edges. The warning is narrower: do not confuse request validation with the rules that give the domain its meaning.

CHAPTER 26

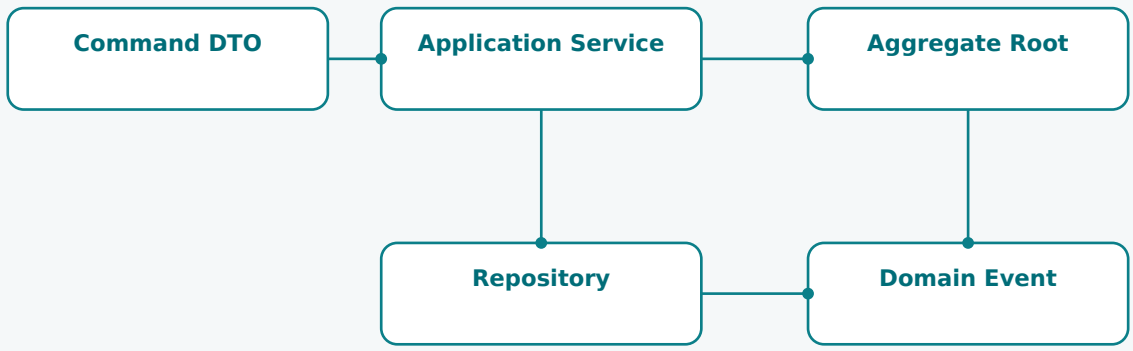
From Command to Application Service

A command is a request to change the system. An application service is the use-case coordinator that handles that request. It should not be a dumping ground for business rules. It should load what is needed, call the domain model, save the result, and publish any events.

This is especially important in NEXUS-1 because many actions look similar from the outside. Opening a root-cause case, acknowledging an alarm, recording a divergence, and generating a recommendation are all commands, but each belongs to a different context and follows different rules.

PLAIN MEANING

The application service is the conductor. It does not play every instrument. It tells the domain object when to play, asks repositories to load and save, and asks infrastructure to publish or notify.



Use case orchestration: load, call domain method, save, publish events. No business rule should be invented in the handler.

Figure 26.1 - A command handler coordinates the use case without stealing the domain rules.

Command names should be verbs

A command asks the system to do something. Its name should sound like an action, not like a table. A good command name is usually close to what the user or system intends.

Poor command name	Better command name	Reason
RootCauseCaseDto	OpenRootCauseCase	The command expresses intent.
UpdateAlarm	AcknowledgeAlarm	The action is specific.
SaveTwinRow	RecordTwinDivergence	The domain meaning is visible.
InsertRecommendation	GeneratePolicyRecommendation	The command is not just database insertion.
ChangeStatus	CloseRootCauseCase	The transition is named.

```
public sealed record CloseRootCauseCaseCommand(
    int RootCauseCaseId,
    string Verdict,
    string RequestedBy);

public sealed record RecordTwinDivergenceCommand(
    int TwinModelId,
    SignalTag SignalTag,
    EngineeringValue Modelled,
    EngineeringValue Measured,
    DateTime ObservedAtUtc);
```

COMMON MISTAKE

Do not name commands after CRUD operations when the domain has a better verb. "Update" hides meaning. "Acknowledge", "Close", "Reject", "Record", and "Generate" reveal meaning.

A good application service is boring

The application service should be easy to read. It should not contain a large decision tree of hidden business rules. Most of the time it does five things: authorize, load, call domain method, save, publish events.

```
public sealed class CloseRootCauseCaseHandler
{
    private readonly IRootCauseCaseRepository _cases;
    private readonly ICurrentUser _currentUser;
    private readonly IClock _clock;
    private readonly IDomainEventPublisher _events;

    public async Task Handle(CloseRootCauseCaseCommand command)
    {
        var user = _currentUser.RequireAuthenticated();

        var rootCauseCase = await _cases.GetAsync(command.RootCauseCaseId);
        if (rootCauseCase is null)
            throw new NotFoundException("Root-cause case not found.");

        rootCauseCase.CloseWithVerdict(
            command.Verdict,
            user.DisplayName,
            _clock.UtcNow);

        await _cases.SaveAsync(rootCauseCase);
        await _events.PublishAsync(rootCauseCase.DomainEvents);
    }
}
```

Step	Application service responsibility	Domain responsibility
Authorize	Check the caller may perform the use case.	Do not know HTTP or user tokens.
Load	Retrieve aggregate from repository.	Expose methods to protect state.
Act	Call one or more domain methods.	Enforce domain rules.
Save	Persist through repository/unit of work.	Remain persistence ignorant.
Publish	Dispatch domain events after save.	Raise meaningful facts.

Commands are not events

This distinction is important enough to repeat. A command asks for something. An event records that something happened. Mixing the two makes systems confusing and audit trails weak.

Command	Event after success	Context
AcknowledgeAlarm	AlarmAcknowledged	Alarm Management
OpenRootCauseCase	RootCauseCaseOpened	Root Cause
RejectHypothesis	HypothesisRejected	Root Cause
RecordTwinDivergence	TwinDivergenceRecorded	Digital Twin
GeneratePolicyRecommendation	PolicyRecommendationGenerated	Reinforcement Learning

```
// Command: a request.
public sealed record RejectHypothesisCommand(
    int RootCauseCaseId,
    HypothesisId HypothesisId,
    string Reason);

// Event: a fact after the command succeeds.
public sealed record HypothesisRejected(
    int RootCauseCaseId,
    HypothesisId HypothesisId,
    string Reason,
    DateTime RejectedAtUtc);
```

SEE IT IN NEXUS-1

When a rejected candidate appears in the root-cause explanation, it should be understood as a fact in the domain history. The command asked to reject it; the event records that it was rejected and why.

What application services should not do

Bad habit	Why it hurts	Better approach
Put all business rules in the handler.	Rules become duplicated and hard to test.	Move rules to aggregate methods or domain services.
Return EF entities directly to screens.	Write model becomes coupled to UI needs.	Use read models or DTOs for queries.
Let handlers update child rows directly.	Aggregate root is bypassed.	Load the root and call a method.
Publish events before the transaction succeeds.	Events may describe facts that were not committed.	Publish after successful save or use an outbox.
Use generic SaveEverything commands.	Intent disappears.	Use specific commands with domain verbs.

HONEST BOUNDARY

An application service is not automatically clean just because it uses DDD words. If it contains all decisions, all rules, all persistence details, and all formatting logic, it has become the same old service layer with better names. Keep it boring on purpose.

NEXT BUILD

The next continuation should proceed with Chapter 27, From Domain Event to Audit Entry, and Chapter 28, Repositories, DbContext, and Unit of Work. Those chapters will connect domain facts to auditability and persistence discipline.

CHAPTER 27

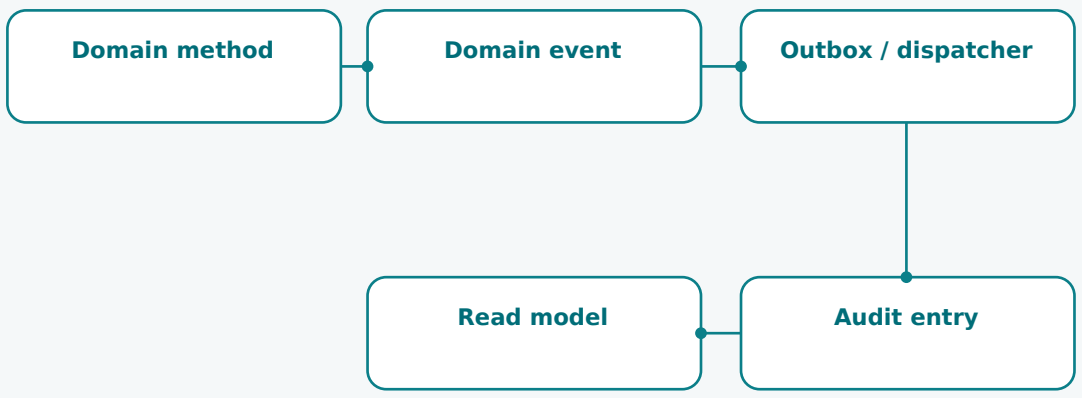
From Domain Event to Audit Entry

A domain event is a fact that happened in the domain. An audit entry is a durable record that proves important actions, state changes, and decisions happened. They are related, but they are not identical.

In NEXUS-1 this distinction matters because the system is not only trying to do work. It is also trying to remain explainable. A rejected hypothesis, a recorded twin divergence, or a generated recommendation should not disappear into a log line. It should become part of the system story.

PLAIN MEANING

A domain event says: this happened. An audit entry says: this happened, at this time, by this actor or process, with this evidence, and here is the record that lets someone check it later.



Events are domain facts. Audit entries are durable records prepared from those facts.

Figure 27.1 - A domain event can feed audit, read models, notifications, and reporting without becoming all of them.

Events are facts, not explanations

When the domain raises an event, the event should be small, clear, and past tense. It should not try to contain the whole report, the whole screen, or every possible side effect. It should record the important fact in the language of the bounded context.

Domain event	What it means	Possible audit meaning
AlarmAcknowledged	An alarm was acknowledged by a user.	Who acknowledged it, when, and from which station.
RootCauseCaseOpened	A diagnostic case started for a unit or incident.	The trigger, operator, and linked alarm flood.
HypothesisRejected	A candidate cause was rejected.	The reason, evidence, and rule used to reject it.
TwinDivergenceRecorded	The model and measurement disagreed.	Values, units, signal tag, and timestamp.
PolicyRecommendationGenerated	The advisory policy produced a recommendation.	State, action, clamp status, and policy version.

COMMON MISTAKE

Do not make a domain event a giant serialized copy of the whole aggregate. It should carry the fact and identifiers needed to follow the trail. Reports and read models can enrich it later.

Example: recording a hypothesis rejection

The aggregate raises a domain event when the rejection is accepted by the domain rules. The audit layer can then turn that event into a permanent audit entry.

```
public void RejectHypothesis(
    HypothesisId hypothesisId,
    string reason,
    string rejectedBy,
    DateTime rejectedAtUtc)
{
    var hypothesis = _hypotheses.Single(x => x.Id == hypothesisId);
    hypothesis.Reject(reason, rejectedBy, rejectedAtUtc);

    AddDomainEvent(new HypothesisRejected(
        RootCauseCaseId,
        hypothesisId,
        reason,
        rejectedBy,
        rejectedAtUtc));
}
```

```
public sealed record HypothesisRejected(
    int RootCauseCaseId,
    HypothesisId HypothesisId,
    string Reason,
    string RejectedBy,
    DateTime RejectedAtUtc);
```

SEE IT IN NEXUS-1

In the Root Cause context, rejected candidates are not garbage. They are evidence of reasoning. The domain event records that a candidate was rejected; the audit entry makes that rejection reviewable.

Audit entries are not normal logs

A normal technical log helps developers diagnose runtime behaviour. An audit entry helps a later reader reconstruct important domain actions. It should be structured, queryable, and connected to the domain object it describes.

Concern	Technical log	Audit entry
Purpose	Debugging and operations.	Accountability and reconstruction.
Shape	Often text-oriented.	Structured domain record.
Retention	May be short or environment-dependent.	Usually longer and governed.
Examples	SQL timeout, HTTP 500, retry count.	Case closed, hypothesis rejected, divergence recorded.
Reader	Developer or operator.	Engineer, reviewer, regulator, future maintainer.

```
public sealed class AuditEntry
{
    public int AuditEntryId { get; private set; }
    public string ContextName { get; private set; } = null!;
    public string EntityName { get; private set; } = null!;
    public string EntityKey { get; private set; } = null!;
    public string Action { get; private set; } = null!;
    public string Actor { get; private set; } = null!;
    public DateTime OccurredAtUtc { get; private set; }
    public string? EvidenceJson { get; private set; }
}
```

HONEST BOUNDARY

Not every domain event must become an audit entry. Some events are useful only for internal read-model updates. But events that affect diagnosis, accountability, safety posture, recommendations, or compliance should be audit candidates.

CHAPTER 28

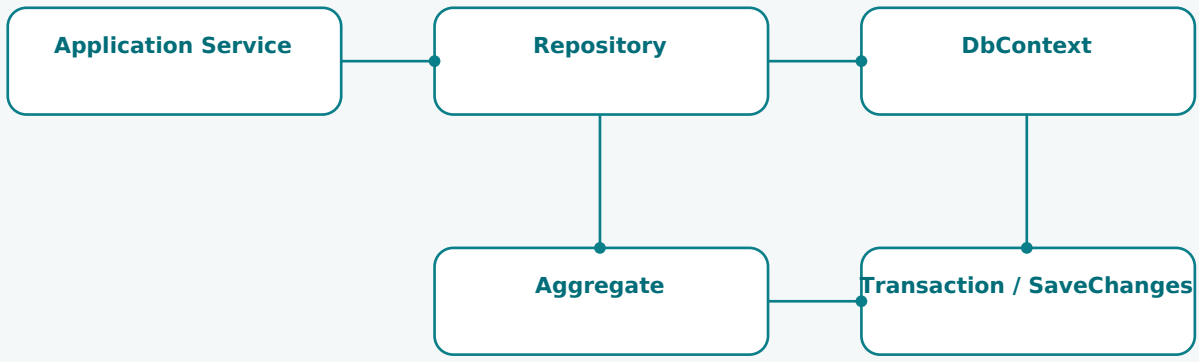
Repositories, DbContext, and Unit of Work

Repositories are often misunderstood because many tutorials make them look like unnecessary wrappers around EF Core. In DDD, the repository has a narrower purpose: it gives the application layer a collection-like way to load and save aggregates without making the domain model depend on persistence details.

NEXUS-1 already has a strong database and EF Core story. The point of this chapter is not to hide SQL Server or pretend DbContext does not exist. The point is to keep the domain model clean while still letting EF Core do the persistence work it is good at.

PLAIN MEANING

A repository is not a magic data-access layer. It is the doorway through which application services load and save aggregate roots. The DbContext can implement that doorway, but the domain model should not know it exists.



Repository gives collection-like access to aggregates; DbContext performs the actual persistence work.

Figure 28.1 - Repository and DbContext working together without making the domain object persistence-aware.

Repository per aggregate root, not per table

A common mistake is to create one repository for every table. That is database thinking, not domain thinking. In a DDD model, repositories normally serve aggregate roots. Child entities are reached through the root because the root protects the consistency boundary.

Aggregate root	Repository	Do not create repository for
RootCauseCase	IRootCauseCaseRepository	Evidence, Hypothesis, RejectedCandidate as separate public repositories.
TwinModel	ITwinModelRepository	SignalBinding and TwinSnapshot as independent mutation paths.
AlarmFlood	IAlarmFloodRepository	AlarmEvent children that bypass flood lifecycle rules.
Policy	IPolicyRepository	QTableEntry as random public table access.
Unit	IUnitRepository	Small component rows when the use case needs the unit boundary.

COMMON MISTAKE

If every table gets a repository, the aggregate boundary disappears. The application service can start changing children directly, and the root no longer protects the rules.

Example repository interface

The interface should speak the language of the aggregate, not the language of arbitrary SQL operations. It should expose operations the application layer genuinely needs.

```
public interface IRootCauseCaseRepository
{
    Task<RootCauseCase?> GetAsync(
        int rootCauseCaseId,
        CancellationToken cancellationToken = default);

    Task AddAsync(
        RootCauseCase rootCauseCase,
        CancellationToken cancellationToken = default);

    Task SaveAsync(
        RootCauseCase rootCauseCase,
        CancellationToken cancellationToken = default);
}
```

```
public sealed class EfRootCauseCaseRepository : IRootCauseCaseRepository
{
    private readonly NexusContext _db;

    public Task<RootCauseCase?> GetAsync(int id, CancellationToken ct = default) =>
        _db.RootCauseCases
           .Include(x => x.Hypotheses)
           .Include(x => x.Evidence)
           .SingleOrDefaultAsync(x => x.RootCauseCaseId == id, ct);

    public async Task AddAsync(RootCauseCase rootCauseCase, CancellationToken ct = default) =>
        await _db.RootCauseCases.AddAsync(rootCauseCase, ct);

    public Task SaveAsync(RootCauseCase rootCauseCase, CancellationToken ct = default) =>
        _db.SaveChangesAsync(ct);
}
```

DbContext already behaves like a unit of work

EF Core DbContext tracks changes and commits them with SaveChanges. That is already a unit-of-work pattern in practical .NET terms. You do not always need to build a second heavy abstraction called UnitOfWork on top of it. What you need is a clear transaction boundary.

Question	Practical answer
Does the domain entity need DbContext?	No. Domain objects should not depend on EF Core.
Can the repository use DbContext?	Yes. It is infrastructure code.
Do we always need a custom UnitOfWork class?	No. DbContext often already provides the unit-of-work behaviour.
When might a custom unit of work help?	When coordinating several repositories, outbox messages, or transaction policies.
Where should SaveChanges happen?	At the application-service boundary, after domain methods succeed.

```
public async Task Handle(RecordTwinDivergenceCommand command)
{
    var twin = await _twins.GetAsync(command.TwinModelId);
    twin.RecordDivergence(command.SignalTag, command.Modelled,
        command.Measured, command.ObservedAtUtc);

    await _db.SaveChangesAsync();       // transaction boundary
    await _outbox.EnqueueAsync(twin.DomainEvents);
}
```

HONEST BOUNDARY

DDD does not require hiding EF Core from the whole application. It requires keeping persistence concerns out of the domain model. The infrastructure layer may know EF Core very well; the aggregate should not.

Queries are different from repositories

A repository loads aggregates for behaviour. Queries often load read models for screens and reports. Mixing these two needs makes repositories bloated and slow. It is usually better to allow separate query services or read models for reporting screens.

Use case	Good tool	Reason
Close a root-cause case.	Repository loads RootCauseCase aggregate.	The domain object must protect rules.
Show last 50 alarms.	Query service / read model.	No aggregate behaviour is needed.
Render fleet dashboard.	Read model.	Optimized shape for screen.
Generate compliance trace.	Query + audit tables.	Historical reconstruction, not mutation.
Record a twin divergence.	Repository loads TwinModel.	The twin model owns the rule.

SEE IT IN NEXUS-1

The Fleet Overview, Root Cause Graph, and Optimization screens do not need to load every aggregate in full just to draw a dashboard. DDD allows separate read models because not every read is a behavioural operation.

NEXT BUILD

The next continuation should proceed with Chapter 29, Queries, Read Models, and Reporting Screens, and Chapter 30, Final Architecture: Domain, Data, Twin, and Decision. That should complete Part III and prepare the back matter/glossary.

CHAPTER 29

Queries, Read Models, and Reporting Screens

Not every screen needs an aggregate. This is one of the practical points that makes DDD less frightening for a working developer. Aggregates protect behaviour. Queries present information. When you confuse those two jobs, your repositories become too large, your entities become too slow to load, and your screens start depending on objects that were designed for rules rather than display.

NEXUS-1 has many screens: fleet overview, root-cause graph, alarms, digital twin, optimization, audit, compliance, robotics, and reporting. Some operations must load a real aggregate and enforce rules. Other screens simply need a shaped, fast, read-only view of state. DDD allows that separation.

PLAIN MEANING

A command asks the system to change something. A query asks the system to show something. A command normally goes through behaviour and rules. A query may go directly to a read model built for the screen.

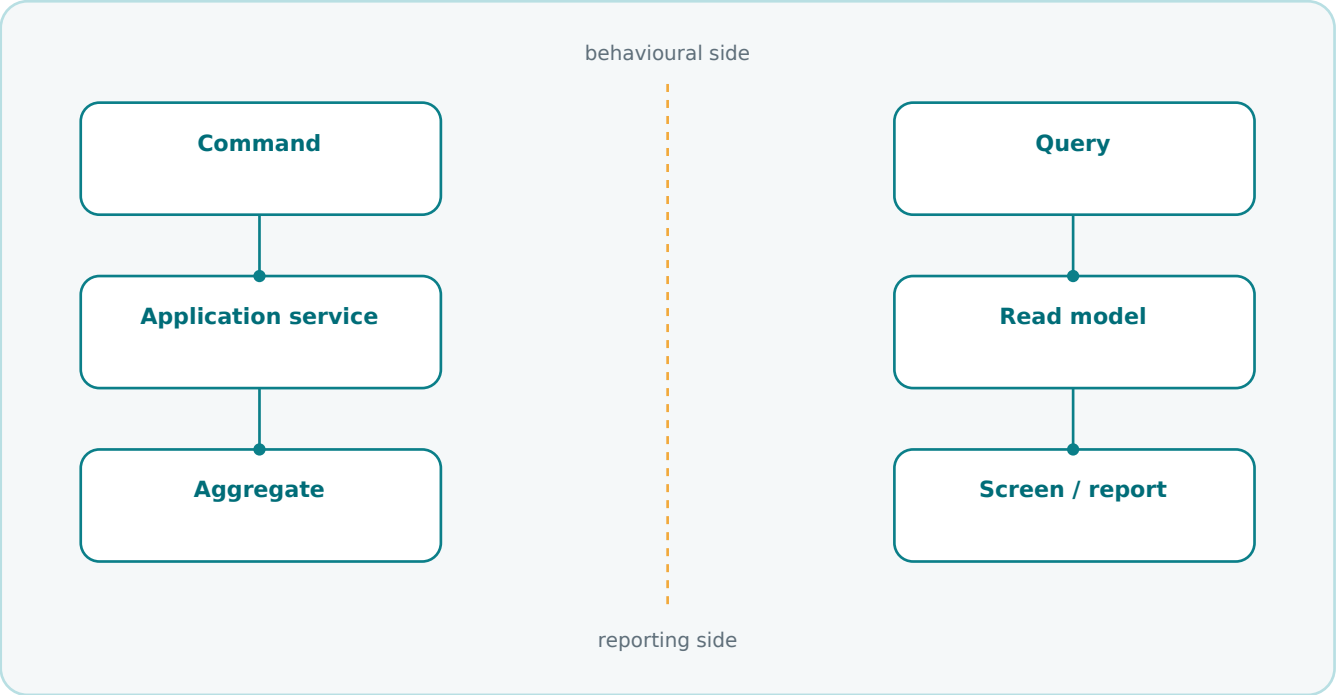


Figure 29.1 - Commands protect behaviour; queries shape information for screens and reports.

Why read models exist

A read model is not a weaker domain model. It is a different tool. The domain model is optimized for correctness and behaviour. The read model is optimized for answering a question quickly and clearly.

Need	Good model	Why
Close a root-cause case.	Aggregate model	Rules must be enforced before state changes.
Show current fleet status.	Read model	The screen needs a fast summary, not full unit behaviour.
Record a twin divergence.	Aggregate model	The TwinModel owns the meaning of divergence.
List open compliance findings.	Read model	Filtering and sorting do not require a domain aggregate.
Generate an operator dashboard.	Read model	The shape belongs to the screen, not the domain.

COMMON MISTAKE

A common mistake is to load a large aggregate only to display a table. That makes the domain model responsible for reporting. Keep behavioural writes and reporting reads separate when the use case is clearly different.

A NEXUS-1 dashboard query

A fleet dashboard may need unit code, power, current alarms, and latest divergence. That is a reporting shape. It is acceptable to build it with a query service or SQL view.

```
public sealed record FleetStatusRow(
    string UnitCode,
    decimal PowerMwe,
    int OpenAlarmCount,
    decimal? LatestDivergence,
    DateTime UpdatedAtUtc);

public interface IFleetStatusQuery
{
    Task<IReadOnlyList<FleetStatusRow>> GetAsync(
        CancellationToken cancellationToken = default);
}
```

```
SELECT
    u.Code              AS UnitCode,
    t.PowerMwe         AS PowerMwe,
    COUNT(a.AlarmEventId) AS OpenAlarmCount,
    MAX(d.AbsoluteError)  AS LatestDivergence
FROM ReactorFleet.Unit u
LEFT JOIN DigitalTwin.TwinSnapshot t ON t.UnitId = u.UnitId
LEFT JOIN AlarmManagement.AlarmEvent a ON a.UnitId = u.UnitId
LEFT JOIN DigitalTwin.TwinDivergence d ON d.UnitId = u.UnitId
GROUP BY u.Code, t.PowerMwe;
```

SEE IT IN NEXUS-1

The Fleet Overview screen is naturally a read-model screen. It presents a cross-context view. It should not pretend to be the owner of Unit, AlarmEvent, TwinSnapshot, or TwinDivergence.

Reporting is allowed to cross contexts

Earlier chapters were strict about bounded contexts because writes need ownership. Reporting is different. A report may join across contexts because its job is to explain what happened. The rule is that reporting must not become a hidden write path.

Operation	May cross contexts?	May change domain state?
Fleet dashboard query	Yes	No
Compliance trace report	Yes	No
Root-cause case closure	Usually no	Yes, through the RootCauseCase aggregate
Alarm acknowledgement	Uses passports	Yes, through Alarm Management rules
Audit reconstruction	Yes	No

HONEST BOUNDARY

CQRS does not have to mean two databases, event sourcing, or a large framework. At the level this book uses it, the idea is modest: separate the model used to change state from the model used to read state when the two have different needs.

CHAPTER 30

Final Architecture: Domain, Data, Twin, and Decision

The whole book has been moving toward one simple architectural picture. NEXUS-1 is not only a set of pages and not only a schema. It is a domain language, expressed in code, persisted in a database, reflected in a digital twin, and used by diagnostic and advisory engines.

DDD does not replace the database book, the EF Core atlas, the root-cause book, or the reinforcement-learning book. It gives them a shared meaning. It says which words belong where, which object protects each rule, which context owns each truth, and where a recommendation stops being a command.

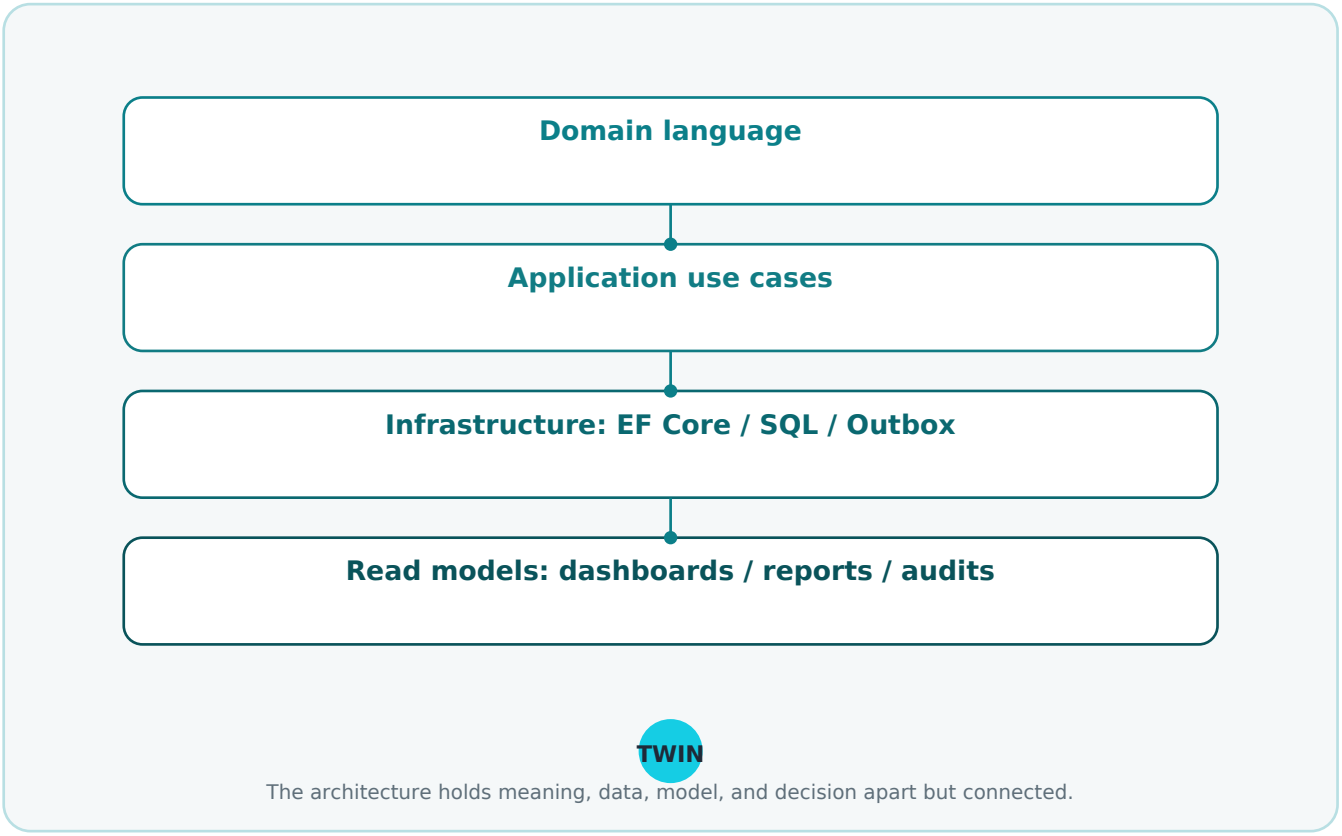


Figure 30.1 - The final NEXUS-1 DDD architecture: language, use cases, persistence, read models, and twin held in one disciplined shape.

The final map in plain language

The final architecture can be read without special vocabulary. The domain layer gives names to the important ideas. The application layer coordinates use cases. The infrastructure layer stores and publishes state. The read side makes dashboards and reports efficient. The digital twin binds the model to measured reality.

Layer or concern	NEXUS-1 meaning	Typical implementation
Domain language	Unit, Signal, AlarmEvent, RootCauseCase, TwinModel, Policy.	C# entities, value objects, domain events.
Application use cases	Acknowledge alarm, close case, record divergence.	Command handlers and application services.
Persistence	Keep the model durable and queryable.	EF Core, SQL Server schemas, migrations.
Read models	Show dashboards and reports.	SQL views, query services, DTOs.
Audit and outbox	Make important facts durable and publishable.	AuditEntry, OutboxMessage, domain-event handlers.
Digital twin	Compare model state with measured state.	TwinModel, SignalBinding, TwinSnapshot, TwinDivergence.

THE CENTRAL SENTENCE

A Unit is not a string, a Signal is not an alarm, an AlarmEvent is not a cause, a Hypothesis is not a verdict, a TwinModel is not the plant, and a Recommendation is not a command. This is the practical value DDD gives to NEXUS-1.

What DDD added to NEXUS-1

DDD did not add nuclear physics, machine learning, or database technology. It added boundaries and language. That may sound modest, but in a system like NEXUS-1 it is exactly what prevents the platform from becoming a pile of clever screens.

Before DDD thinking	After DDD thinking
Screens show related data.	Contexts own meanings and rules.
Tables store many facts.	Aggregates protect consistency boundaries.
Services do whatever is needed.	Application services coordinate use cases.
Logs record technical messages.	Audit entries preserve domain accountability.
The RL result looks like an action.	The policy produces an advisory recommendation.
The twin looks like the plant.	The twin is a model bound to measurements.

COMMON MISTAKE

Do not use DDD to make the system sound more important than it is. DDD is not decoration. If the boundaries are not enforced and the language is not used consistently, the terminology becomes theatre.

How the companion volumes now fit together

This book becomes the semantic layer of the NEXUS-1 companion series. It explains the meaning of the system that the other books explain from the viewpoints of physics, diagnosis, learning, database design, and EF Core configuration.

Volume	Main question answered
From Grid to Core	How does the demonstrator represent the plant and its behaviour?
From Flood to Cause	How does the system collapse alarms into a grounded cause?
From Trial to Policy	How does the advisory learning agent produce a readable policy?
From Table to Twin	Where does the data backbone live?
From Entity to Context	How does EF Core map the schema cleanly?
From Domain to Twin	What does the system mean, who owns each word, and where do the rules belong?

HONEST BOUNDARY

This book is not a theoretical contribution to DDD. It is a practical DDD learning path and architecture reading of NEXUS-1. Its value is in making the demonstrator easier to understand, extend, test, and explain.

PART III CHECKPOINT

What the implementation part has built

Part III translated the model into practical .NET architecture. It connected bounded contexts to project structure and SQL schemas, aggregates to C# entities, value objects to owned types, rules to methods, commands to application services, events to audit, repositories to DbContext, and queries to read models.

DDD idea	Implementation habit in NEXUS-1
Bounded context	Project folder, namespace, SQL schema, explicit passport.
Aggregate root	Public mutation path for important consistency rules.
Value object	Immutable type, often mapped as an owned type.
Domain rule	Method on the object that owns the rule.
Command	Application-service request to change state.
Domain event	Past-tense fact that can feed audit and outbox.
Repository	Aggregate-focused loading and saving.
Read model	Screen- or report-shaped query object.

NEXT BUILD

The next continuation should add the back matter: glossary of DDD terms, NEXUS-1 language checklist, aggregate checklist, bounded-context checklist, and a short epilogue.

PART I DEEPENING

Expanded Chapters 6-10

Additional explanations, examples, and diagrams for the school-style DDD foundation

These pages deepen the first part of the book before the final back matter. They expand Chapters 6, 7, 8, and 9 with more human-language explanations, more NEXUS-1 examples, and more implementation guidance. They then add a Chapter 10 case study that uses every major DDD term introduced in Part I.

Expanded topic	Purpose
Repositories and persistence ignorance	Clarifies what a repository is, what it is not, and why EF Core should not leak into the domain model.
Domain services and application services	Separates business meaning from use-case coordination.
Domain events	Shows how facts that happened in the domain can feed audit, projections, and integration.
Bounded contexts	Explains language boundaries, ownership, and safe crossing between contexts.
Chapter 10 case study	Uses a single NEXUS-1 alarm-to-root-cause scenario to connect all Part I DDD terms.

READING NOTE

This section is intentionally placed as a visible deepening block so the reader can first finish the basic Part I flow, then return to a richer explanation of the same concepts before applying them to NEXUS-1.

CHAPTER 6 - EXPANDED

Repositories and Persistence Ignorance

A repository is one of the most misunderstood DDD patterns because it looks familiar. Many developers see it and think: this is just another CRUD class around DbContext. That is not the real idea.

In DDD, a repository gives the application a collection-like way to load and save aggregate roots. It hides persistence details from the domain model. The domain should not know whether the data came from SQL Server, a document store, a test double, or an in-memory list.

HUMAN EXPLANATION

Persistence ignorance means the domain model does not think like a database. A RootCauseCase should know how to accept evidence, reject a hypothesis, and close with a verdict. It should not know how to build an EF Core Include, what table its evidence lives in, or whether the RowVersion column is binary.

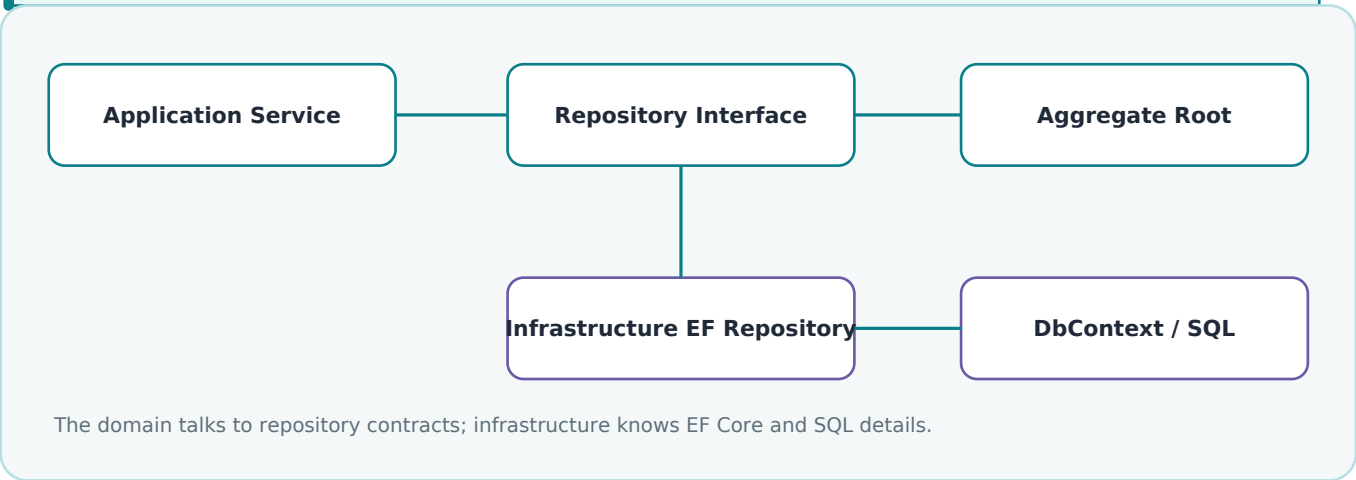


Figure 6.1 - Repository contracts sit between use cases and persistence. The aggregate stays focused on domain behavior.

The repository is not the owner of business rules

The repository loads and saves. It may enforce persistence concerns such as tracking, concurrency, and query shape. But it should not decide domain meaning. If the rule is "a root-cause case cannot be closed without evidence", that rule belongs in RootCauseCase, not in RootCauseCaseRepository.

Bad placement	Why it hurts	Better placement
Repository decides if a case may close.	Business rule hides inside infrastructure; tests need database setup.	RootCauseCase.CloseWithVerdict( ) enforces the rule.
Repository returns all tables as IQueryable.	Callers build accidental domain logic outside the model.	Repository exposes intention-revealing methods.
Generic CRUD repository for every table.	Treats aggregates, child rows, lookups, and audit entries as equal.	Repositories are designed around aggregate roots.

```
public interface IRootCauseCaseRepository
{
    Task<RootCauseCase?> GetByIdAsync(RootCauseCaseId id, CancellationToken ct);
    Task AddAsync(RootCauseCase rootCauseCase, CancellationToken ct);
    Task SaveAsync(RootCauseCase rootCauseCase, CancellationToken ct);
}

// Notice what is missing: no AddEvidenceRow(), no UpdateVerdictColumn(),
// no exposed DbSet, and no IQueryable leaking outside the boundary.
```

NEXUS-1 repository examples

NEXUS-1 has several possible repositories, but not every table deserves one. DDD repositories usually focus on aggregate roots. Lookup tables, projections, audit entries, and read models can be handled differently.

Aggregate root	Repository name	Typical methods
Unit	IUnitRepository	GetByCodeAsync, GetWithPlantTreeAsync
RootCauseCase	IRootCauseCaseRepository	GetByIdAsync, AddAsync, SaveAsync
TwinModel	ITwinModelRepository	GetForUnitAsync, SaveSnapshotAsync
TrainingRun or Policy	IPolicyRepository	GetActivePolicyAsync, SaveTrainingRunAsync
AuditEntry	Usually not aggregate repository	Append through audit service or event handler

COMMON MISTAKE

A repository per table looks organized, but it often recreates the database in code. DDD asks a different question: which object protects consistency? That object is the usual repository boundary.

A practical EF Core implementation

```
public sealed class EfRootCauseCaseRepository : IRootCauseCaseRepository
{
    private readonly NexusContext _db;

    public EfRootCauseCaseRepository(NexusContext db) => _db = db;

    public Task<RootCauseCase?> GetByIdAsync(RootCauseCaseId id, CancellationToken ct)
        => _db.RootCauseCases
              .Include(x => x.Hypotheses)
              .Include(x => x.Evidence)
              .SingleOrDefaultAsync(x => x.Id == id, ct);

    public async Task AddAsync(RootCauseCase rootCauseCase, CancellationToken ct)
        => await _db.RootCauseCases.AddAsync(rootCauseCase, ct);

    public Task SaveAsync(RootCauseCase rootCauseCase, CancellationToken ct)
        => Task.CompletedTask; // Unit of Work commits through DbContext.SaveChangesAsync()
}
```

HONEST BOUNDARY

DDD repositories do not remove the need to understand EF Core. They only keep EF Core details from becoming the language of the domain. A developer still needs to choose query shapes, indexes, includes, tracking behavior, and concurrency strategy carefully.

CHAPTER 7 - EXPANDED

Domain Services and Application Services

The word service is dangerous because it is used for almost everything. In many projects, every class that does work becomes SomethingService. DDD is more precise. It separates services that express domain logic from services that coordinate a use case.

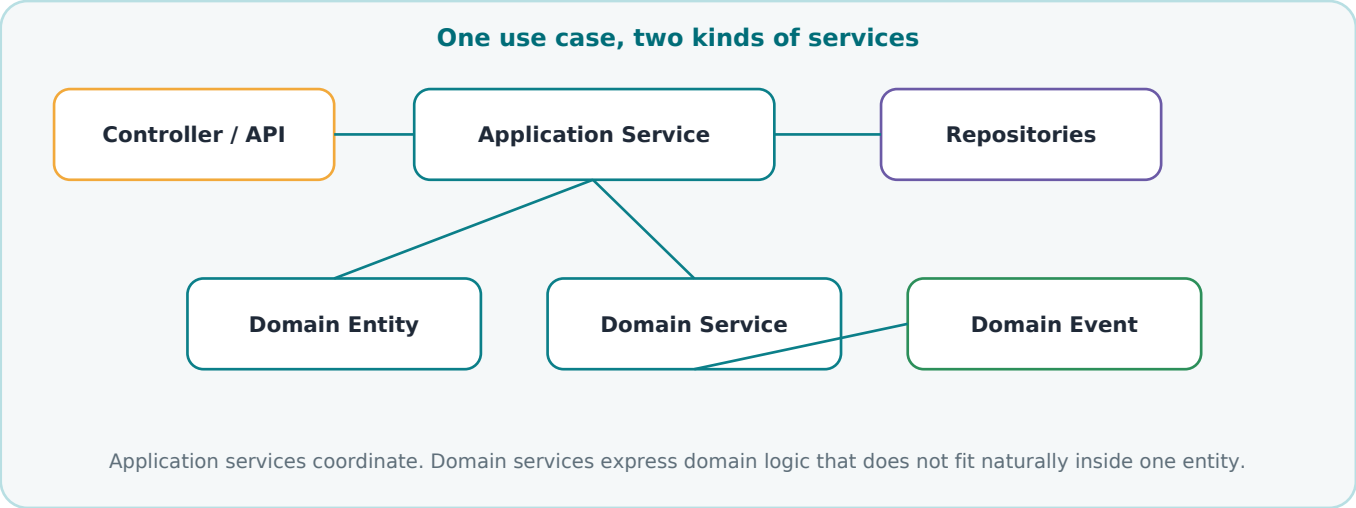


Figure 7.1 - Application services coordinate a use case. Domain services express domain knowledge when no single entity naturally owns the operation.

Service type	Plain meaning	NEXUS-1 example
Application service	Coordinates a use case: load, call domain logic, save, publish events.	OpenRootCauseCaseApplicationService
Domain service	A domain calculation or decision that does not belong naturally inside one entity.	CausalRankingService, DivergenceClassifier
Infrastructure service	Talks to external systems, files, telemetry, email, clock, message bus.	TelemetryReader, AuditWriter, NotificationSender

Application service example

Suppose the UI asks to open a root-cause case from an alarm flood. The application service should not contain the causal reasoning itself. Its job is to coordinate the use case: load the alarm flood, ask the domain to create the case, persist it, and publish the event.

```
public sealed class OpenRootCauseCaseApplicationService
{
    private readonly IAlarmFloodRepository _floods;
    private readonly IRootCauseCaseRepository _cases;
    private readonly ICausalRankingService _ranking;
    private readonly IUnitOfWork _unitOfWork;

    public async Task<RootCauseCaseId> Handle(OpenRootCauseCase command, CancellationToken ct)
    {
        var flood = await _floods.GetByIdAsync(command.AlarmFloodId, ct);
        if (flood is null) throw new InvalidOperationException("Alarm flood not found.");

        var candidates = _ranking.RankCandidates(flood);
        var rootCauseCase = RootCauseCase.Open(flood.Id, candidates);

        await _cases.AddAsync(rootCauseCase, ct);
        await _unitOfWork.SaveChangesAsync(ct);
        return rootCauseCase.Id;
    }
}
```

Domain service example

A domain service is useful when the rule is real domain knowledge but does not belong inside one entity. In NEXUS-1, causal ranking may need the flood, graph edges, timing windows, and evidence. Putting all of that inside AlarmFlood would make AlarmFlood too large. Putting it in the application service would hide domain reasoning inside workflow code.

```
public interface ICausalRankingService
{
    IReadOnlyList<CandidateCause> RankCandidates(AlarmFlood flood);
}

public sealed class CausalRankingService : ICausalRankingService
{
    public IReadOnlyList<CandidateCause> RankCandidates(AlarmFlood flood)
    {
        // Domain-level calculation: it speaks in alarms, nodes,
        // candidates, timing windows, and causal weight.
        // It does not save to the database and does not call the UI.
        return flood.AlarmedNodes
            .Select(node => CandidateCause.FromNode(node))
            .OrderByDescending(candidate => candidate.CausalWeight)
            .ToList();
    }
}
```

Question	Application service answer	Domain service answer
Does it know HTTP or UI details?	It may receive commands from API/controllers.	No. It should be pure domain language.
Does it save changes?	Usually yes, through unit of work.	No. It calculates or decides.
Does it contain business meaning?	It coordinates business work.	It expresses domain rule or calculation.
Can it be unit tested without database?	Usually with repository mocks/fakes.	Ideally yes, with plain objects.

COMMON MISTAKE

Do not put all business logic into application services because they are easy to call from controllers. That creates procedural transaction scripts. The domain model becomes a bag of properties again.

HONEST BOUNDARY

Not every operation needs a domain service. If a rule belongs naturally to an entity or aggregate root, put it there. Domain services are for domain behavior that is real, important, and awkward to place inside one object.

CHAPTER 8 - EXPANDED

Domain Events

A domain event is a fact that happened in the domain. It is written in the past tense because it already happened. This simple grammar helps prevent a common confusion: commands ask the system to do something; events record that something occurred.

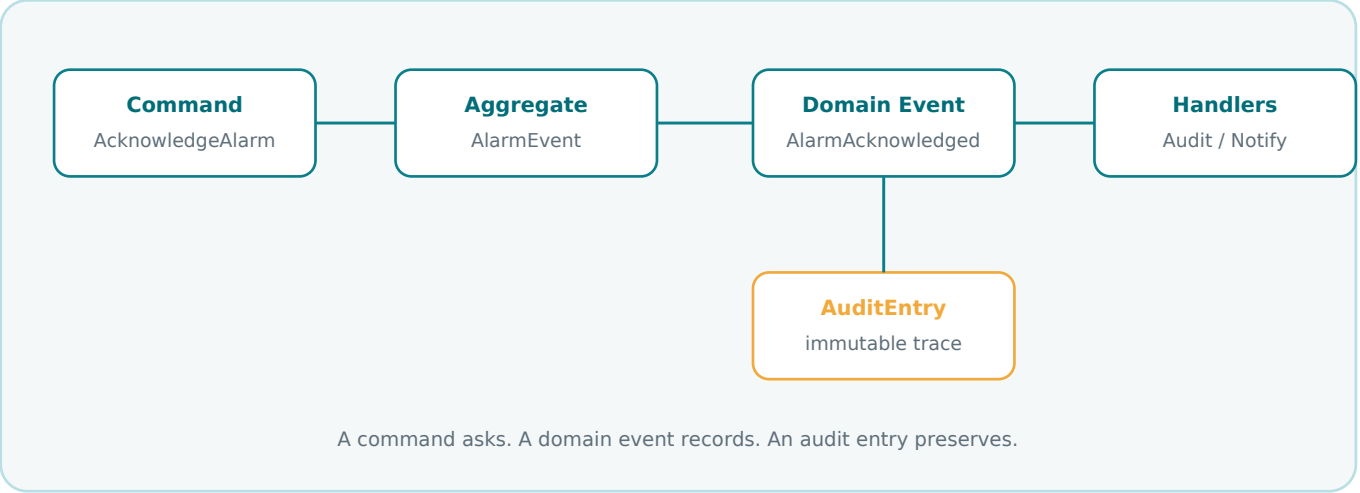


Figure 8.1 - Commands, aggregate decisions, domain events, handlers, and audit entries have different roles.

Command	Domain event	Meaning difference
AcknowledgeAlarm	AlarmAcknowledged	Ask versus fact.
OpenRootCauseCase	RootCauseCaseOpened	Request versus recorded domain occurrence.
RecordTwinSnapshot	TwinSnapshotRecorded	Action request versus captured result.
GenerateRecommendation	RecommendationGenerated	A calculation was requested versus advice exists.

Events should use domain language

A weak event name says something technical, such as RowInserted or StatusChanged. A strong domain event says what happened in the domain: HypothesisRejected, TwinDivergenceRecorded, PolicyRecommendationGenerated. The stronger name tells future readers why the fact matters.

```
public sealed record HypothesisRejected(
    RootCauseCaseId CaseId,
    HypothesisId HypothesisId,
    string Reason,
    DateTime OccurredAtUtc);

public sealed record TwinDivergenceRecorded(
    TwinModelId TwinModelId,
    string SignalTag,
    decimal ModelValue,
    decimal MeasuredValue,
    DateTime ObservedAtUtc);
```

How events help NEXUS-1

Events let one context announce a fact without forcing every other context to be part of the same object model. Alarm Management can publish AlarmFloodDetected. Root Cause can react by opening a case. Audit can append a trace. Reporting can update a read model. The contexts remain separate, but the system still moves.

Event	Owner context	Possible subscribers
AlarmFloodDetected	Alarm Management	Root Cause, Audit, Reporting
HypothesisRejected	Root Cause	Audit, Reporting
TwinDivergenceRecorded	Digital Twin	Root Cause, Audit, Reporting
PolicyRecommendationGenerated	Reinforcement Learning	Audit, Operator Console, Reporting
ComplianceFindingOpened	Compliance	Audit, Reporting

A small event handler

```
public sealed class AuditHypothesisRejectedHandler
{
    private readonly IAuditWriter _audit;

    public Task Handle(HypothesisRejected e, CancellationToken ct)
        => _audit.AppendAsync(new AuditEntry(
            area: "RootCause",
            action: "HypothesisRejected",
            entityId: e.CaseId.Value,
            description: e.Reason,
            occurredAtUtc: e.OccurredAtUtc), ct);
}
```

COMMON MISTAKE

Do not use domain events as a hidden command bus. If the event name is DoSomething, StartSomething, or UpdateSomething, it is probably a command wearing event clothing.

HONEST BOUNDARY

Domain events are not automatically reliable integration. In a real distributed system, delivery, retries, ordering, idempotency, and the outbox pattern matter. This book uses events first to teach domain meaning, then later connects them to audit and persistence.

CHAPTER 9 - EXPANDED

Bounded Contexts

A bounded context is a boundary where a model and its language are valid. It is not just a folder. It is not automatically a microservice. It is a promise: inside this boundary, words mean what this context says they mean.

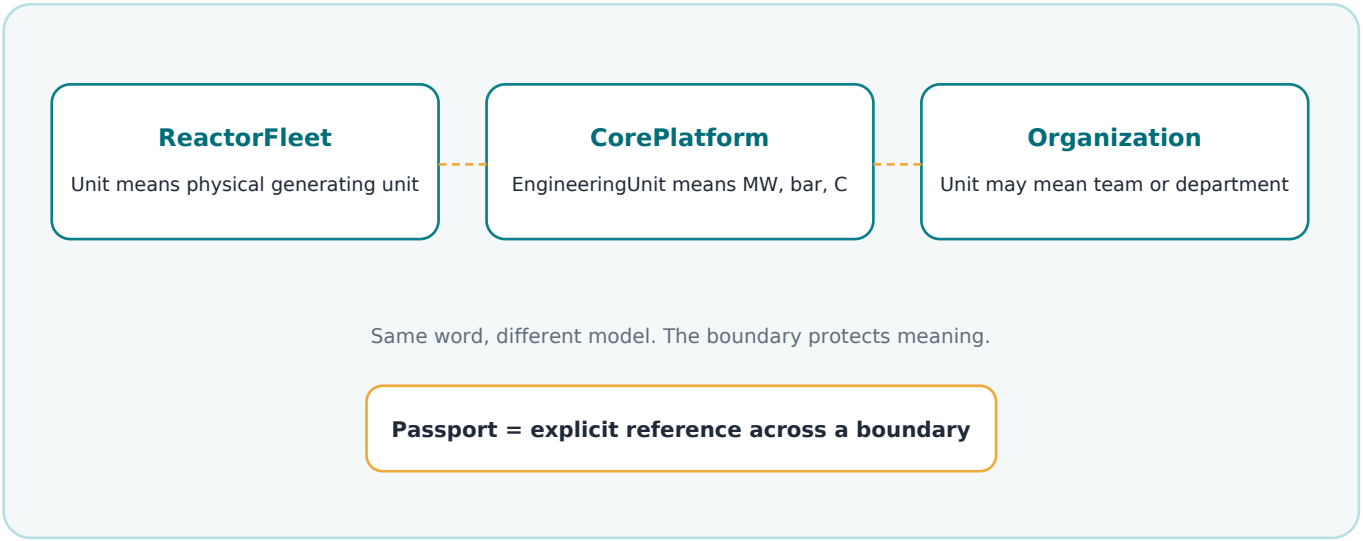


Figure 9.1 - The same word can be safe only when its context is clear.

HUMAN EXPLANATION

A bounded context is like a room in a building. Inside the control room, "unit" may mean a physical generating unit. Inside a measurement catalogue, "unit" may mean MW or bar. Inside an organization chart, "unit" may mean a department. The wall prevents the words from colliding.

Context boundaries in code and SQL

In NEXUS-1, bounded contexts appear in several places. At the language level, each context owns a vocabulary. At the C# level, the code can use namespaces and folders. At the database level, the SQL schema can make the boundary visible. At the integration level, passports and events cross the boundary deliberately.

Boundary form	Developer view	NEXUS-1 example
Language boundary	Different words or different meanings of same word.	AlarmEvent is not RootCause.
C# boundary	Namespace, folder, project, interfaces.	Nexus.Domain.RootCause
Database boundary	SQL schema and declared foreign keys.	RootCause.RootCauseCase
Integration boundary	Commands, events, APIs, shared identifiers.	AlarmFloodDetected event, UnitId passport

What a context owns

A bounded context should own its truth. ReactorFleet owns physical Unit identity. Instrumentation owns signal tags and readings. Alarm Management owns alarm lifecycle. Root Cause owns hypotheses, evidence, rejected candidates, and verdicts. Reporting may display all of them, but it does not own the truth of all of them.

Context	Owns	Must not decide
ReactorFleet	Plant, Unit, Reactor, physical asset identity.	Whether an alarm is the root cause.
Instrumentation	Signal, Tag, Reading, EngineeringValue.	Whether a reading is a diagnosis.
Alarm Management	AlarmEvent, AlarmFlood, severity, acknowledgement.	The final causal verdict.
Root Cause	RootCauseCase, Hypothesis, Evidence, Verdict.	Whether the RL policy may command plant hardware.
Reinforcement Learning	Policy, QTableEntry, Recommendation.	Whether advice bypasses operator approval.
Reporting	Read models and projections.	Original domain truth.

COMMON MISTAKE

Do not let a reporting screen become the owner of everything it displays. A dashboard can join facts from many contexts, but ownership remains in the source contexts.

Crossing a boundary carefully

When one context needs another context, it should cross through an explicit contract: a shared identifier, an event, a query model, or an adapter. This is the meaning of the NEXUS-1 "passport" metaphor. A UnitId lets Alarm Management point to a physical Unit without becoming the owner of Unit.

```
// Alarm Management may reference a Unit by passport.
public sealed class AlarmEvent
{
    public AlarmEventId Id { get; private set; }
    public UnitId UnitId { get; private set; }      // passport to ReactorFleet
    public SignalTag SignalTag { get; private set; } // passport to Instrumentation language
    public AlarmSeverity Severity { get; private set; }
}

// But AlarmEvent should not contain the full Reactor, Plant,
// training policy, root-cause verdict, and reporting projection.
```

HONEST BOUNDARY

A bounded context is a modelling boundary before it is a deployment boundary. You can implement several contexts in one application and one database. You can also split them later. The important thing is that the language and ownership are already clear.

CHAPTER 10 - EXPANDED

Context Maps and Integration Boundaries

Chapter 10 is the right place to connect all Part I terms. A context map shows how bounded contexts relate. The example below starts with one signal crossing a threshold and follows it through alarms, root-cause reasoning, digital-twin evidence, advisory recommendation, repository persistence, domain events, and audit.

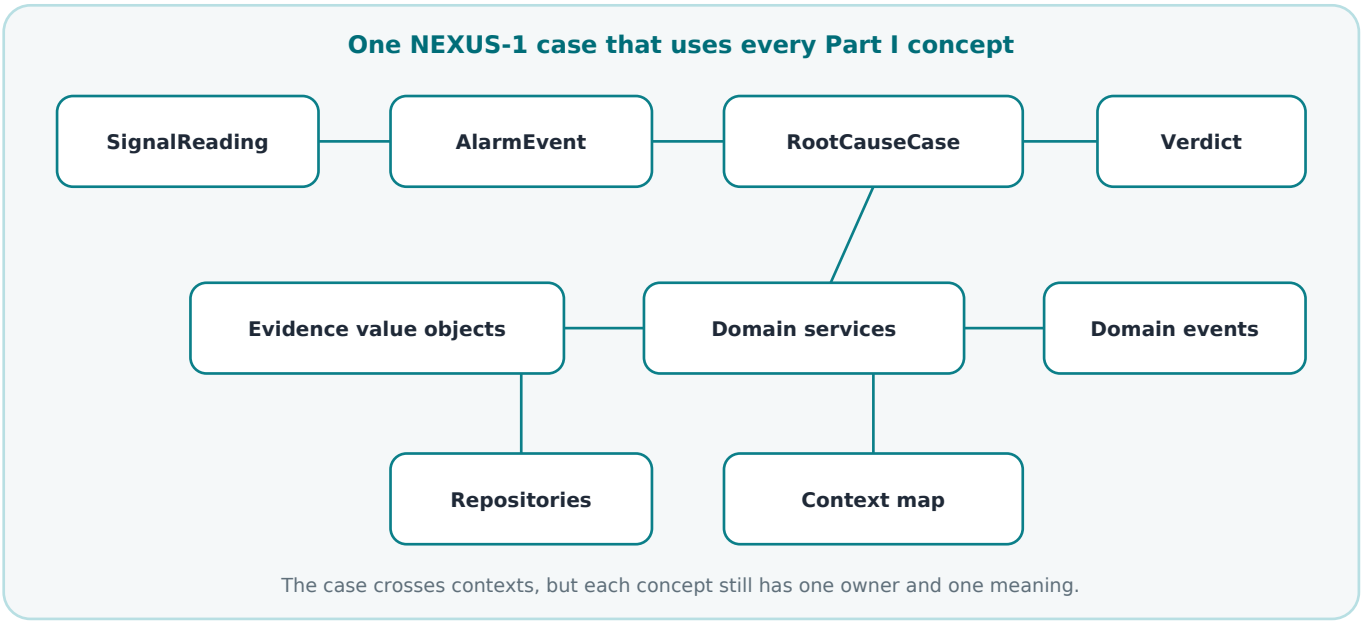


Figure 10.1 - One case study that exercises the Part I vocabulary.

The case: FV-104 actuator latency

A feedwater control valve begins responding slowly. A signal reading crosses a configured threshold. Alarm Management raises an AlarmEvent. Root Cause opens a RootCauseCase. The diagnostic model tests hypotheses. The Digital Twin contributes divergence evidence. The advisory learning context may produce a Recommendation, but not a command. Audit records what happened.

DDD term	Plain meaning	Case example
Domain	The problem area the software is about.	Industrial digital twin and operational reasoning.
Model	Useful simplification of that domain.	Units, signals, alarms, cases, evidence, policies.
Ubiquitous language	Shared controlled words.	AlarmEvent is not root cause; Recommendation is not command.
Entity	Object with identity over time.	Unit NX1-U1, AlarmEvent, RootCauseCase.
Value object	Meaning by value, not identity.	EngineeringValue(323, "C"), SignalTag("FV-104-LAT").
Aggregate root	Object protecting consistency.	RootCauseCase protects hypotheses, evidence, verdict.

DDD term	Plain meaning	Case example
Repository	Collection-like access to aggregate roots.	IRootCauseCaseRepository loads and saves the case.
Domain service	Domain calculation not natural inside one entity.	CausalRankingService ranks candidate causes.
Application service	Use-case coordinator.	OpenRootCauseCaseApplicationService coordinates loading, ranking, saving.
Domain event	Fact that happened.	AlarmFloodDetected, HypothesisRejected, VerdictRecorded.
Bounded context	Boundary where words and rules have one meaning.	Alarm Management, Root Cause, Digital Twin, RL Advisory.
Context map	How contexts relate and exchange facts.	UnitId passport, SignalTag passport, domain events, read models.

One possible flow in code

```
public async Task<RootCauseCaseId> HandleAlarmFlood(AlarmFloodDetected e, CancellationToken ct)
{
    // 1. Application service coordinates the use case.
    var flood = await _alarmFloods.GetByIdAsync(e.AlarmFloodId, ct);

    // 2. Domain service performs domain reasoning.
    var candidates = _causalRanking.RankCandidates(flood);

    // 3. Aggregate root protects consistency.
    var rootCauseCase = RootCauseCase.Open(flood.Id, candidates);

    // 4. Repository persists the aggregate.
    await _rootCauseCases.AddAsync(rootCauseCase, ct);

    // 5. Unit of Work commits and domain events can be dispatched.
    await _unitOfWork.SaveChangesAsync(ct);

    return rootCauseCase.Id;
}
```

Why this example matters

The case is useful because it prevents DDD from staying abstract. Every term has a job. If a term does not change the design, we should not use it. If it does change the design, the reader should be able to point to the code, table, event, boundary, or rule that changed.

COMMON MISTAKE

The bad version of this case is one giant AlarmService that reads every table, decides the root cause, updates the policy, writes audit rows, and returns a dashboard DTO. It works for a demo, but it hides ownership and makes the model fragile.

HONEST BOUNDARY

This example is educational. It does not claim that the Phase-0 browser console is backed by the full database or eventing stack. The purpose is to show the domain design that the final architecture can implement.

Expanded Part I checkpoint

After the expansion, Part I should feel more practical. A mid-level .NET developer should now be able to explain the difference between entity and value object, aggregate root and child entity, repository and DbContext, application service and domain service, command and event, bounded context and project folder, context map and database diagram.

Question	Good answer
Where do business rules live?	As close as possible to the aggregate or domain concept that owns them.
Does every table need a repository?	No. Repositories focus on aggregate roots.
Can an event ask the system to do something?	No. That is a command. An event records what happened.
Is a bounded context a microservice?	Not automatically. It is first a language and model boundary.
Can one case cross many contexts?	Yes, but each context keeps its own ownership and vocabulary.

NEXT STEP

The next build can continue with finalization work: add a proper index, source/reference section, and page-aware contents, or further expand selected NEXUS-1 implementation chapters before final pagination.

Page-Aware Contents

Back matter navigation - final edition

PAGE NUMBER RULE

The page numbers below refer to visible PDF pages in this final edition.

Page-Aware Contents

continued

FINAL NAVIGATION NOTE

This final edition includes the school-style DDD foundation, the NEXUS-1 domain reading, the implementation chapters, expanded Part I explanations, sources, glossary, and index.

BACK MATTER

Sources and Reference Notes

This book is not a theoretical DDD research work. It is a practical learning and architecture companion. The sources below identify the conceptual background that informs the explanations, while the NEXUS-1 companion volumes provide the concrete domain material used throughout the examples.

Source / reference	Why it matters for this book
Eric Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software, Addison-Wesley, 2003.	Foundational source for domain model, ubiquitous language, entities, value objects, aggregates, repositories, services, and strategic design.
Vaughn Vernon, Implementing Domain-Driven Design, Addison-Wesley, 2013.	Practical treatment of aggregates, bounded contexts, context mapping, application services, domain events, and implementation trade-offs.
Vaughn Vernon, Domain-Driven Design Distilled, Addison-Wesley, 2016.	Compact introduction to DDD concepts, useful for the school-style explanatory tone used here.
Martin Fowler, Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.	Repository, Unit of Work, domain model, service layer, and data mapper vocabulary for enterprise developers.
Martin Fowler, public essays on Bounded Context, CQRS, and Unit of Work.	Plain-language architectural framing for context boundaries, read/write separation, and persistence coordination.
Microsoft Learn, .NET architecture and EF Core documentation.	Reference background for DbContext, EF Core mapping, owned types, migrations, and layered .NET implementation.
Microsoft .NET microservices architecture guidance and eShopOnContainers materials.	Practical examples of DDD-inspired .NET architecture, commands, integration events, and bounded contexts.
NEXUS-1 companion volume: From Grid to Core.	The plant/domain background: unit, reactor, signals, operator console, model boundaries, and demonstrator scope.
NEXUS-1 companion volume: From Flood to Cause.	The root-cause background: alarm flood, causal graph, hypotheses, evidence, rejected candidates, and auditability.
NEXUS-1 companion volume: From Trial to Policy.	The advisory-learning background: agent, state, action, reward, Q-table, policy, and recommendation boundaries.
NEXUS-1 companion volume: From Table to Twin.	The data backbone: one platform, SQL schemas as bounded contexts, cross-schema passports, and digital-twin persistence.
NEXUS-1 companion volume: From Entity to Context.	The EF Core mapping discipline: one entity, one configuration, clean DbContext, explicit names, and schema folders.

SOURCE BOUNDARY

The external DDD and .NET sources explain software architecture concepts. The NEXUS-1 companion volumes provide demonstrator examples. None of these sources turns the NEXUS-1 browser console into operational plant software or safety-class guidance.

How the references are used

Topic	How this book uses the source
DDD theory	Used as a teaching foundation, but simplified into human language for mid-level .NET developers.
NEXUS-1 examples	Used as demonstration architecture, not as validated operational engineering or safety-class software.
EF Core and SQL examples	Used to show implementation direction; real production projects still require review, testing, migration discipline, and performance measurement.
Nuclear terminology	Used as part of the NEXUS-1 demonstrator language. It is not a substitute for nuclear engineering training, regulation, or operational qualification.

Recommended reading order

A reader new to DDD can read this book first, then read Vernon Distilled for a compact DDD overview, Evans for the original depth, Fowler for enterprise architecture patterns, and Microsoft guidance for .NET implementation details. A reader focused on NEXUS-1 should read From Grid to Core, From Flood to Cause, From Trial to Policy, From Table to Twin, and From Entity to Context as companion viewpoints on the same platform.

Glossary

DDD and NEXUS-1 terms in plain language

HOW TO USE THIS GLOSSARY

The definitions below are intentionally practical. They explain how each DDD term is used in this book and how it maps to NEXUS-1 examples.

Aggregate	A small group of domain objects that must stay correct together. Outside code changes the group through the aggregate root, not by editing every child object directly.
Aggregate root	The main object that protects an aggregate. In NEXUS-1, RootCauseCase can protect hypotheses, evidence, rejected candidates, and final verdict.
Application service	A use-case coordinator. It loads data, calls domain objects or domain services, saves results, and publishes events. It should not become the place where all business rules live.
Bounded context	A boundary where a model and its language are valid. ReactorFleet.Unit and CorePlatform.EngineeringUnit can both use the word unit safely because the contexts are different.
Command	A request to do something, such as AcknowledgeAlarm or OpenRootCauseCase. A command may fail validation or be rejected by a domain rule.
Context map	A map showing how bounded contexts relate, which context owns which truth, and how facts cross boundaries.
Core domain	The part of the system that gives the project its main value. For NEXUS-1, the digital twin, root-cause reasoning, plant model, and advisory decision layer are close to the core.
Decision	A concluded choice or judgment made by the system or a human. In the DDD framing of NEXUS-1, a decision must not be confused with an advisory recommendation.

Glossary

DDD and NEXUS-1 terms in plain language

Domain	The real problem area the software is about. In this book, the domain is not only screens and tables; it is the industrial digital-twin problem space NEXUS-1 represents.
Domain event	A fact that already happened in the domain, such as AlarmFloodDetected, HypothesisRejected, TwinDivergenceRecorded, or PolicyRecommendationGenerated.
Domain model	A useful simplification of the domain. It keeps the concepts the software needs and leaves out everything that does not support the system purpose.
Domain rule	A rule that belongs to the meaning of the domain, not merely to a form or API. Example: a root-cause case should not be closed without evidence.
Domain service	Domain logic that does not naturally belong inside one entity or value object. Example: a causal ranking service that compares multiple candidate causes.
Entity	An object with identity over time. Its values may change, but it remains the same thing. Unit, AlarmEvent, and RootCauseCase are examples.
Evidence	A grounded fact used to support or reject a hypothesis. In NEXUS-1, evidence can come from alarms, telemetry, twin divergence, records, or audit history.
Generic domain	A supporting capability that many systems need, such as authentication, localization, feature flags, or simple reporting infrastructure.

Glossary

DDD and NEXUS-1 terms in plain language

Hypothesis	A candidate explanation that still needs to be tested. In the Root Cause context, a hypothesis is not a final verdict.
Integration boundary	The line crossed when one context needs facts from another. The crossing should be explicit through identifiers, events, query models, APIs, or adapters.
Passport	The NEXUS-1 metaphor for an explicit cross-context reference, usually a declared identifier such as UnitId or SignalTag.
Persistence ignorance	The design habit of keeping domain objects focused on business meaning rather than EF Core details, SQL table names, or migration mechanics.
Policy	A learned or defined mapping from situation to action. In NEXUS-1 reinforcement learning, a policy is advisory and readable; it is not an automatic command path.
Projection	A read-side representation shaped for queries or screens. It may combine facts from several contexts but should not become the owner of the original truth.
Query	A request to read information. A query should not change domain state.
Read model	A model optimized for reading and reporting. It can flatten or join data for a screen, while aggregates protect the write-side rules.

Glossary

DDD and NEXUS-1 terms in plain language

Recommendation	An advisory output, especially from the reinforcement-learning context. A recommendation is not a command and does not bypass operator approval.
Repository	A collection-like way to load and save aggregate roots. It hides persistence details from the application service and domain model.
RootCauseCase	A NEXUS-1 aggregate root candidate that owns hypotheses, evidence, rejected candidates, reasoning state, and final verdict.
Supporting domain	A domain that matters to the system but is not the main differentiator. Alarm management, instrumentation, maintenance, robotics, and emergency preparedness can support the core.
Twin divergence	A recorded disagreement between the modelled value and measured value. It is evidence or a signal for investigation, not automatically a failure by itself.
Ubiquitous language	The shared controlled vocabulary used by developers and domain readers inside a bounded context.
Unit of Work	A persistence coordination pattern. In EF Core, DbContext often acts as the Unit of Work by tracking changes and committing them together.
Value object	An object defined by its values rather than identity. EngineeringValue, SignalTag, and SafetyBand can be value objects in NEXUS-1.
Verdict	The final conclusion of a diagnostic case. It is stronger than a hypothesis and should be protected by the aggregate rules that produced it.

BACK MATTER

Index

The index emphasizes the DDD concepts and NEXUS-1 terms that a developer is likely to search for while reading or implementing the architecture. Page ranges are compact and point to the main explanation pages.

Index

continued

Final checkpoint

Final edition readiness summary

This edition is finalized as a complete NEXUS-1 DDD companion volume. It includes the main chapters, expanded Part I explanations, page-aware contents, source and reference notes, a glossary, and an index.

Finalization item	Status
Cover	Full-page dark NEXUS-1 cover preserved and updated to final edition.
Contents	Page-aware contents included at the front and repeated in back matter.
References	Sources and reference notes included for DDD, .NET/EF Core, and the NEXUS-1 companion volumes.
Glossary	Practical DDD + NEXUS-1 glossary included on pages 130-133.
Index	Developer-oriented index included after the glossary on pages 134-135.
Pagination	Visible page numbers are stable for this final edition.
Boundary	Educational demonstrator boundary is stated and preserved.

HONEST BOUNDARY

This book remains a learning and architecture companion. It is not safety-class software, not certified operational guidance, and not connected to any real facility.